

The logo of Université Ibn Zohr - Agadir is a circular emblem. It features a dark blue outer ring with the text "UNIVERSITÉ IBN ZOHR - AGADIR" in white capital letters. Inside this ring is a white circle containing the university's name in Arabic calligraphy, "جامعة ابن زهر - آگادير", and a stylized white emblem in the center.

Numéro d'ordre :

Prédiction des Cas de Leishmaniose dans la province de Ouarzazate à l'aide d'un modèle hybride d'apprentissage profond

ANASS EL MANSOURI

PROF. DRISS BOUTAAYAMOU

SOUTENU LE : 30 JUIN 2024

Devant le jury :

PROFESSEUR À LA FACULTÉ POLYDISCIPLINAIRE DE OUARZAZATE
DOCTORANT À LA FACULTÉ POLYDISCIPLINAIRE DE OUARZAZATE

Année Universitaire : 2023-2024

Remerciements

Avant de commencer cet mémoire, je tiens d'abord à remercier Allah pour le courage et la force qu'il m'a donnés pour bien accomplir ce projet.

Je souhaite également adresser mes sincères remerciements à mon encadrent **Prof. Driss BOUTAAYAMO** et au doctorant, ainsi que mon professeur **Mr. Adnane HAKAM**, pour m'avoir donné l'opportunité de travailler sur ce projet passionnant, et pour leurs conseils avisés, leur compréhension et tout ce qu'ils ont fait pour m'aider à accomplir ce travail. Je remercie également les Professeurs **Mr. X** et **Mr. Y** d'avoir accepté d'évaluer mon travail. Leurs retours et recommandations vont être d'une grande valeur pour moi.

Un grand merci à mes parents et à mes frères pour leur soutien indéfectible, leurs encouragements et leurs conseils tout au long de mes études. Leur amour et leur soutien ont été mes plus grandes motivations.

Je tiens également à remercier toute l'équipe pédagogique et tout le corps professoral de la Faculté Polydisciplinaire de Ouarzazate (FPO) pour leurs efforts et pour nous avoir offert un bon environnement d'étude.

Enfin, je tiens à exprimer ma reconnaissance à tous mes amis pour leur soutien et leur compréhension pendant cette période intense. Merci à tous pour votre précieuse contribution à la réussite de ce projet.

Résumé

La leishmaniose est considérée comme l'une des maladies les plus dangereuses de la planète, souvent caractérisée par sa capacité à se propager rapidement au sein des populations. L'impact que cette maladie a eu dans le monde en fait une priorité pour les chercheurs de développer des méthodes sophistiquées qui peuvent aider à modéliser ses incidences et à limiter son danger. Dans la continuité de cette recherche, l'objectif de ce projet était de créer un modèle puissant qui exploite les capacités des modèles d'apprentissage profond pour prédire les cas de leishmaniose dans la province d'Ouarzazate en se basant sur des variables climatiques et des dépendances temporelles. Nous avons ainsi développé un modèle hybride ConvBiLSTM qui combine les avantages des couches convolutionnelles pour extraire des informations complexes avec les capacités des couches BiLSTM pour traiter les séquences temporelles, ajustant après ses hyperparamètres pour maximiser ses performances. Pour comparer ses performances, nous avons développé trois modèles d'apprentissage automatique : SVR, régression polynomiale et un modèle hybride combinant les deux avec CatBoost comme métamodèle. Les modèles ont été entraînés et évalués à l'aide de 12 années de données mensuelles sur la leishmaniose et le climat, de janvier 2010 à décembre 2022. Neuf années de ces données ont été utilisées pour l'entraînement et les 3 années restantes pour les tests. Les résultats indiquent que notre solution ConvBiLSTM surpasse les trois modèles d'apprentissage automatique avec des scores R^2 élevés de 0,94 et 0,89 respectivement pour les ensembles d'entraînement et de test, tout en enregistrant également des scores RMSE faibles de 8,65 et 11,99 respectivement pour les ensembles d'entraînement et de test. Cette solution a été améliorée avec la méthode SHAP pour fournir des interprétations pour les prédictions de ce modèle complexe.

Les mots clés : Leishmaniose, Modélisation des maladies, Prédiction des séries temporelles, Epidémiologie, Apprentissage profond, Réseaux de Neurones Convolutionnels, Réseaux Long Short-term Memory Bidirectionnels, Analyse des données climatiques, SHAP, Interprétabilité des modèles

Table des matières

Introduction Générale	2
1 Etude de l'état de l'art	3
1.1 La leishmaniose	3
1.1.1 Aperçu	3
1.1.2 Facteurs de risque et déterminants de la transmission	4
1.1.2.1 Facteurs biologiques	4
1.1.2.2 Facteurs environnementaux et climatiques	4
1.1.2.3 Facteurs socio-économiques	5
1.1.3 Méthodes de diagnostic de la leishmaniose	5
1.1.3.1 Méthodes directes	5
1.1.3.2 Méthodes indirectes	6
1.1.3.3 Méthodes moléculaires	6
1.1.4 Données et défis en détection et surveillance de la leishmaniose	7
1.1.5 L'évolution des cas de leishmaniose dans la province de Ouarzazate	7
1.1.5.1 Contexte de la leishmaniose dans la province	8
1.1.5.2 Mesures de prévention et de contrôle mis en place dans la province	9
1.2 Apprentissage Profond (Deep Learning)	10
1.2.1 Introduction au apprentissage profond	10
1.2.2 Différentes approches d'apprentissage profond	10
1.2.3 Les Réseaux de Neurones Artificiels (RNA)	13
1.2.4 Les Réseaux de Neurones Convolutifs (RNC)	14
1.2.5 Les Réseaux de Neurones Récurrents (RNR)	17
1.2.5.1 Introduction aux RNR	17
1.2.5.2 Réseaux Long-Short Term Memory (LSTM)	20
1.2.5.3 Réseaux Long-Short Term Memory Bidirectionnels (Bi-LSTM)	22
1.3 Interprétabilité des modèles d'apprentissage automatique et profond	22
1.3.1 Introduction au méthode de SHAP	22
1.3.2 Le fonctionnement de SHAP	23
2 Prédiction des cas des maladies infectieuses : Objectifs, Défis et Solutions Analytiques	25
2.1 Aperçu	25
2.2 Les objectifs de la prédiction des cas des maladies infectieuses	26
2.3 Les défis de la prédiction des cas des maladies infectieuses	26
2.4 Analyse des solutions existantes pour la prédiction des cas des maladies infectieuses	26
2.4.1 Synthèse des solutions existantes	26
2.4.2 Tableau comparatif et Discussion	40
3 Analyse Exploratoire des données	42
3.1 Collecte des données	42
3.2 Prétraitement des données	44
3.3 Exploration Préliminaire des Données	48
3.4 Analyse Univarié des données	49
3.4.1 Analyse du variable dépendante	49
3.4.2 Analyse des variables explicatives	52
3.5 Analyse Multivarié des données	54
3.5.1 Analyse de corrélation	54
3.5.2 Analyse de corrélation croisée	55

4	Implémentation et Résultats de la solution	57
4.1	Environnement du travail	57
4.1.1	Matériels de travail	57
4.1.2	Outils de programmation	57
4.2	Architecture du modèle proposé	60
4.3	Implémentation du structure du système proposée	61
4.3.1	Phase de transformation des données	61
4.3.2	Les métriques d'évaluation des performances du modèle	62
4.3.3	Détermination du nombre optimale des pas de temps (ou des séquences)	63
4.3.4	Détermination du sous-ensemble des variables explicatives optimal	64
4.3.5	Ajustement des hyperparamètres du modèle.	65
4.4	Evaluation, Résultats et Comparaisons	67
4.4.1	Résultats	67
4.4.2	Comparaison avec d'autres modèles	68
4.5	Explications des prédictions du modèle avec SHAP	69
4.5.1	Explications locales des prédictions	69
4.5.2	Explications globales des prédictions	71
	Conclusion générale et perspectives	73

Table des figures

1.1	Lésion cutanée causée par la leishmaniose cutanée.	3
1.2	Lésion cutanée sur le visage	3
1.3	Une Phlébotome	4
1.4	La province de Ouarzazate présenté dans (F.Hajji et al., 2021)	8
1.5	Evolution des cas de leishmaniose dans certains province au Maroc entre 2002 et 2009 présenté dans (F.Hajji et al., 2021)	8
1.6	Evolution des cas de leishmaniose dans la province de Ouarzazate entre 2010 et 2018	9
1.7	Différence entre l'apprentissage automatique et l'apprentissage profond	10
1.8	Le fonctionnemnt d'apprentissage supervisé	10
1.9	Le fonctionnement d'apprentissage non-supervisé	11
1.10	Le fonctionnement d'apprentissage par renforcement	12
1.11	Le fonctionnement d'apprentissage pronfond par transfert	12
1.12	Représentation d'un RNA	13
1.13	Le fonctionnement d'un neurone dans un RNA	13
1.14	Exemple d'un RNC	14
1.15	Une martice d'image 5 x 5 avec une matrice de filtre 3 x 3	15
1.16	Le processus de convolution	15
1.17	La fonction d'activation ReLu	15
1.18	Max Pooling	16
1.19	Un exemple d'une couche entièrement connectée	17
1.20	Différence entre un RNA et un RNR	17
1.21	Un RNR simple déplié (ou déroulé) en un réseau complet	18
1.22	La différence entre RNR et les réseaux LSTM	20
1.23	Une cellule LSTM	21
1.24	Une cellule Bi-LSTM	22
1.25	La méthode de SHAP	23
1.26	La méthode Deep SHAP	24
2.1	Analyse de corrélation croisée mentionnée dans cette étude	27
2.2	Résultats des modèles utilisés dans cette étude	27
2.3	Les variables les plus importants dans le modèle du cette étude	27
2.4	Les données utilisés dans cette étude	28
2.5	Le schéma de la phase de modélisation utilisé dans l'étude	28
2.6	Les résultats des modèles utilisés dans cette étude. (a) Les dix meilleurs modèles d'apprentissage profond et les modèles d'analyse traditionnels avec les RMSE les plus bas pour la varicelle, (b) Les dix meilleurs modèles d'apprentissage profond et les modèles d'analyse traditionnels avec les RMSE les plus bas pour la scarlatine, (c) Les dix meilleurs modèles d'apprentissage profond et les modèles d'analyse traditionnels avec les RMSE les plus bas pour le paludisme.	29
2.7	L'architecture du modèle utilisé dans cette étude	30
2.8	Les résultats du modèle utilisé dans cette étude	30
2.9	Les résultats de l'analyse de la multicollinéarité utilisé dans cette étude	31
2.10	Les résultats de l'analyse d'autocorrélation spatiale utilisé dans cette étude	31
2.11	Les résultats des modèles utilisés dans cette étude	32
2.12	L'évaluation des cartes de risque de leishmaniose créés par les modèles utilisés dans cette étude	32
2.13	Les données utilisés dans cette étude	33
2.14	Les résultats des modèles utilisés dans cette étude	33
2.15	La matrice de corrélation pour la districte de Muzaffapur mentionnée dans cette étude	34
2.16	La matrice de corrélation pour la districte de Saran mentionnée dans cette étude	35
2.17	Les résultats des modèles utilisés dans cette étude	35
2.18	L'architecture du modèle proposé dans cette étude	36

2.19	Les configurations des modules utilisées dans l'architecture proposée	36
2.20	Les configurations des modèles de références utilisées dans cette étude	37
2.21	la performance du modèle hybride CNN-BiLSTM pour la prévision à une semaine	37
2.22	la performance du modèle hybride CNN-BiLSTM pour la prévision à quatre semaine	38
2.23	A. le format des données utilisés pour l'entraînement des modèles mutivariés muti-étapes LSTM, B. L'architecture du modèle hybride multivarié muti-étapes ARIMA-LSTM	39
2.24	Les résultats des modèles utilisés dans cette étude	39
3.1	Les données cliniques obtenus au sein d'hôpital Sidi Hsein	42
3.2	Exemple des données climatiques obtenue depuis NASA POWER de la commune de Taznakhte .	43
3.3	Chargement du jeu des données cliniques en sélectionnant les variables importantes	44
3.4	Exemple d'une valeur erronée dans la date de diagnostic	44
3.5	Transformation du variable de date de diagnostic en type <i>Datetime</i>	45
3.6	Extraction du mois depuis la variable de date de diagnostic de type <i>Datetime</i>	45
3.7	Aggrégation des cas de leishmaniose par mois et par année	45
3.8	Création du variable <i>Date</i> pour l'utiliser comme l'index de notre jeu des données	46
3.9	Code de préparation du matrice des variables climatiques mensuelles de chaque commune dans la province de Ouarzazate	46
3.10	Code de transformation du matrice des variables climatiques mensuelles de chaque commune dans la province de Ouarzazate	47
3.11	Code de préparation du matrice finale des données climatiques mensuelles de la province de Ouarazazate	47
3.12	Présentation des données	48
3.13	Quelques statistiques descriptives de nos données	48
3.14	La distribution des cas de leishmaniose de 2010 à 2022 dans la province de Ouarzazate	49
3.15	Distribution des cas de leishmaniose par mois	49
3.16	Distribution des cas de leishmaniose par année	50
3.17	Distribution des cas de leishmaniose par année et par saison	50
3.18	Le résultat du test de stationarité ADF	51
3.19	Le résultat du test ADF après différenciation	51
3.20	Les graphiques ACF et PACF	52
3.21	Les distributions de chaque variable explicative	52
3.22	Les dépendances temporelles des variables explicatives	53
3.23	Les nuages de points entre les variables explicatives et les cas de leishmaniose	54
3.24	la matrice des corrélation entre les variables climatiques et les cas de leishmaniose	55
3.25	Les résultats de la corrélation croisée	56
4.1	Google Colaboratory	57
4.2	Le langage de programmation Python	58
4.3	la bibliothèque Pandas	58
4.4	la bibliothèque NumPy	58
4.5	la bibliothèque Matplotlib	58
4.6	la bibliothèque Scikit-learn	59
4.7	la bibliothèque TensorFlow	59
4.8	la bibliothèque Keras	59
4.9	la bibliothèque CatBoost	59
4.10	L'architecture globale du système proposé	60
4.11	Fonction de transformation du DataFrame en matrices X et y pour l'entraînement de notre modèle.	61
4.12	Le résultat d'application de la fonction <i>df_to_X_y</i> sur notre matrice des données	61
4.13	Fonction de division de nos matrices X et y en données d'entraînement et de validation	62
4.14	Fonction de détermination du nombre optimale de pas de temps sur la base des données de validation	63
4.15	Le nombre optimale de pas de temps obtenu par la fonction	63
4.16	La méthode utilisé pour déterminer les variables explicatives qui donnent la performance optimale du modèle sur les données de validation	64
4.17	Le code qui implémente la méthode d'ajustement des hyperparamètres	66
4.18	Les résultats de GridSearchCV	66
4.19	L'architecture finale de notre modèle	67
4.20	La courbe d'apprentissage du modèle	67
4.21	Les prédictions de modèle sur les données d'entraînement et du test comparé avec les cas de leishmaniose réels	68

4.22	Les impacts des pas de temps de chaque variable climatique sur la prédiction (Partie 1)		70
4.23	Les impacts des pas de temps de chaque variable climatique sur la prédiction (Partie 2)		70
4.24	Les impacts des variables explicatives sur les prédictions du modèle ConvBiLSTM pour la période allant de janvier 2018 à décembre 2022.		71

Introduction Générale

Au cours des dernières décennies, la maladie de la leishmaniose a continué de susciter de grandes préoccupations à travers le monde en raison de sa propagation rapide dans de nouvelles régions et de son impact sur les populations pauvres. Dans ce contexte, elle est devenue une priorité pour les autorités de santé publique et les chercheurs, qui cherchent à comprendre ses facteurs de risque, à identifier les zones du monde menacées et à mettre en place des stratégies efficaces pour sa prévention et son contrôle.

Cette maladie exerce un impact significatif sur les villes et villages vulnérables tels que la ville d'Ouarzazate et ses villages environnants, car leurs conditions climatiques et leurs caractéristiques géographiques les rendent propices à la propagation de cette maladie. Pour faire face à ces défis, une meilleure compréhension de la dynamique de transmission de la maladie dans la province d'Ouarzazate est essentielle pour établir des initiatives de prévention et de contrôle, mais aussi pour faciliter l'accès aux hôpitaux pour sa population et sensibiliser cette dernière afin de lutter contre cette maladie infectieuse.

Les techniques traditionnelles d'apprentissage automatique (la régression logistique, les forêts aléatoires et la machine à vecteurs de support) ne parviennent pas à apprendre des modèles séquentiels de données pour la prédiction des cas de leishmaniose. Elles ne sont pas assez développées pour répondre aux problèmes complexes du monde réel. Contrairement aux techniques citées auparavant, les modèles d'apprentissage profond (Deep Learning) donnent une meilleure précision et plus de satisfaction, ce qui fait l'objet des études approfondies dans divers domaines de la science des données tels que le traitement naturel du langage (NLP), la classification d'images et la prédiction sur la base des données séquentiels.

Pour répondre au problème de prédiction des cas de leishmaniose dans la province de Ouarzazate, nous avons conçu et implémenté un modèle de prédiction basé sur les réseaux de neurones récurrents, notamment les réseaux de neurones à mémoire longue et courte terme bidirectionnels (BiLSTM) ainsi que les réseaux convolutionnels, entraîné en utilisant plusieurs variables climatiques. Cette approche combine deux des modèles les plus performants pour ce type de problème, en plus de la méthode SHAP pour améliorer son interprétabilité et fournir des explications simples pour ses prédictions. Comme mentionné dans le chapitre 4, notre solution, ConvBiLSTM, surpasse tous les autres modèles couramment utilisés pour la prédiction des cas de leishmaniose, en raison de son architecture qui s'adapte parfaitement à la nature séquentielle des données de notre problème. Ce modèle a été amélioré en ajustant ses paramètres à l'aide d'algorithmes tels que la validation croisée (cross-validation) et la recherche sur grille (grid search).

Le présent rapport de mémoire doit être bien structuré pour être exploité après la mise en place de la solution, et nous l'avons organisé de la manière suivante :

- * Dans le premier chapitre, nous avons effectué une analyse approfondie sur la leishmaniose afin de dévoiler les facteurs de sa transmission, les sources de données disponibles et les mesures de preventions utilisés par le royaume du maroc pour répondre au problème de propagation de cette maladie. Ensuite, nous avons fourni une introduction à l'apprentissage profond, aux réseaux de neurones convolutionnels ainsi qu'aux réseaux de neurones récurrents, en particulier les réseaux à mémoire longue et courte unidirectionnels et bidirectionnels. Nous avons également abordé l'interprétabilité des modèles avec la méthode SHAP.
- * Dans le deuxième chapitre, nous avons étudié le sujet de la prédiction des cas de leishmaniose en présentant les objectifs de cette recherche, les défis rencontrés ainsi qu'une étude des solutions existantes pour ce problème.
- * Dans le troisième chapitre, nous avons réalisé une analyse exploratoire des données en détaillant le processus de collecte et de prétraitement des données, ainsi que les analyses univariées et multivariées des données.
- * Dans le quatrième et dernier chapitre, nous avons détaillé le processus d'implémentation de notre solution, ainsi que les explications globales et locales de nos prédictions.

Chapitre 1

Etude de l'état de l'art

Introduction

Dans ce chapitre, nous menons une étude approfondie sur la leishmaniose afin d'identifier les éléments importants dans la prédiction de ses cas, tels que ses facteurs de risque, les sources de ses données, les méthodes utilisées pour son diagnostic, ainsi que l'évolution de ses cas dans la province de ouarzazate. Par la suite, nous fournissons une introduction à l'apprentissage profond en mettant l'accent sur les réseaux de neurones convolutifs, les réseaux de neurones récurrents, en particuliers les réseaux de mémoire à long terme unidirectionnels et bidirectionnels.

1.1 La leishmaniose

1.1.1 Aperçu

La leishmaniose (Leishmaniasis en anglais) est une maladie parasitaire qui affecte principalement les régions tropicales et subtropicales, mais elle peut également être présente dans des régions tempérées. Elle est causée par différents parasites du genre *Leishmania* et peut provoquer des affections cutanées ou viscérales très graves, même mortelles dans certains cas.

La leishmaniose peut se présenter sous plusieurs formes, chacune étant caractérisée par sa nature :

- **Leishmaniose cutanée** : C'est la forme la plus fréquente de la leishmaniose, elle se caractérise par des lésions ulcérées (parfois nombreuses) qui peuvent apparaître sur des parties exposées du corps et laisser des cicatrices permanentes.
- **Leishmaniose muco-cutanée** : Cette forme se caractérise par des lésions cutanées, souvent localisées sur le visage, qui peuvent entraîner la destruction totale ou partielle des muqueuses du nez, de la bouche et de la gorge.
- **Leishmaniose viscérale (kala-azar)** : Considérée comme la forme la plus sévère et mortelle de la leishmaniose dans la plupart des cas si elle n'est pas traitée, la leishmaniose viscérale affecte les organes internes tels que le foie, la rate et la moelle osseuse. Elle entraîne souvent une perte de poids, des poussées irrégulières de fièvre et une anémie.



FIGURE 1.1 – Lésion cutanée causée par la leishmaniose cutanée.



FIGURE 1.2 – Lésion cutanée sur le visage

1.1.2 Facteurs de risque et déterminants de la transmission

La transmission de la leishmaniose est un processus complexe influencé par plusieurs facteurs environnementaux, climatiques, biologiques et socio-économiques. C'est pour cela qu'il est essentiel de bien comprendre ces facteurs afin d'évaluer le risque de transmission de cette maladie, de identifier les populations les plus exposées et de développer des stratégies de prévention et de contrôle efficaces.

1.1.2.1 Facteurs biologiques

Selon une étude menée sur l'extension de *Leishmania major* au nord de l'Algérie, la leishmaniose est particulièrement causée par trois parasites différents : *Leishmania major* MON-25, *Leishmania infantum* MON-24 et *Leishmania killicki* qui sont transmis par les piqûres de phlébotomes, des insectes vecteurs de la maladie (Bull. Soc. Pathol. Exot. 105 (2012)).

Les phlébotomes sont des petits insectes qui se nourrissent de sang et qui sont très importants dans la propagation de la leishmaniose. Ils passent par plusieurs étapes de développement, comme les larves, les nymphes et les adultes. Les femelles ont besoin de sang pour que leurs œufs se développent, donc elles piquent des mammifères, plus souvent les humains, et c'est ainsi qu'elles transmettent les parasites responsables de la leishmaniose (Killick-Kendrick R.). Le cycle de vie des parasites de la leishmaniose se compose de deux étapes : d'abord, ils vivent à l'extérieur des cellules dans l'insecte vecteur, le phlébotome. Ensuite, après que le phlébotome pique les mammifères, ils sont transférés à l'intérieur des cellules des mammifères, ce qui permet la transmission de la maladie. Après, les parasites se multiplient dans certaines cellules du sang des mammifères, ce qui provoque différents symptômes de la maladie. Selon une étude récente, les parasites de la leishmaniose peuvent aussi se transmettre lors de l'accouplement des phlébotomes mâles, ce qui pose des questions sur la propagation de la maladie dans les régions où elle est fréquente (Alvar J, Vélez ID, Bern C, et al.).



FIGURE 1.3 – Une Phlébotome

1.1.2.2 Facteurs environnementaux et climatiques

Leishmania major MON-25, *Leishmania infantum* MON-24 et *Leishmania killicki* vivent dans des conditions climatiques spécifiques, telles que celles observées avec *Leishmania major*, qui est plus actif dans les régions sahariennes en raison de **la désertification**. La désertification peut entraîner des changements dans l'environnement, tels que la perte de végétation et la modification des habitats naturels, ce qui peut influencer la distribution des vecteurs de la maladie, tels que les phlébotomes (Bull. Soc. Pathol. Exot. 105 (2012)).

La température peut également être considérée comme un facteur climatique important dans la transmission de la leishmaniose. En effet, dans de nombreux cas, la leishmaniose est transmise aux humains par certains types de phlébotomes très actifs dans les régions où les températures sont élevées. Cela a été mentionné dans une étude sur l'évolution de la leishmaniose cutanée en Algérie, qui a confirmé que les paramètres contribuant

le plus à l'évolution de la maladie sont les températures moyennes élevées qui peuvent créer des conditions favorables au développement des phlébotomes (M. Taleb et S. Bradai (2014)).

Cependant, la relation entre la température et la transmission de la leishmaniose peut être conditionnée par d'autres facteurs climatiques importants tels que **l'humidité**. Dans certaines régions, une température élevée combinée à une forte humidité peut créer des conditions idéales pour que les phlébotomes vivent et se reproduisent, augmentant ainsi le risque de transmission de la maladie. Cela peut ne pas être le cas dans d'autres régions, car une humidité excessive ou insuffisante peut également affecter la survie et l'activité des phlébotomes, réduisant ainsi la transmission de la leishmaniose. Cela a été mentionné dans une étude sur l'impact du climat dans les régions humides en Algérie, montrant que la période de faible incidence de la maladie coïncide avec une sécheresse de mai à novembre et des conditions humides de décembre à mars, confirmant l'impact de l'humidité mais aussi **des précipitations** sur la transmission de la leishmaniose (Houti et al. 2016).

L'urbanisation est également devenue un facteur environnemental majeur dans la propagation de la leishmaniose ces dernières années. En effet, dans les pays en développement, les grandes villes se développent rapidement mais ont souvent des logements et des systèmes d'assainissement insuffisants. Cela facilite la vie des phlébotomes à l'intérieur de ces maisons et la propagation de maladies telles que la leishmaniose (Organisation mondiale de la santé. (2002). Urbanisation : un facteur de risque croissant pour la leishmaniose).

1.1.2.3 Facteurs socio-économiques

Les conditions socio-économiques des personnes influencent à la fois les conditions environnementales recherchées par les vecteurs de maladies et les comportements individuels qui peuvent augmenter le risque de contraction. En fait, selon une étude menée sur la leishmaniose viscérale dans le sud-est du Népal et ses facteurs de risque, **les conditions de logement** telles que la proximité de la maison aux étangs, les maisons construites avec des sols en terre, et la taille des familles nombreuses (6 membres ou plus) sont des facteurs très importants dans la transmission de la leishmaniose. Cela peut s'expliquer par le fait qu'à travers le séchage et l'érosion, des fissures et des crevasses apparaissent dans les murs en terre et peuvent servir de sites de reproduction pour les phlébotomes femelles. De plus, la transmission de la maladie pourrait être plus facile dans les familles nombreuses.

Les activités d'**agriculture vivrière**, y compris l'élevage d'animaux, est considérée dans de nombreux cas comme un facteur socio-économique important dans la transmission de la maladie. Selon la même étude mentionnée ci-dessus, la présence de petits animaux tels que les poulets et les chèvres peut attirer les phlébotomes en raison de leur chaleur corporelle et de l'odeur de leur peau. De plus, les poulaillers et les enclos de chèvres peuvent fournir des endroits sombres et humides qui favorisent la reproduction des phlébotomes.

Le niveau d'éducation peut également influencer la transmission de la leishmaniose. Par exemple, les individus ayant un faible niveau d'éducation peuvent être moins informés des risques de piqûres de phlébotomes et des mesures préventives à prendre, ce qui peut les rendre plus susceptibles de contracter facilement la maladie.

1.1.3 Méthodes de diagnostic de la leishmaniose

Identifier les cas d'infection précocement et avec précision peut être un aspect crucial de la lutte contre la leishmaniose. Cela peut être réalisé en utilisant plusieurs méthodes de diagnostic qui ont été développées au fil des ans, chacune ayant ses propres avantages et limitations. Ces méthodes peuvent être regroupées en trois catégories principales : **les méthodes directes**, **les méthodes indirectes** et **les méthodes moléculaires**.

1.1.3.1 Méthodes directes

Les méthodes directes du diagnostic des cas de leishmaniose servent à détecter directement la parasite *Leishmania* ou de ses produits dans les échantillons biologiques du patient. Ils permettent de confirmer l'infection et identifier le type de *Leishmania* présent. Il y'a plusieurs méthodes directes utilisés, mais les plus communs sont :

1. **Microscopie** : C'est une méthode traditionnelle qui implique l'observation des parasites dans les échantillons biologiques des patients. Par exemple, dans des cas de la leishmaniose viscérale, on peut observer les amastigotes (forme intracellulaire du parasite) dans les macrophages des tissus (comme la moelle osseuse ou la rate) à l'aide d'un microscope. Pour la leishmaniose cutanée, les amastigotes peuvent être observés dans les frottis de la lésion cutanée.
2. **Culture** : La culture consiste à faire croître les parasites prélevés sur le patient dans un milieu de culture approprié. Cette approche permet de confirmer l'infection en observant la croissance des parasites, tout en permettant également d'identifier le type de *Leishmania* présent.
3. **Biopsie** : Elle implique le prélèvement d'un petit échantillon de tissu infecté en vue d'un examen microscopique ou d'une culture ultérieure. Les sites de prélèvement peuvent inclure la moelle osseuse, la

peau ou les ganglions lymphatiques. Cette méthode est souvent réservée aux cas où d'autres méthodes de diagnostic n'ont pas permis d'obtenir des résultats concluants.

Bien que ces méthodes de diagnostic soient largement utilisées et puissent être très efficaces, elles présentent des défauts dans certains aspects. En fait, les méthodes directes peuvent être sensibles, mais leur sensibilité peut varier en fonction du stade de la maladie et de la qualité de l'échantillon prélevé. De plus, l'interprétation des résultats des méthodes directes nécessite souvent des compétences spécialisées, ce qui peut limiter leur utilisation dans les régions où le personnel qualifié est limité.

1.1.3.2 Méthodes indirectes

Les méthodes indirectes de diagnostic de la leishmaniose ne détectent pas directement le parasite lui-même, mais plutôt les anticorps ou les cellules immunitaires spécifiques générés en réponse à l'infection. C'est-à-dire, Ils permettent la détection de la réponse immunitaire de l'organisme à l'infection par *Leishmania*. Parmi les méthodes indirectes les plus connues on trouve :

1. **Sérologie** : Elle consiste à rechercher la présence d'anticorps dirigés contre les antigènes de *Leishmania* dans le sérum du patient. Les tests les plus couramment utilisés sont le test d'agglutination directe (DAT), le test d'immunofluorescence indirecte (IFI) et les tests ELISA. Ces tests sont utiles pour confirmer une infection passée ou en cours, mais ils ne permettent pas de distinguer entre les différentes formes de leishmaniose.
2. **Tests de stimulation lymphocytaire** : Ce sont des tests qui évaluent la réponse des lymphocytes du patient à des antigènes spécifiques de *Leishmania*. Par exemple, le test de transformation lymphoblastique (LTT), qui mesure la prolifération des lymphocytes en réponse à la stimulation par des antigènes de *Leishmania*. Ces tests peuvent être utiles pour évaluer l'activité de l'infection et la réponse au traitement.
3. **Tests de détection d'antigènes** : Ces tests sont moins courants, mais certains d'entre eux peuvent détecter directement des antigènes de *Leishmania* dans les échantillons biologiques du patient. Par exemple, le test de détection d'antigènes solubles (rK39) est utilisé pour détecter les antigènes de *Leishmania* dans le sérum des patients atteints de leishmaniose viscérale.

Ces méthodes de diagnostic indirect de la leishmaniose sont souvent utilisées en association avec des méthodes directes pour obtenir un diagnostic plus complet. Elles peuvent être très utiles dans les cas où les méthodes directes ne fournissent pas de résultats concluants ou lorsque la détection du parasite est difficile. Cependant, elles présentent également certains inconvénients. Par exemple, leur sensibilité et leur spécificité peuvent varier en fonction du type d'antigène utilisé et de la population étudiée, ce qui peut entraîner des résultats faux positifs ou faux négatifs. De plus, les résultats des tests sérologiques doivent, dans certains cas, être confirmés par d'autres méthodes, ce qui peut augmenter le coût et le temps nécessaires au diagnostic.

1.1.3.3 Méthodes moléculaires

Les méthodes moléculaires de diagnostic de la leishmaniose sont basées sur la détection de l'ADN du parasite *Leishmania* dans les échantillons biologiques du patient. Elles sont hautement sensibles et spécifiques, ce qui les rend très utiles pour confirmer l'infection et identifier le type de *Leishmania* présent. Les principales méthodes moléculaires utilisées sont :

1. **PCR (Réaction de Polymérisation en Chaîne)** : La PCR est une technique qui permet d'amplifier de manière spécifique de petits morceaux d'ADN. Dans le cas de la leishmaniose, Elle peut être utilisée pour détecter l'ADN de *Leishmania* dans des échantillons biologiques tels que le sang, la moelle osseuse ou les tissus infectés.
2. **Q-PCR (PCR quantitative)** : C'est une variante de la PCR qui permet de détecter l'ADN du parasite en plus de la quantification de la charge parasitaire dans l'échantillon. Cette méthode peut être utile pour suivre la progression de l'infection et évaluer l'efficacité du traitement.
3. **LAMP (Loop-mediated isothermal amplification)** : Le LAMP est une technique prometteuse d'amplification de l'ADN qui ne nécessite pas de cycles de température complexes comme la PCR. Elle est donc plus facile à mettre en œuvre sur le terrain, c'est une méthode qui facilite le diagnostic de la leishmaniose dans les régions où les ressources sont limitées.

Les méthodes de diagnostic moléculaire de la leishmaniose présentent plusieurs avantages, notamment leur sensibilité et leur spécificité élevées, ainsi que leur capacité à détecter l'ADN du parasite dans les stades précoces de l'infection. Cependant, elles nécessitent des équipements spécialisés et une expertise technique, ce qui limite leur utilisation dans les régions à ressources limitées.

1.1.4 Données et défis en détection et surveillance de la leishmaniose

Pour vaincre cette maladie infectieuse et limiter ses effets, il est nécessaire d'analyser ses facteurs de risque et la dynamique de sa transmission ainsi que prédire l'évolution de ses cas afin d'identifier les zones à risque et de contrôler la situation. Pour ce faire, une grande quantité de données sur les cas de leishmaniose est requise et elle doit contenir des informations suffisantes et des motifs pertinents. Ces données peuvent provenir de différentes sources telles que les hôpitaux et les structures de santé publique, et peuvent être variées en termes d'informations fournies.

Voici une liste des types des données concernant la leishmaniose qu'on peut avoir ainsi que leur sources :

- * **Données épidémiologiques :** Les données épidémiologiques contiennent souvent les informations sur l'incidence de la maladie, les cas signalés, les tendances temporelles et géographiques, ainsi que les groupes de population les plus touchés, ils sont fournis par des services de santé locaux, nationaux et internationaux sous forme de rapports.
- * **Données cliniques :** Ces données viennent souvent des cliniques et des hôpitaux sous forme de dossiers médicaux des patients atteints de leishmaniose fournissent des informations sur les symptômes, les traitements et les résultats. Ils aident ainsi à mieux comprendre la gravité de la maladie et son impact sur la santé publique.
- * **Données entomologiques :** Ces données comportent principalement des informations sur les vecteurs de la leishmaniose, tels que les phlébotomes, comme leur répartition géographique et leur densité, ce qui est essentiel pour la surveillance et la prévention de la maladie. Ils peuvent être collectés à travers des études entomologiques sur les zones où la leishmaniose est endémique.
- * **Données environnementales :** Ce sont des informations sur les facteurs environnementaux et climatiques, tels que l'humidité et la température, ainsi que les changements environnementaux liés à l'activité humaine qui peuvent influencer la distribution de la leishmaniose. Ils viennent des organisations qui fournissent des données climatiques pour différentes zones ou bien à travers l'imagerie satellite utilisée pour surveiller les changements environnementaux.
- * **Données sociodémographiques :** Ce sont les données sur la population, telles que la densité de population, les conditions socioéconomiques et les pratiques de soins de santé qui peuvent aider à identifier les populations les plus à risque et mettre en œuvre les efforts de prévention et de contrôle. Ces données peuvent être collectées à travers des enquêtes sur terrain dans les zones touchées.
- * **Données de surveillance vétérinaire :** Étant donné que la leishmaniose peut affecter les animaux, en particulier les chiens, la surveillance vétérinaire peut fournir des informations collectées depuis les cliniques vétérinaires sur la prévalence de l'infection chez les animaux et servir d'indicateur de risque pour les humains.

Parfois, la collecte de ces données pour l'analyse et pour la prédiction des cas de la leishmaniose présente beaucoup de défis, y compris :

- * **Biais de notification :** Dans plusieurs cas, les données peuvent être biaisées en raison de la mauvaise qualité des rapports, car certains cas peuvent ne pas être pris en compte en raison de divers problèmes, tels que l'accès limité aux services de santé ou la mauvaise communication entre les parties prenantes.
- * **Lacunes dans la couverture géographique :** Les données peuvent également être incomplètes en raison d'une couverture géographique mal conduite. Dans la plupart des cas, les zones rurales ou isolées ne sont pas bien couvertes par les systèmes de surveillance, ce qui entraîne un manque de données représentatives.
- * **Problèmes de qualité des données :** Les données peuvent être de mauvaise qualité en raison d'erreurs de saisie, d'erreurs de classification des cas ou d'incohérences dans les rapports.

1.1.5 L'évolution des cas de leishmaniose dans la province de Ouarzazate

La province de Ouarzazate, située au Sud-est du Maroc, s'étend sur une superficie de 19 464 km². Elle appartient à une région pré-saharienne et elle est délimitée par les provinces du Haouz et d'Azilal au Nord, par la province d'Errachidia à l'Est, par la province de Tata à l'Ouest et au Sud par la province de Zagora (Figure 1.4).

Le climat de la province d'Ouarzazate est semi-désertique avec une forte influence continentale. Les précipitations annuelles moyennes sont très faibles, atteignant seulement 150 mm. Les températures varient considérablement du Nord au Sud de la zone : en hiver, elles oscillent entre 0,5°C et 20°C, tandis qu'en été, elles se situent entre 30°C et 40°C. Les vents dominants sont généralement de Nord-ouest, apportant parfois le Sirocco ou le Chergui. La population de la province de Ouarzazate s'élève à 439 072 habitants, répartis en 61 517 ménages, sur une superficie de 19 464 km². La majorité de la population, soit 75%, vit en milieu rural. En ce qui concerne l'activité économique, 61% de la population active est employée dans le secteur agricole.

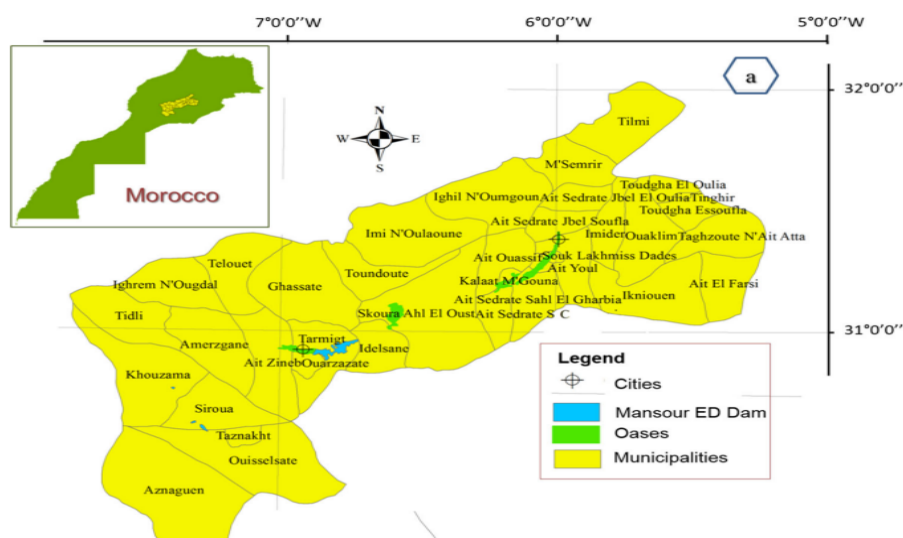


FIGURE 1.4 – La province de Ouarzazate présenté dans (F.Hajji et al., 2021)

1.1.5.1 Contexte de la leishmaniose dans la province

L'Afrique du Nord est l'une des régions les plus touchées par la leishmaniose, et le Maroc figure parmi les pays présentant une forte incidence de la maladie. La province d'Ouarzazate a connu une évolution significative des cas de leishmaniose entre 2002 et 2018. En 2002, le nombre de cas était inférieur à 500, indiquant une incidence relativement faible de la maladie. Cependant, en 2003, la province a connu une forte augmentation des cas, atteignant 2064 cas, marquant ainsi un pic significatif. En 2004, le nombre de cas est passé à moins de 1000, ce qui peut s'expliquer par une réponse efficace des autorités sanitaires locales pour contenir la maladie. Les années suivantes ont été marquées par une diminution progressive du nombre de cas, avec moins de 1000 cas en 2005, moins de 400 cas en 2006, moins de 300 en 2007, et une légère augmentation en 2008 avec plus de 300 mais moins de 500 cas. En 2009, le nombre de cas est remonté à 500, signalant une possible reprise de l'incidence de la maladie après une période de déclin (Figure 1.5).

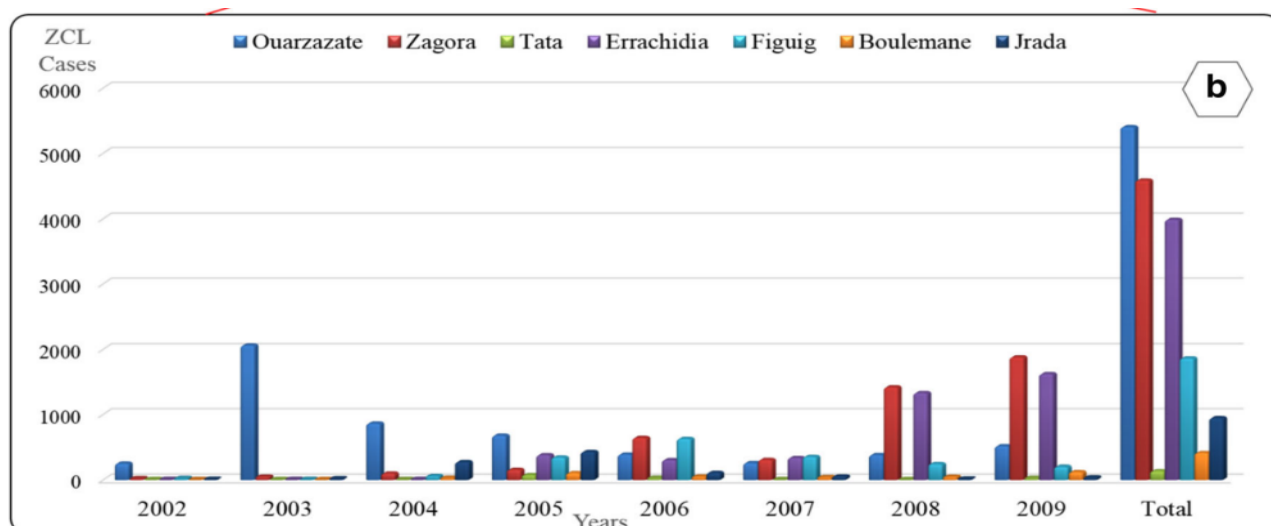


FIGURE 1.5 – Evolution des cas de leishmaniose dans certains province au Maroc entre 2002 et 2009 présenté dans (F.Hajji et al., 2021)

L'évolution des cas de leishmaniose dans la province de Ouarzazate entre 2002 et 2009 a été suivie par une période de fluctuations plus modérées entre 2010 et 2018. En 2010, il y a eu 337 cas, un peu moins qu'en 2009. En 2011, il y en a eu 297, et en 2012, seulement 129. Les années suivantes ont vu une augmentation progressive, mais le nombre est resté relativement bas par rapport aux années précédentes. En 2013, il y a eu 87 cas, en 2014, 254, en 2015, 399, et en 2016, 434. En 2017, il y a eu une légère baisse à 347 cas, mais en 2018, il y en a eu 602, une augmentation significative par rapport aux années précédentes (Figure 1.6).

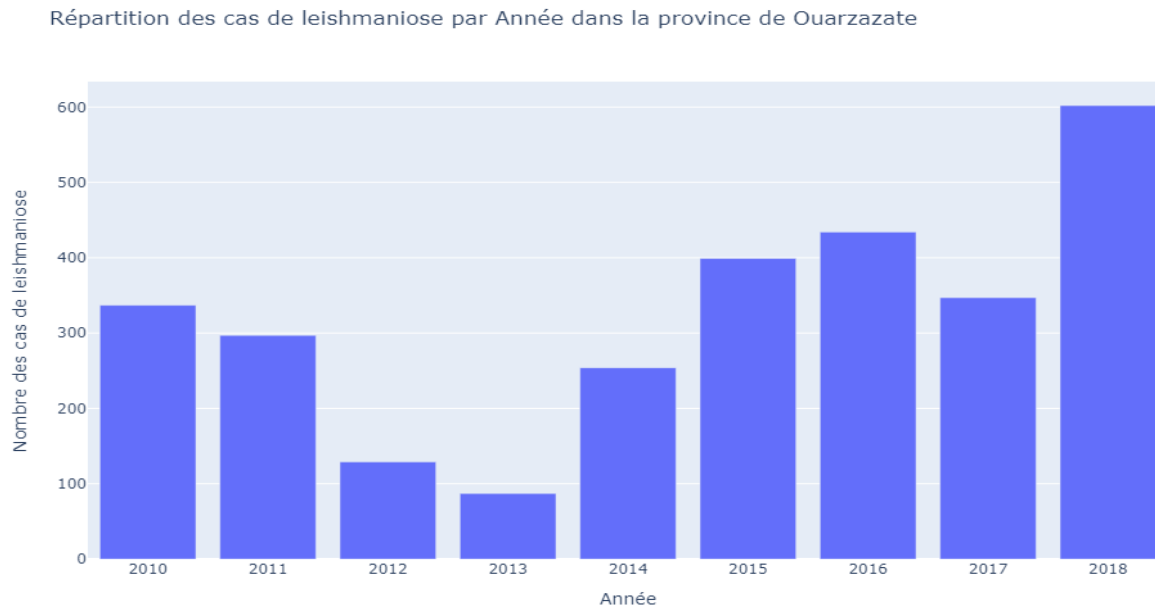


FIGURE 1.6 – Evolution des cas de leishmaniose dans la province de Ouarzazate entre 2010 et 2018

Comme expliqué dans la section des facteurs de risques de la leishmaniose, l'évolution des cas de leishmaniose dans la province de Ouarzazate peut être aussi expliqué par des facteurs climatiques, environnementaux et socioéconomiques. La province a connue des changements climatiques, caractérisés par une augmentation des températures et des variations dans les précipitations dans cette période, qui ont eu un impact direct sur la distribution des vecteurs de la leishmaniose, tels que les phlébotomes. De plus, elle a aussi connue des changements environnementaux liés à l'urbanisation et à la déforestation qui ont modifiés les habitats naturels des phlébotomes et des réservoirs de la maladie, favorisant leur proximité avec les populations humaines.

1.1.5.2 Mesures de prévention et de contrôle mis en place dans la province

Le Royaume du Maroc prend plusieurs mesures pour lutter contre la maladie de la leishmaniose, en commençant par contrôler les insectes qui propagent la maladie. Une des principales actions entreprises à cet égard est de distribuer des moustiquaires traitées avec un insecticide spécial aux communautés à risque, surtout dans les zones rurales où la maladie est plus courante, comme la province de Ouarzazate.

De plus, des insecticides sont pulvérisés à l'intérieur des maisons à Ouarzazate et dans d'autres zones où la maladie est répandue afin de réduire le nombre de phlébotomes. Cette méthode vise à limiter la propagation du parasite en réduisant le nombre de vecteurs à proximité des habitations.

Outre le contrôle des insectes vecteurs, le gouvernement marocain œuvre à sensibiliser la population à la leishmaniose, ce qui implique d'éduquer les communautés sur l'importance de la protection individuelle. Par exemple, leur enseigner l'importance de porter des vêtements longs et d'utiliser des répulsifs contre les insectes, surtout pendant les périodes où les phlébotomes sont les plus actifs (Laboudi M. et al., 2018).

En plus des mesures de prévention primaires, le Maroc met en place des mesures de diagnostic et de traitement de la leishmaniose dans la province de Ouarzazate et d'autres régions endémiques. Pour le diagnostic, plusieurs méthodes sont utilisées, comme mentionné dans la section des méthodes de diagnostic, notamment la microscopie pour détecter les parasites dans les tissus, la sérologie pour détecter les anticorps produits en réponse à l'infection, et les méthodes moléculaires telles que la PCR pour détecter l'ADN du parasite. Ces méthodes aident à confirmer le diagnostic et à déterminer le type de leishmaniose affectant le patient.

Pour traiter cette maladie, plusieurs options sont disponibles, notamment le traitement par le méglumine antimoniate, un médicament antiparasitaire utilisé pour les formes graves de la maladie. Le traitement par la paromomycine est également utilisé, en particulier pour les formes cutanées de la maladie. D'autres options de traitement incluent la thermothérapie, la cryothérapie, la thérapie par photodynamie et la thérapie vaccinale. Ces traitements visent à éliminer le parasite et à réduire les symptômes de la maladie, contribuant ainsi à la prévention de sa propagation dans la population (Laboudi M. et al., 2018).

1.2 Apprentissage Profond (Deep Learning)

1.2.1 Introduction au apprentissage profond

De nos jours, l'intelligence artificielle progresse à un rythme soutenu, et l'apprentissage profond en est l'un des principaux moteurs. L'apprentissage profond, ou deep learning, est un ensemble de méthodes d'apprentissage automatique, où des couches de traitement empilées sont utilisées pour créer des représentations abstraites des données, formant un "réseau neuronal" artificiel (LeCun Y, Bengio Y, Hinton G.). En d'autres termes, il reproduit le fonctionnement de notre cerveau. Les algorithmes d'apprentissage profond imitent la structure du système nerveux, où chaque neurone est connecté aux autres et transmet des informations. La figure 1.7 montre qu'une différence clé entre les modèles d'apprentissage automatique traditionnels et ceux de l'apprentissage profond réside dans l'extraction des caractéristiques. Alors que dans l'apprentissage automatique classique, cette extraction est effectuée par les humains, dans l'apprentissage profond, le modèle apprend à identifier les caractéristiques par lui-même.

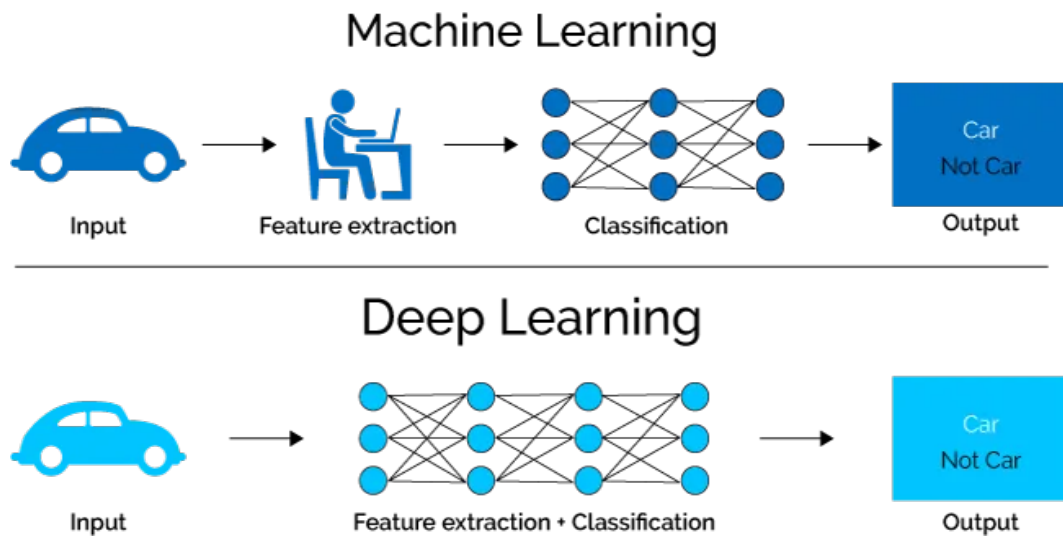


FIGURE 1.7 – Différence entre l'apprentissage automatique et l'apprentissage profond

1.2.2 Différentes approches d'apprentissage profond

Les méthodologies d'apprentissage profond, tout comme celles de l'apprentissage automatique, peuvent être divisées en plusieurs catégories : supervisé, semi-supervisé et non supervisé. En plus de ces catégories traditionnelles, il existe une méthode distincte appelée apprentissage par renforcement (RL) ou apprentissage par renforcement profond (DRL), souvent considérée dans le contexte de l'apprentissage semi-supervisé ou non supervisé. Ces approches permettent de former des modèles capables d'apprendre à partir d'une variété de types de données et de scénarios d'entraînement.

Apprentissage supervisé

Les algorithmes d'apprentissage supervisé construisent un modèle mathématique à partir d'un ensemble de données contenant à la fois les entrées et les sorties souhaitées. Nous formons la machine en utilisant des données "étiquetées," ce qui signifie que les réponses correctes sont fournies avec les données. Cela ressemble à un processus d'apprentissage sous la supervision d'un instructeur. Un algorithme d'apprentissage supervisé apprend à partir de ces données étiquetées, ce qui permet de prédire les résultats pour de nouvelles données.

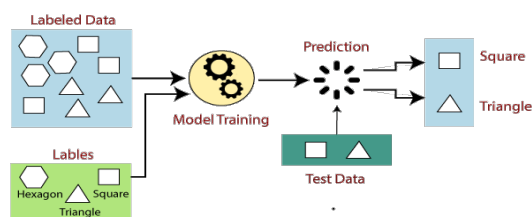


FIGURE 1.8 – Le fonctionnement d'apprentissage supervisé

Dans les approches d'apprentissage profond supervisées, l'entraînement du modèle se base sur des ensembles de données comprenant des entrées et leurs sorties correspondantes. Le réseau ajuste ses paramètres de manière itérative pour améliorer la précision des prédictions des sorties désirées. Plusieurs techniques d'apprentissage supervisé sont employées en deep learning, parmi lesquelles les réseaux de neurones profonds (DNN), les réseaux de neurones convolutifs (CNN), et les réseaux de neurones récurrents (RNN). Ces derniers incluent des variantes avancées telles que les mémoires à long terme (LSTM) et les unités récurrentes fermées (GRU), chacune apportant des avantages spécifiques pour traiter différents types de données et problèmes.

Apprentissage non-supervisé

L'apprentissage non supervisé concerne l'apprentissage à partir de données non étiquetées, où le modèle cherche à découvrir des motifs, des structures ou des regroupements au sein des données sans aucune orientation explicite. Contrairement à l'apprentissage supervisé, il n'y a pas de sorties cibles prédéfinies, ce qui rend ce type d'apprentissage particulièrement utile pour l'analyse exploratoire des données.

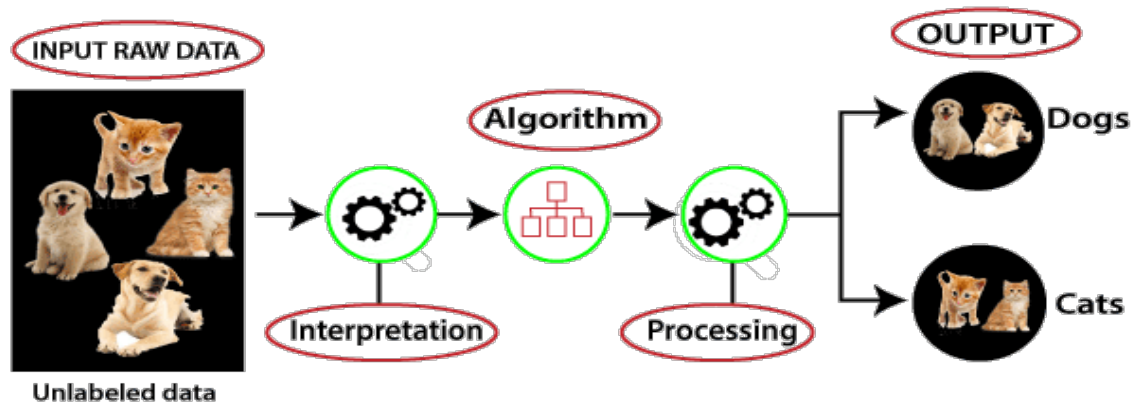


FIGURE 1.9 – Le fonctionnement d'apprentissage non-supervisé

L'objectif principal de l'apprentissage non supervisé dans l'apprentissage profond est d'extraire automatiquement des motifs significatifs à partir des données, sans orientation explicite de l'utilisateur sur ce que le modèle doit apprendre. Les algorithmes d'apprentissage profond non supervisé visent à découvrir des caractéristiques essentielles, des regroupements naturels ou des schémas récurrents dans les données, souvent pour faciliter des tâches telles que la classification, la génération de données ou la réduction de dimensionnalité. Les techniques les plus courantes de l'apprentissage profond non supervisé sont le regroupement (clustering), qui vise simplement à regrouper des données similaires pour former des clusters, l'apprentissage de représentations, qui cherche à apprendre des caractéristiques utiles à partir des données, et les réseaux générateurs adverses (GAN), qui génèrent de nouvelles données réalistes à partir d'un ensemble de données donné. Ces approches sont essentielles pour explorer et comprendre des ensembles de données complexes et sont appliquées dans de nombreux domaines, tels que la reconnaissance d'images, le traitement du langage naturel, et la biologie computationnelle.

Apprentissage par renforcement (RL)

Dans l'apprentissage par renforcement (RL), l'approche est totalement différente de celles de l'apprentissage supervisé et non supervisé. Au lieu de travailler avec des ensembles de données étiquetées ou non étiquetées, l'agent d'apprentissage interagit directement avec un environnement et apprend de ses actions et des récompenses ou punitions qu'il reçoit en conséquence. L'objectif de l'apprentissage par renforcement est d'apprendre une politique, c'est-à-dire une stratégie qui guide les actions de l'agent en fonction de l'état de l'environnement, afin de maximiser une récompense cumulative à long terme. Contrairement à l'apprentissage supervisé où l'on sait quelles actions sont correctes, dans l'apprentissage par renforcement, l'agent doit découvrir quelles actions lui permettent d'obtenir les meilleures récompenses en explorant et en expérimentant. Les algorithmes d'apprentissage par renforcement sont utilisés dans des domaines tels que les jeux, la robotique, et la gestion des stocks, où les agents doivent apprendre à prendre des décisions optimales dans des environnements complexes et dynamiques. L'apprentissage par renforcement de base est modélisé comme un processus de décision markovien (MDP) (Figure 1.10).

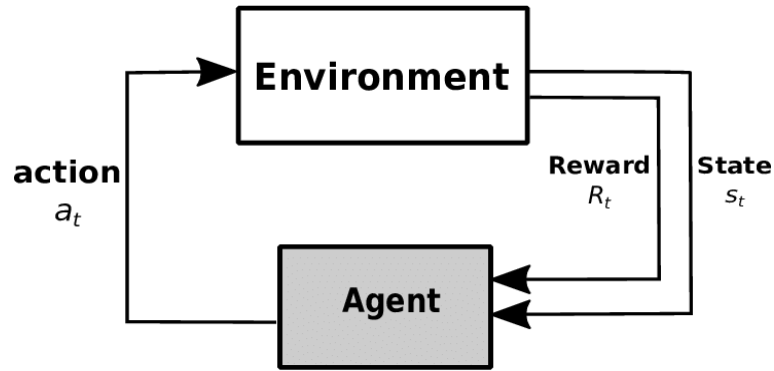


FIGURE 1.10 – Le fonctionnement d'apprentissage par renforcement

Les algorithmes d'apprentissage profond par renforcement, tels que le Deep Q-Network (DQN), sont très populaires dans ce domaine, en raison de leur capacité à apprendre des politiques complexes dans des environnements divers tels que les jeux vidéo, la robotique ou la gestion des ressources énergétiques. Les chercheurs ont découvert qu'en utilisant des réseaux de neurones profonds pour représenter les politiques ou les fonctions de valeur, ces approches peuvent gérer des états et des actions complexes, ce qui les rend adaptées à des tâches hautement dimensionnelles et dynamiques. L'apprentissage profond par renforcement est actuellement à l'avant-garde de la recherche en intelligence artificielle, car il peut aider à créer des systèmes autonomes capables d'apprendre et d'agir de manière adaptative dans des environnements complexes et variés.

Apprentissage par transfert

L'apprentissage par transfert est une nouvelle approche en apprentissage automatique où un modèle qui a été entraîné sur une tâche est utilisé comme point de départ pour un modèle destiné à être entraîné sur une nouvelle tâche similaire. Cela se fait en transférant les connaissances que le premier modèle a acquises sur les caractéristiques des données au second modèle. En apprentissage profond, l'apprentissage par transfert est souvent utilisé pour résoudre des problèmes avec des données limitées. Cela est dû au fait que les modèles d'apprentissage profond nécessitent généralement une grande quantité de données pour s'entraîner, ce qui peut être difficile et coûteux à obtenir.

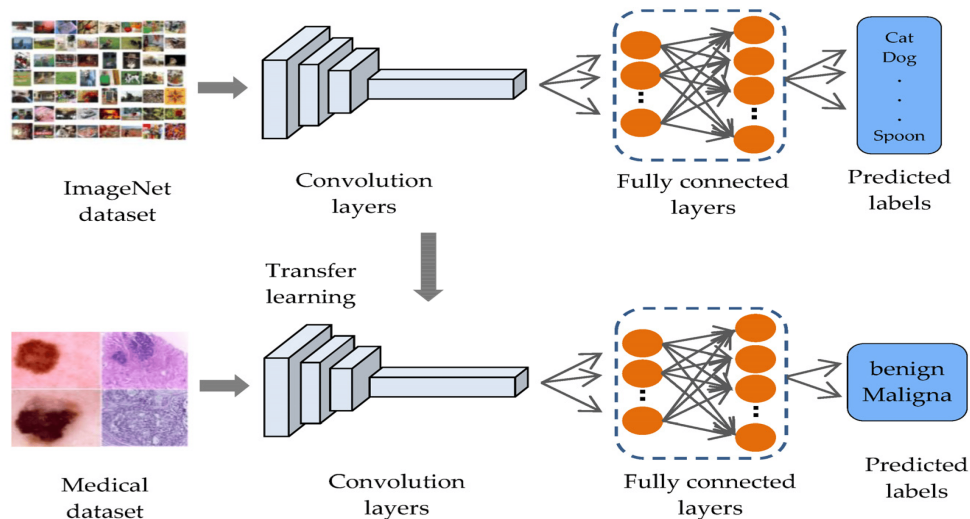


FIGURE 1.11 – Le fonctionnement d'apprentissage profond par transfert

Dans l'apprentissage profond par transfert, les modèles pré-entraînés les plus utilisés sont BERT et GPT pour le traitement du langage naturel, ResNet et VGG pour la classification d'images, Inception et Xception pour leurs architectures avancées de convolutions, MobileNet pour les applications mobiles, et YOLO pour la

détection d'objets en temps réel. Ces modèles ont gagné dans la popularité en raison de leur efficacité et de leurs performances élevées sur diverses tâches, ce qui les rend essentiels pour les applications de reconnaissance des images et les tâches avancées d'intelligence artificielle.

1.2.3 Les Réseaux de Neurons Artificiels (RNA)

La famille des réseaux de neurones artificiels (RNA) suscite un intérêt particulier en raison de leur structure flexible, qui permet de les adapter à une grande variété de contextes dans les tâches de classification, de régression et de regroupement (Janiesch, C., Zschech, P. & Heinrich, K. (2021)). Ces algorithmes d'apprentissage profond constituent la base des réseaux de neurones profonds. Ils sont particulièrement efficaces pour apprendre des hypothèses complexes non linéaires avec de grands ensembles de données comportant de nombreuses caractéristiques. Les RNA se composent de plusieurs couches de traitement mathématique, avec des unités appelées neurones. Les neurones de la couche d'entrée reçoivent des informations du monde extérieur, qui sont ensuite transformées par les neurones des couches cachées avant d'atteindre les unités de sortie. La figure 1.12 montre un RNA avec une couche d'entrée avec 4 unités, deux couches cachées avec respectivement 4 et 3 neurones, et une couche de sortie avec 2 unités.

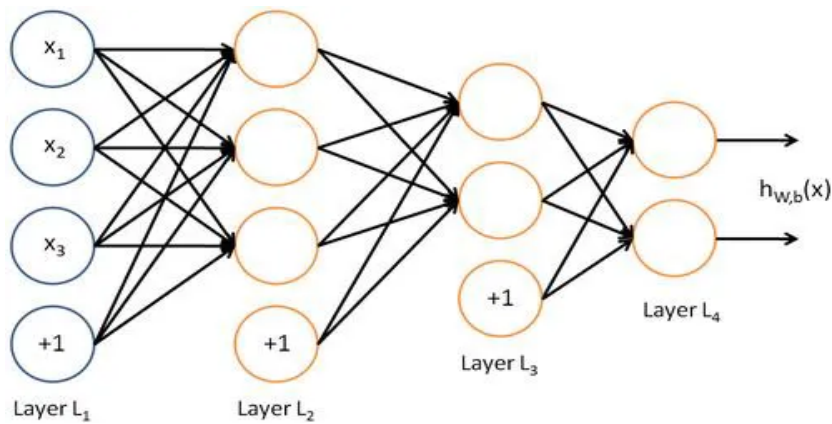


FIGURE 1.12 – Représentation d'un RNA

Pour commencer, un neurone dans un RNA typique cumule les valeurs de chaque neurone de la couche précédente auquel il est connecté. Par exemple, dans la Figure 1.13, le neurone reçoit trois entrées, qui sont ensuite multipliées par des poids spécifiques (w_1 , w_2 , w_3) et additionnées. Les poids déterminent la force de la connexion entre deux neurones et sont ajustés pendant l'apprentissage. Après cette addition, une valeur de biais est ajoutée au résultat, puis une fonction d'activation est appliquée à cette somme pour produire une sortie finale.

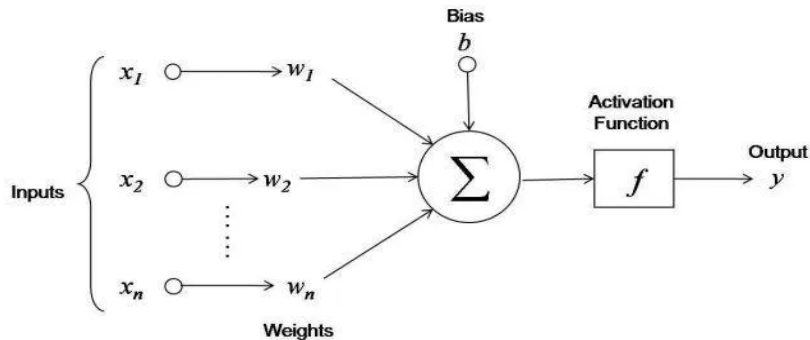


FIGURE 1.13 – Le fonctionnement d'un neurone dans un RNA

Le fonctionnement décrit d'un seul neurone doit être appliqué à l'ensemble du réseau par un processus appelé propagation avant. On commence avec les activations des unités d'entrée, puis on les transmet à la couche cachée où l'on calcule les activations, avant de les propager à la couche de sortie pour obtenir les activations finales.

Pour déterminer les poids optimaux, on utilise un processus appelé rétropropagation. Ce processus, couplé avec des méthodes d'optimisations comme la descente de gradient, optimise les poids afin que le réseau de neurones puisse apprendre à associer correctement les entrées aléatoires aux sorties souhaitées.

1.2.4 Les Réseaux de Neurones Convolutifs (RNC)

Dans l'apprentissage profond, un réseau neuronal convolutif (RNC) est un type particulier de réseau de neurones qui peut extraire automatiquement des caractéristiques à partir des données en utilisant des structures de convolution. Contrairement aux méthodes traditionnelles, le RNC n'a pas besoin d'extraire manuellement les caractéristiques des données. Son architecture s'inspire du fonctionnement de la perception visuelle.

La figure 1.13 montre un RNC, qui est composé de plusieurs éléments clés pour traiter efficacement les données d'entrée : les couches de convolution, Les fonctions d'activation, les couches de pooling et les couches entièrement connectées.

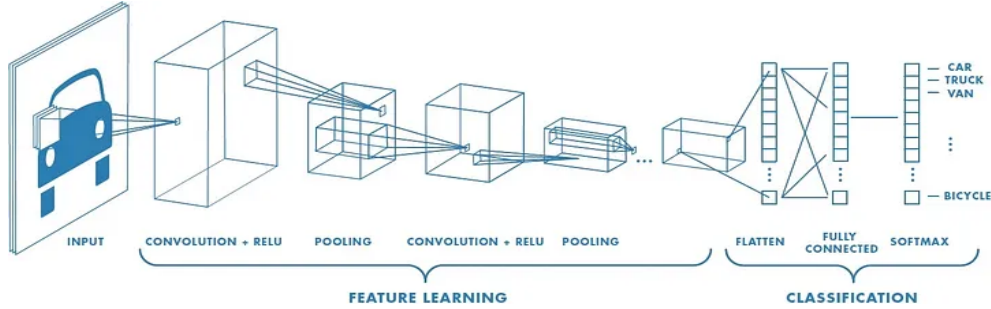


FIGURE 1.14 – Exemple d'un RNC

La couche de convolution

La convolution est une intégrale qui exprime le degré de chevauchement entre une fonction g lorsqu'elle est décalée sur une autre fonction f . Elle permet ainsi de "mélanger" une fonction avec une autre. Par exemple, en imagerie de synthèse, la carte brute mesurée est une convolution de la carte "propre" réelle avec le faisceau brut (la transformée de Fourier de la distribution d'échantillonnage). Mathématiquement, la convolution de deux fonctions continues f et g est définie par l'intégrale suivante :

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau \quad (1.1)$$

Dans le contexte des réseaux neuronaux convolutifs, nous travaillons généralement avec des données discrètes. La convolution discrète de deux séquences x et h est donnée par :

$$(x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]h[n - k] \quad (1.2)$$

Pour des séquences finies, cette expression se simplifie en :

$$(x * h)[n] = \sum_{k=0}^{K-1} x[k]h[n - k] \quad (1.3)$$

où K est la longueur de la séquence h .

Lorsqu'on applique cette opération dans un réseau neuronal convolutif, une image d'entrée I est convoluée avec un filtre F pour produire une sortie O . En notation matricielle, si I est une matrice de taille $M \times N$ et F est un filtre de taille $m \times n$, la sortie O est donnée par :

$$O[i, j] = \sum_{u=0}^{m-1} \sum_{v=0}^{n-1} I[i + u, j + v]F[u, v] \quad (1.4)$$

La couche de convolution est la première étape dans l'extraction des caractéristiques d'une image d'entrée. Elle préserve les relations entre les pixels en apprenant les traits saillants de l'image à partir de petits carrés de données d'entrée.

Prenons une matrice 5 x 5 dont les valeurs des pixels de l'image sont 0 et 1, et une matrice de filtre 3 x 3 comme indiqué dans la figure 1.15 :

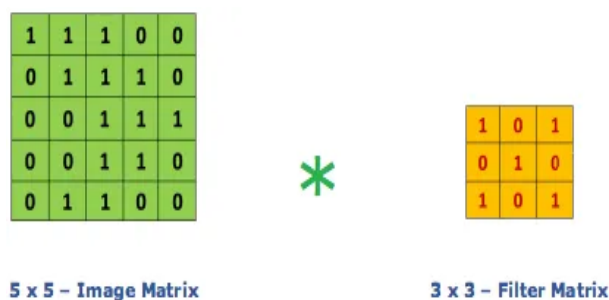


FIGURE 1.15 – Une matrice d'image 5 x 5 avec une matrice de filtre 3 x 3

Pour appliquer la convolution, nous plaçons le filtre 3x3 sur une région de 3x3 de l'image, puis nous multiplions chaque valeur du filtre par la valeur correspondante de l'image et additionnons les résultats. Nous répétons ensuite ce processus pour chaque région de 3x3 en déplaçant le filtre à travers l'image, ce qui crée une nouvelle matrice appelée "carte des caractéristiques", comme illustré dans la figure 1.16. Cette carte des caractéristiques met généralement en évidence les caractéristiques importantes de l'image, telles que les bords ou les textures, en fonction des valeurs du filtre.



FIGURE 1.16 – Le processus de convolution

Les fonction d'activation

Les fonctions d'activation sont une partie cruciale d'un RNC, elles introduisent de la non-linéarité, ce qui permet au RNC de modéliser des relations complexes entre les caractéristiques extraites des données. ReLU (Rectified Linear Unit), figure parmi les fonctions d'activation les plus couramment utilisées dans les RNC. C'est une fonction d'activation non linéaire efficace pour lutter contre le problème de disparition du gradient et accélérer la convergence de l'apprentissage. Sa formule mathématique, $f(x) = \max(0, x)$, signifie qu'elle met à zéro les valeurs négatives lorsqu'elle est appliquée, comme illustré dans la figure 1.17.

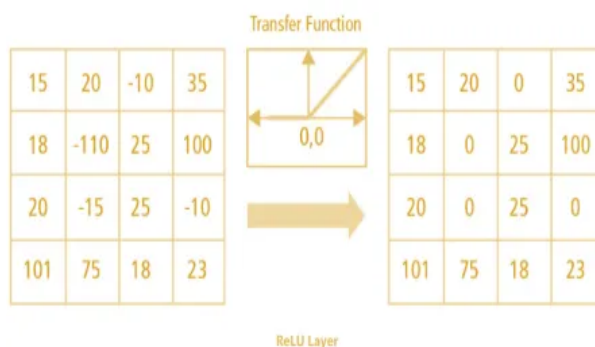


FIGURE 1.17 – La fonction d'activation ReLU

Il existe également d'autres fonctions d'activation, qui permettent d'introduire de la non-linéarité dans un réseau des neurones, mais chacune avec sa façon de le faire, parmi les fonctions d'activation les plus connus :

- * **Sigmoïde** : La fonction sigmoïde, également appelée fonction logistique, transforme les valeurs d'entrée en une sortie comprise entre 0 et 1, ce qui la rend utile pour les tâches de classification binaire. Elle est définie par la formule :

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (1.5)$$

- * **Softmax** : La fonction softmax est utilisée principalement pour les problèmes de classification multiclasse. Elle convertit un vecteur de valeurs en une distribution de probabilités. La formule est :

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad (1.6)$$

où x_i est l'entrée i -ème et la somme est prise sur toutes les entrées.

- * **Tangente hyperbolique (Tanh)** : La fonction tangente hyperbolique transforme les valeurs d'entrée en une sortie comprise entre -1 et 1, offrant une meilleure centration de la donnée par rapport à la sigmoïde. Elle est définie par :

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.7)$$

Les couches de pooling

Les couches de pooling jouent un rôle crucial dans un RNC en réduisant le nombre de paramètres lorsque les images sont trop grandes. L'objectif de l'utilisation des couches de pooling est de diminuer la dimensionnalité de chaque carte des caractéristiques tout en préservant uniquement les informations importantes. Il existe différents types de pooling spatial : max pooling, average pooling, et sum pooling. Le max pooling sélectionne le plus grand élément de la carte des caractéristiques rectifiée, comme le montre la figure 1.18, alors que l'average pooling calcule la moyenne des valeurs de la région. Enfin, le sum pooling calcule la somme de tous les éléments de la carte des caractéristiques.

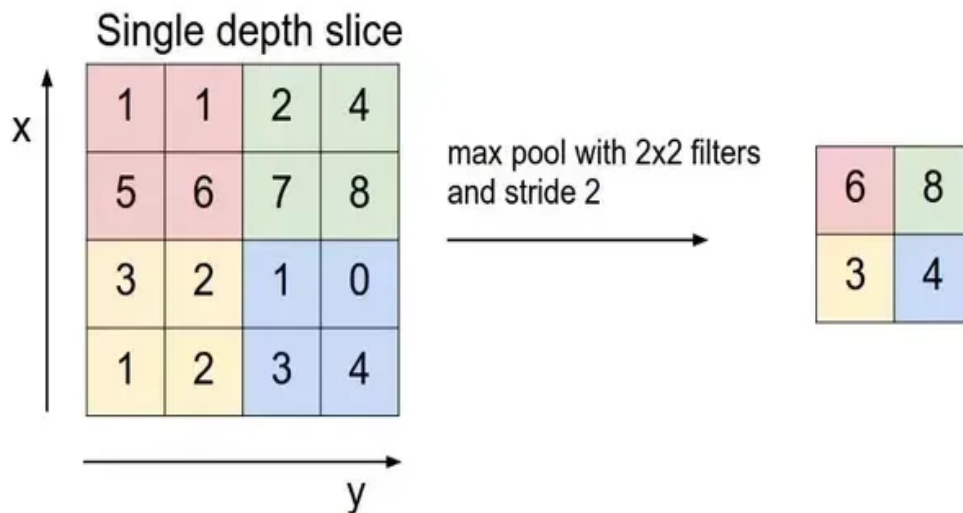


FIGURE 1.18 – Max Pooling

Les couches entièrement connectées

Une couche entièrement connectée, ou FCL (Fully Connected Layer), est une couche qui agit comme un classificateur en associant les caractéristiques extraites aux différentes classes. Après avoir extrait les caractéristiques pertinentes de l'image à l'aide des couches de convolution et de pooling, le RNC aplatit la sortie en un vecteur et la passe à une couche entièrement connectée.

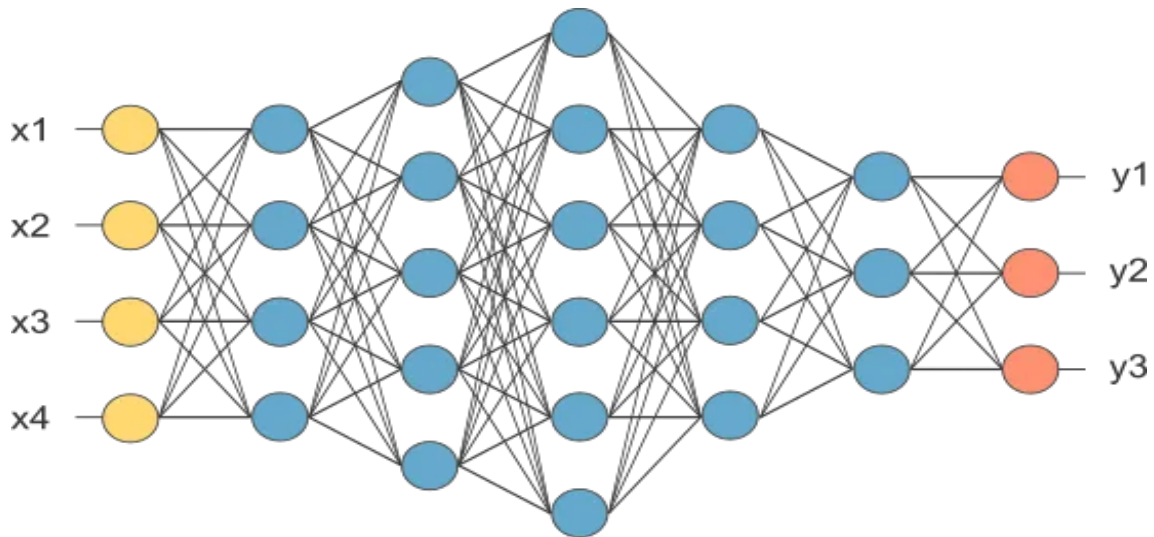


FIGURE 1.19 – Un exemple d’une couche entièrement connectée

Comme montré par la figure 1.19, la matrice de la carte des caractéristiques sera convertie en vecteur ($x_1, x_2, x_3, \dots$). Avec les couches entièrement connectées, nous combinons ces caractéristiques pour créer un modèle. Enfin, nous utilisons une fonction d’activation telle que softmax ou sigmoid pour classifier les sorties comme étant une voiture ou un camion, etc.

1.2.5 Les Réseaux de Neurons Récurrents (RNR)

1.2.5.1 Introduction aux RNR

Un réseau des neurones récurrent (RNR) est une extension des réseaux neuronaux artificiels qui inclut une mémoire interne. Les RNR se distinguent par leur nature récurrente, répétant le même traitement pour chaque entrée de données, sauf que la sortie actuelle est influencée par le calcul précédent. Après avoir généré une sortie, celle-ci est réintroduite dans le réseau pour les étapes suivantes. Ainsi, pour prendre une décision, le RNR évalue l’entrée courante en tenant compte des informations acquises des entrées précédentes. Comme le montre la figure 1.20, ce qui distingue les RNR des RNA est la manière dont l’information circule à travers le réseau. Alors que les RNA transfèrent l’information de manière linéaire sans boucles, les RNR ont des cycles et réinjectent l’information en eux-mêmes (Schmidt, R. M., 2019).

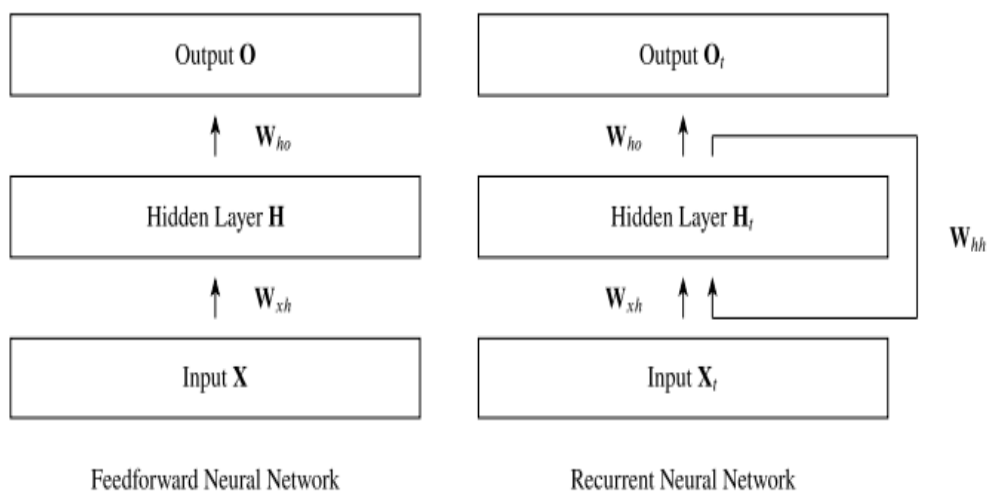


FIGURE 1.20 – Différence entre un RNA et un RNR

L’architecture d’un RNR se caractérise par sa capacité à traiter des séquences de données en maintenant une mémoire des états précédents grâce à ses connexions récurrentes. Pour bien comprendre son architecture, prenons un exemple d’un RNR simple :

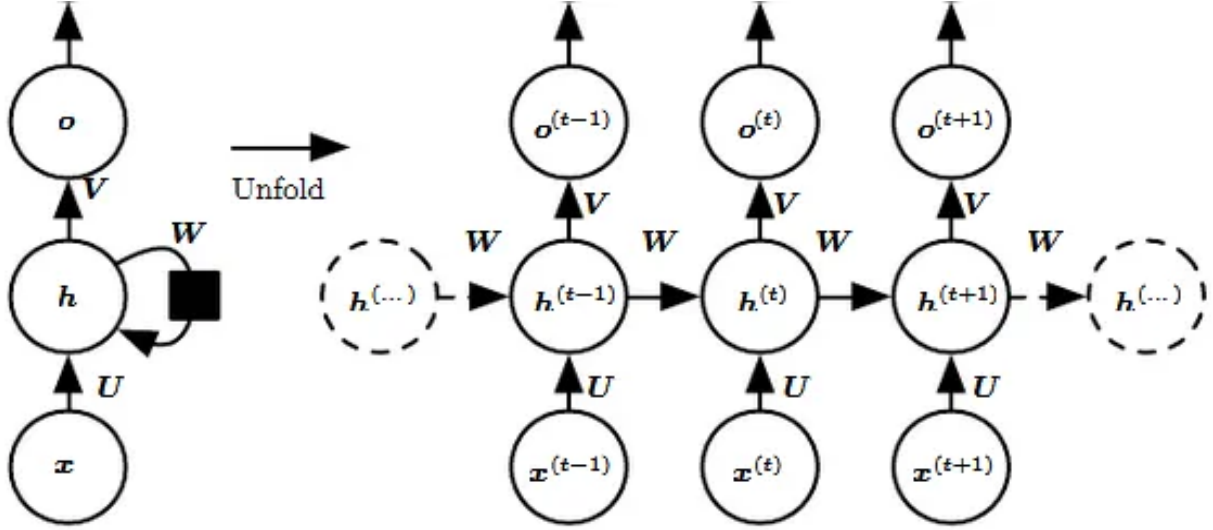


FIGURE 1.21 – Un RNR simple déplié (ou déroulé) en un réseau complet

La partie gauche de la figure 1.21 montre la notation de ce RNR, tandis que la partie droite montre ce RNR déplié (ou déroulé) en un réseau complet. Déplier un RNR signifie que nous représentons le réseau pour toute la séquence. Par exemple, si la séquence que nous étudions est une phrase de trois mots, le réseau serait déplié en un réseau de neurones à trois couches, une couche pour chaque mot. L'architecture de ce RNR simple peut être expliqué comme cela :

- * **Entrée** : L'entrée du réseau à chaque instant t est représentée par $x^{(t)}$. Par exemple, $x^{(1)}$ peut être un vecteur one-hot représentant un mot d'une phrase.
- * **Poids** : Les connexions dans le RNN sont paramétrées par des matrices de poids. Les connexions de l'entrée vers l'état caché sont définies par U , les connexions récurrentes de l'état caché vers lui-même par W , et les connexions de l'état caché vers la sortie par V . Ces poids (U, V, W) sont constants à travers le temps.
- * **État caché** : $h^{(t)}$ désigne l'état caché à l'instant t et fonctionne comme la "mémoire" du réseau. Cet état est calculé en fonction de l'entrée actuelle et de l'état caché précédent : $h^{(t)} = f(Ux^{(t)} + Wh^{(t-1)})$. La fonction f est une fonction d'activation que l'on suppose être la tangente hyperbolique (Tanh) dans cet exemple.
- * **Sortie** : $o(t)$ représente la sortie du réseau à chaque instant t . Dans la figure 1.21, une flèche après $o(t)$ montre la sortie, qui est souvent transformée par une fonction d'activation, en particulier lorsque le réseau contient des couches supplémentaires par la suite. On suppose que ce modèle est entraîné pour prédire des valeurs discrètes.

Si on suppose que ce RNR est utilisé pour prédire des mots ou des caractères, la manière naturelle de représenter les variables discrètes est de considérer que la sortie o donne les probabilités logarithmiques non normalisées de chaque valeur possible de la variable discrète. Ensuite, nous pouvons appliquer l'opération softmax comme étape de post-traitement pour obtenir un vecteur $\hat{y}$ de probabilités normalisées sur la sortie. Le calcul du sortie de ce modèle se déroule dans un processus appelé Forward Pass (FP) qui peut donc être représenté par l'ensemble d'équations ci-dessous :

$$a^{(t)} = b + Wh^{(t-1)} + Ux^{(t)} \quad (1.8)$$

$$h^{(t)} = \tanh(a^{(t)}) \quad (1.9)$$

$$o^{(t)} = c + Vh^{(t)} \quad (1.10)$$

$$y(t) = \text{softmax}(o(t)) \quad (1.11)$$

Où :

- * $a^{(t)}$ est le vecteur d'activation à l'étape temporelle t
- * b est le vecteur du biais
- * W est la matrice de poids pour la couche cachée
- * $h^{(t-1)}$ est le vecteur caché à l'étape temporelle précédente, $t - 1$
- * U est la matrice de poids pour la couche d'entrée
- * $x^{(t)}$ est le vecteur d'entrée à l'étape temporelle t
- * $h^{(t)}$ est le vecteur caché à l'étape temporelle t
- * c est un autre vecteur de biais
- * V est la matrice de poids pour la couche de sortie
- * $o^{(t)}$ est le vecteur de sortie à l'étape temporelle t
- * $y^{(t)}$ est la sortie de la fonction softmax appliquée à $o^{(t)}$ à l'étape temporelle t

Pour entrainer ce RNR, une technique algorithmique appelé Backpropagation through time (BPTT) est utilisée. Elle permet de propager les erreurs à travers le temps, en tenant compte des dépendances entre les sorties successives du réseau. L'objectif de l'entraînement est de minimiser une fonction de perte, telle que la perte par entropie croisée catégorielle :

$$L(y, \hat{y}) = - \sum_{t=1}^T y_t \log(\hat{y}_t) \quad (1.12)$$

Où :

- * y est la séquence de cibles réelles
- * $\hat{y}$ est la séquence de prédictions du modèle

BPTT fonctionne en déroulant le RNR dans le temps, en calculant les gradients de la fonction de perte par rapport aux paramètres du réseau et en les accumulant. Les gradients sont calculés à l'aide de la règle de la chaîne, en tenant compte des dépendances entre les sorties successives du réseau.

Pour calculer le gradient de la fonction de perte par rapport à un paramètre w du réseau, on utilise la formule suivante :

$$\frac{\partial L}{\partial w} = \sum_{t=1}^T \frac{\partial L}{\partial o_t} \frac{\partial o_t}{\partial a_t} \frac{\partial a_t}{\partial w} \quad (1.13)$$

Où :

- * $\frac{\partial L}{\partial o_t}$ est la dérivée de la fonction de perte par rapport à la sortie o_t à l'étape temporelle t
- * $\frac{\partial o_t}{\partial a_t}$ est la dérivée de la sortie o_t par rapport à l'activation a_t à l'étape temporelle t
- * $\frac{\partial a_t}{\partial w}$ est la dérivée de l'activation a_t par rapport au paramètre w

Les gradients accumulés sont ensuite utilisés pour mettre à jour les poids du réseau à l'aide d'un algorithme d'optimisation, tel que la descente du gradient.

Les RNR peuvent être très efficaces pour les tâches de modélisation de séquences telles que la prédiction de séries temporelles et fournissent une solution fiable dans ces cas. Cependant, ils souffrent de deux problèmes :

- * **Le problème des gradients vanishing (évanouissants) :** Les RNR rencontrent ce problème lors de la rétropropagation, où les gradients utilisés pour ajuster les poids du réseau peuvent devenir extrêmement petits en remontant les couches. En conséquence, les premières couches du réseau apprennent lentement, voire pas du tout, car les informations sur l'erreur provenant des couches ultérieures s'affaiblissent considérablement jusqu'à disparaître, d'où le nom du problème.
- * **Le problème des gradients explosifs :** Dans un autre scénario inverse, les gradients peuvent également exploser de façon démesurée en remontant les couches. Cela conduit à des mises à jour instables des poids du réseau et empêche son apprentissage correct.

Heureusement, il existe des variantes des RNR qui permettent d'atténuer les problèmes des gradients vanishing et des gradients explosifs, comme les réseaux Long-Short Term Memory (LSTM) et les réseaux Long-Short Term Memory Bidirectionnels (BiLSTM) qu'on va expliquer dans la sous-section suivante.

1.2.5.2 Réseaux Long-Short Term Memory (LSTM)

Les réseaux de mémoire à long terme (LSTM) sont une variante des RNR conçus pour surmonter le problème du gradient vanishing qui affecte les RNR traditionnels. Contrairement aux RNR traditionnels, les LSTM peuvent apprendre des dépendances à long terme dans les données séquentielles, ce que les RNR ne peuvent pas accomplir en raison de leur architecture. Cela rend les LSTM beaucoup plus efficaces pour des tâches telles que la prédiction des épidémies, la reconnaissance vocale et la génération de texte.

Les RNR standard traitent les données séquentielles en faisant passer l'information d'une étape à une autre à travers un état caché. Cette architecture est incapable de capturer les dépendances à long terme dans les données, ce qui entraîne le problème du vanishing gradient. Comme montré dans la figure 1.22, Les LSTM contournent ce problème grâce à leur architecture particulière qui intègre des cellules de mémoire. Ces cellules permettent de stocker et de contrôler le flux d'information à travers le réseau.

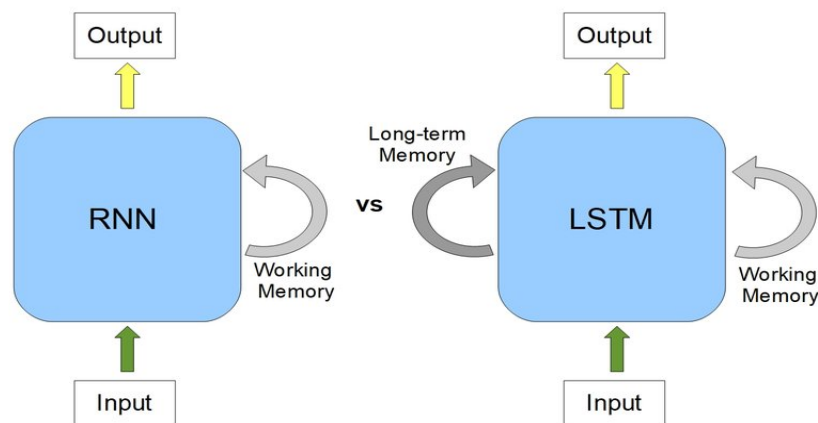


FIGURE 1.22 – La différence entre RNR et les réseaux LSTM

Un réseau LSTM est généralement composé de une ou plusieurs cellules LSTM qui se composent de plusieurs éléments :

- * **La cellule de mémoire :** Celle-ci stocke les informations pertinentes à la tâche en cours. Son état est généralement noté $c^{(t)}$ à l'instant t
- * **Les portes :** Les portes sont des mécanismes qui régulent le flux d'information entrant et sortant de la cellule de mémoire. Il existe trois portes principales dans une cellule LSTM :
 - * *La porte d'entrée ($i^{(t)}$) :* elle détermine quelle information de l'entrée courante ($x^{(t)}$) doit être stockée dans la cellule de mémoire ([Hochreiter & Schmidhuber, 1997]).

* *La porte d'oubli ($c^{(t)}$)* : elle décide quelle information de l'état précédent de la cellule ($c^{(t-1)}$) doit être conservée ou oubliée (voir équation dans [Hochreiter & Schmidhuber, 1997]).

* *La porte de sortie ($o^{(t)}$)* : elle contrôle quelle information de l'état actuel de la cellule ($c^{(t)}$) doit être transmise à la couche suivante du réseau ([Hochreiter & Schmidhuber, 1997]).

Le fonctionnement d'une cellule LSTM se décrit mathématiquement comme suit :

$$f^{(t)} = \sigma(W_f \cdot [h^{(t-1)}, x^{(t)}] + b_f) \quad (1.14)$$

$$i^{(t)} = \sigma(W_i \cdot [h^{(t-1)}, x^{(t)}] + b_i) \quad (1.15)$$

$$\tilde{c}^{(t)} = \tanh(W_c \cdot [h^{(t-1)}, x^{(t)}] + b_c) \quad (1.16)$$

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)} \quad (1.17)$$

$$o^{(t)} = \sigma(W_o \cdot [h^{(t-1)}, x^{(t)}] + b_o) \quad (1.18)$$

$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)}) \quad (1.19)$$

Où :

* σ est la fonction sigmoïde et $\tanh$ est la fonction tangente hyperbolique.

* W_f , W_i , W_c et W_o sont les matrices de poids des portes.

* b_f , b_i , b_c et b_o sont les vecteurs de biais des portes.

* $\odot$ désigne le produit Hadamard (produit élément par élément).

* $h^{(t-1)}$ est l'état caché de la couche précédente.

La figure 1.23 montre un exemple d'une cellule LSTM.

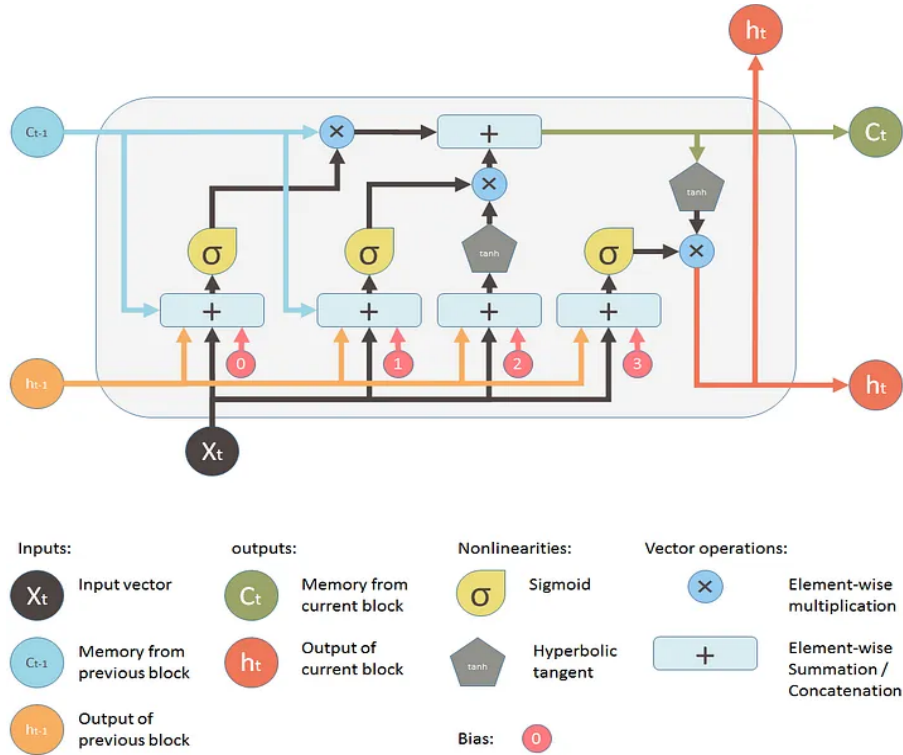


FIGURE 1.23 – Une cellule LSTM

1.2.5.3 Réseaux Long-Short Term Memory Bidirectionnels (Bi-LSTM)

Les réseaux de neurones à mémoire à long terme bidirectionnels (Bi-LSTMs) sont construits sur le concept des LSTM traditionnels en les étendant pour traiter les séquences d'entrée dans les deux sens, en avant et en arrière. En effet, les LSTM classiques ne prennent en compte que le contexte passé pour prédire la sortie suivante, tandis que les Bi-LSTM intègrent également le contexte futur. Cette capacité à exploiter l'information dans les deux directions permet aux Bi-LSTMs d'apprendre des dépendances à long terme plus riches. Comme la figure 1.24 montre, une cellule Bi-LSTM est constitué de deux couches LSTM, une qui traite la séquence de données dans le sens chronologique (avant en arrière) et l'autre dans le sens inverse (arrière en avant). Cela permet au modèle de capturer des informations contextuelles à la fois du passé et du futur pour chaque point temporel de la séquence.

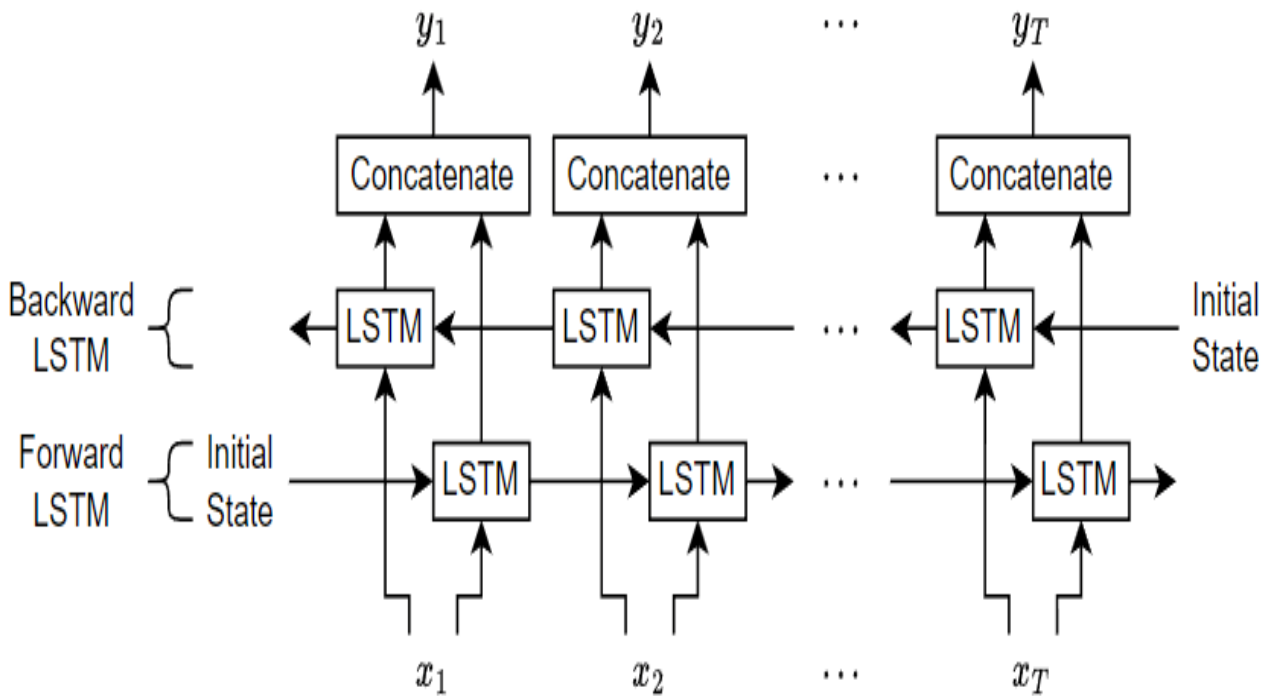


FIGURE 1.24 – Une cellule Bi-LSTM

1.3 Interprétabilité des modèles d'apprentissage automatique et profond

1.3.1 Introduction au méthode de SHAP

Avec la croissance rapide de l'apprentissage automatique et profond, l'interprétabilité des modèles est devenue l'un des axes principaux de recherche dans ce domaine. Malgré la puissance et les performances élevées des modèles complexes d'apprentissage profond et automatique, leur interprétabilité pose un problème majeur, ce qui a conduit les chercheurs à essayer de proposer des approches pour améliorer l'interprétabilité de ces modèles complexes afin de pouvoir expliquer les prédictions à des publics non techniques. Parmi les approches les plus intéressantes développées par les chercheurs au cours de la dernière décennie dans ce domaine, SHAP émerge comme l'une des approches les plus avancées pour expliquer les modèles d'apprentissage automatique et d'apprentissage profond. Comme la figure 1.25 le montre, SHAP est une méthode basée sur la théorie des jeux et la théorie des valeurs de shapley qui offre à la fois des explications locales et globales pour n'importe quel modèle d'apprentissage automatique ou d'apprentissage profond, offrant ainsi aux praticiens de l'apprentissage automatique un moyen prometteur d'expliquer les prédictions de leur modèle complexe à des publics non techniques.



FIGURE 1.25 – La méthode de SHAP

1.3.2 Le fonctionnement de SHAP

La théorie des valeurs de Shapley est une méthode introduite par LLOYD Shapley en 1953, qui vise à distribuer équitablement le gain total obtenu par une coalition de joueurs entre eux. En termes d'apprentissage automatique, cela signifie que nous pouvons considérer chaque variable comme un "joueur" dans un jeu collaboratif afin de répartir la prédiction entre toutes les variables, ce qui signifie que la valeur de shapley pour une variable i est définie comme suit :

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} (v(S \cup \{i\}) - v(S)) \quad (1.20)$$

où :

- * N est l'ensemble de toutes les caractéristiques,
- * S est un sous-ensemble de N ne contenant pas i ,
- * $v(S)$ est la valeur de la fonction caractéristique pour la coalition S , qui, dans notre cas, est la prédiction du modèle en utilisant uniquement les caractéristiques de S .

La méthode SHAP vise à utiliser la théorie des valeurs de Shapley pour calculer les importances des variables pour un modèle d'apprentissage automatique donné en créant un modèle explicatif additif dont les prédictions peuvent être représentées comme la somme des contributions de chaque variable ajoutée à une valeur constante :

$$g(z') = \phi_0 + \sum_{j=1}^M \phi_j z'_j \quad (1.21)$$

où z' est une version binaire du vecteur d'entrée z , M est le nombre total de caractéristiques, ϕ_j est la valeur de Shapley de la caractéristique j , et ϕ_0 est le terme de base (souvent la moyenne des prédictions sur les données d'entraînement).

Le calcul exact des valeurs de Shapley peut être coûteux en termes de calcul, surtout pour les modèles entraînés sur de grands ensembles de données avec de nombreuses variables. Pour surmonter cela, SHAP utilise plusieurs approches d'approximation telles que le noyau SHAP, une méthode qui échantillonne les données pour approximer les valeurs de Shapley en calculant les contributions marginales de manière stochastique, ou Tree SHAP, une méthode spécialement créée pour les algorithmes utilisant des arbres de décision qui exploite leur structure pour calculer les valeurs de Shapley exactes.

Pour les modèles d'apprentissage profond, tels que les réseaux des neurones, SHAP utilise un autre type de méthode pour expliquer leurs prédictions appelée Deep SHAP (Figure 1.26). Cette méthode utilise une approche basée sur l'échantillonnage pour approximer les valeurs de Shapley pour chaque variable d'entrée. Cette approche fonctionne en créant un ensemble de données de fond constitué d'échantillons représentant la distribution des caractéristiques d'entrée. Ensuite, elle perturbe ces échantillons pour créer un ensemble d'entrées perturbées. Pour chaque entrée perturbée, Deep SHAP calcule la différence de sortie entre l'entrée perturbée et l'entrée originale, ce qui est ensuite utilisé pour approximer les valeurs de Shapley pour chaque variable.

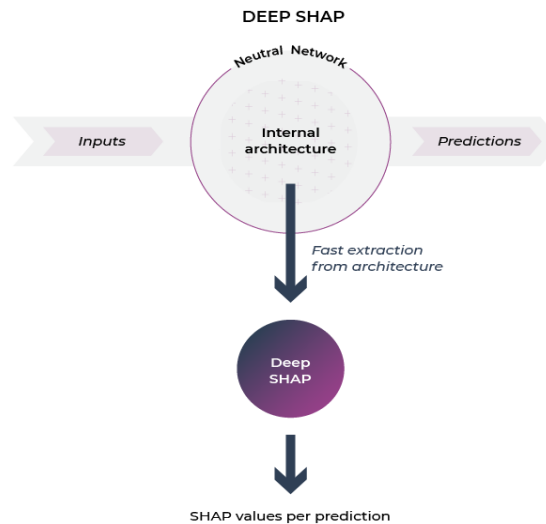


FIGURE 1.26 – La méthode Deep SHAP

Pour les réseaux de neurones plus avancés dont l'architecture peut être différente d'un réseau des neurones artificielles ordinaire, tels que les RNR et les LSTM, SHAP utilise une autre méthode appelée Gradient SHAP. Cette méthode cherche aussi à approximer les valeurs de Shapley pour chaque variable d'entrée, mais en utilisant les gradients de la sortie du modèle par rapport aux variables d'entrée. Elle sélectionne d'abord un point de données spécifique du base de données pour lequel l'utilisateur souhaite expliquer la prédiction faite par le modèle LSTM, par exemple. Ensuite, elle crée une valeur de référence, une valeur qui représente l'absence ou la valeur nulle pour chaque caractéristique, qui est ensuite utilisée comme point de référence pour calculer l'impact de chaque caractéristique sur la prédiction. Puis, pour chaque caractéristique, Gradient SHAP perturbe les valeurs des caractéristiques entre la valeur de référence et l'entrée réelle. Elle calcule la différence de sortie du modèle pour chaque valeur de caractéristique perturbée par rapport à la valeur de référence, puis calcule les gradients de la sortie du modèle par rapport aux caractéristiques d'entrée. Les gradients calculés représentent la sensibilité des prédictions du modèle aux changements de chaque caractéristique. Enfin, Gradient SHAP combine les gradients et les différences de sortie du modèle pour approximer les valeurs de Shapley pour chaque caractéristique en prenant une moyenne pondérée des gradients, où les poids sont les différences de sortie du modèle.

Conclusion

Dans ce chapitre, nous avons essayé de couvrir les concepts nécessaires pour élaborer une solution à la problématique traitée dans ce mémoire de fin d'études. Tout d'abord, nous avons effectué de nombreuses recherches sur la leishmaniose afin de comprendre ses types et sa nature, ses facteurs de risque, les données dont nous aurons besoin et leurs sources. Ensuite, nous avons discuté des mesures prises au Maroc et dans la province de Ouarzazate pour limiter son danger. Deuxièmement, nous avons exploré la théorie derrière les différentes approches de l'apprentissage profond connues pour être efficaces pour notre problème, telles que les CNNs et les RNNs et leurs variations (LSTMs et BiLSTMs). Troisièmement, nous avons expliqué l'une des principales approches pour résoudre le problème de l'interprétabilité dans les modèles complexes de ML et DL, qui est SHAP. Après les recherches effectuées dans ce chapitre, nous avons conclu que les BiLSTMs et les CNNs pourraient être une très bonne approche pour résoudre notre problématique, couplés avec la méthode SHAP pour offrir des prédictions compréhensibles pour les décideurs et les responsables du contrôle épidémiologique dans la province de Ouarzazate. Dans le chapitre suivant, nous aborderons les objectives, les défis et l'analyse des solutions existantes de la prédiction des cas des maladies infectieuses.

Chapitre 2

Prédiction des cas des maladies infectieuses : Objectifs, Défis et Solutions Analytiques

Introduction

Dans ce chapitre, nous abordons la prédiction des cas des maladies infectieuses et son rôle crucial dans la lutte contre ces maladies. Nous examinons ensuite ses objectifs ainsi que les défis qu'elle soulève. Enfin, nous analysons différentes solutions existantes pour ce genre de problème, en mettant en lumière les avantages et les limites de chaque approche.

2.1 Apércu

La prédiction des cas de maladies infectieuses consiste à prévoir le nombre de personnes qui pourraient être atteintes par ces maladies dans une région donnée et sur une période spécifiée. Cette prédiction se base souvent sur des données historiques sur les cas de maladies infectieuses, mais aussi sur d'autres facteurs tels que les conditions environnementales et climatiques d'une région donnée et les caractéristiques démographiques et socio-économiques de la population. En utilisant des méthodes modernes et efficaces comme l'apprentissage automatique, l'apprentissage profond et les techniques statistiques, la prédiction des cas de maladies infectieuses peut présenter plusieurs avantages, notamment :

- * **Utilisation efficace des données limitées :** Les données sur les cas de maladies infectieuses, qu'il s'agisse de données climatiques, cliniques ou sociodémographiques, sont souvent très limitées. C'est pour cela que l'utilisation des méthodes statistiques et d'apprentissage automatique peut être efficace même avec des ensembles de données relativement petits.
- * **Adaptabilité aux changements :** L'utilisation de méthodes sophistiquées comme les modèles d'apprentissage automatique est souvent très efficace parce qu'ils peuvent être adaptés rapidement aux nouvelles données. Cela permet une mise à jour régulière des prévisions en fonction des conditions épidémiologiques, qui changent beaucoup en fonction du temps.
- * **Intégration de divers types de données :** Comme évoqué dans la section sur les facteurs de risque de transmission de la leishmaniose, cette maladie et d'autres maladies infectieuses de nature parasitaire dépendent souvent de multiples facteurs biologiques, climatiques, environnementaux et sociodémographiques. Cette complexité nécessite l'utilisation de divers types de données. Les méthodes d'apprentissage automatique peuvent intégrer efficacement ces données provenant de différentes sources et dans différents formats, notamment les données épidémiologiques, environnementales et sociodémographiques, ce qui permet d'améliorer la précision des prévisions.
- * **La rapidité des prédictions :** En utilisant des algorithmes d'analyse statistiques modernes ainsi que des modèles d'apprentissage automatique et profond, on peut avoir beaucoup plus rapidement des prédictions des cas ce qui peut aider les autorités sanitaires et les décideurs de prendre des décisions rapides pour lutter contre ces maladies.

2.2 Les objectifs de la prédiction des cas des maladies infectieuses

L'importance de cette prédiction réside dans sa capacité à aider les autorités sanitaires et les décideurs à atteindre certains objectifs, parmi eux on trouve :

1. **Allocation efficace des ressources** : Prédire les cas de maladies infectieuses aide les autorités sanitaires à allouer de manière intelligente les ressources telles que les médicaments, les insecticides et le personnel médical là où les gens en ont le plus besoin.
2. **Prévention des épidémies** : En prévoyant les tendances des cas des maladies infectieuses, les autorités sanitaires peuvent identifier les zones à risque élevé d'épidémies et mettre en place des mesures préventives telles que le renforcement de la surveillance et la sensibilisation des populations locales pour éviter la propagation de ces maladies.
3. **Évaluation des interventions** : Prédire les cas des maladies infectieuses aide aussi à évaluer l'efficacité des actions de santé publique pour contrôler les maladies. En comparant les prédictions aux données réelles, les décideurs peuvent voir si les mesures prises fonctionnent et ajuster leurs stratégies sinon.
4. **Réduction des coûts de santé** : Lorsqu'ils anticipent les cas des maladies infectieuses, les autorités sanitaires peuvent réduire les coûts associés à la prise en charge des cas graves de ces maladies.

2.3 Les défis de la prédiction des cas des maladies infectieuses

La complexité de ces maladies et la variabilité des données disponibles, ce sont des problèmes qui se présentent toujours lorsqu'il s'agit de la prédiction des cas des maladies, surtout les maladies parasitaires. Ces défis peuvent avoir un impact sur la fiabilité des prévisions et la capacité des autorités sanitaires à prendre des mesures préventives efficaces. Les principaux défis comprennent :

1. **Données limitées et qualité des données** : Comme mentionnés dans la section des sources des données relatives à la leishmaniose, ces données peuvent être limitées en raison de l'absence de systèmes de surveillance efficaces dans certaines régions. En plus, la qualité des données peut varier, ce qui peut affecter la précision des prévisions.
2. **Complexité des facteurs de risque** : Nous avons déjà parlé du fait que la leishmaniose est souvent causée par des facteurs de risque très complexes comme les conditions climatiques d'une région et les conditions socioéconomiques des populations, ce qui fait que la modélisation de ces interactions complexes peut être un défi majeur.
3. **Surdétection et sous-détection** : Les modèles statistiques et d'apprentissage automatique de prédiction souffrent toujours de la surdétection (prédire des épidémies qui ne se produisent pas) ou de la sous-détection (ne pas prédire les épidémies qui se produisent), ce qui peut affecter la prise de décision en matière de santé publique.
4. **Interprétabilité des modèles** : Certains modèles de machine learning, en particulier les modèles complexes comme les réseaux de neurones profonds, souffrent toujours des problèmes d'interprétabilité, ce qui rend difficile de comprendre comment les prévisions sont générées.

2.4 Analyse des solutions existantes pour la prédiction des cas des maladies infectieuses

L'analyse des solutions existantes pour des problèmes complexes comme le nôtre peut être considéré comme une étape très importante dans le processus de conception de la solution. Cette section se divise en deux parties. Dans un premier temps, nous examinons plusieurs solutions existantes pour la prédiction des cas de maladies infectieuses en fournissant des résumés de ces travaux. Dans la seconde partie, nous comparons ces différentes approches selon des critères préalablement définis, en utilisant un tableau comparatif, suivi d'une discussion approfondie des résultats obtenus.

2.4.1 Synthèse des solutions existantes

Article 1 : Climate based malaria forecasting system for Andhra Pradesh, India (Mopuri et al. (2020))

Cet article étudie la prédiction des cas de paludisme dans le district de Visakhapatnam, en Inde, en utilisant des modèles statistiques et d'apprentissage automatique. Les données mensuelles sur les cas de paludisme, obtenues du Programme national de lutte contre les maladies vectorielles de Visakhapatnam, sont considérées comme les données sur la variable dépendante. Ces données ont été ensuite transformées en logarithme pour stabiliser

la variance. Les données climatiques mensuelles et les valeurs des cas de paludisme à un décalage (lag 1), ainsi que des données sur la population, ont été utilisées comme variables explicatives.

Une analyse de corrélation a montré que la température moyenne, la vitesse moyenne du vent, les précipitations totales et l'humidité relative avaient une corrélation significative avec les cas de paludisme. L'analyse de l'autocorrélation a montré que les décalages 1 et 2 étaient significatifs, tandis que l'analyse de corrélation croisée a montré des corrélations significatives pour différents décalages des variables climatiques (Figure 2.1).

Predictor (lagged)	Lag-0	Lag-1	Lag-2	Lag-3	Lag-4	Lag-5
Mean temperature	0.65	0.72*	0.57	0.30	− 0.04	− 0.34
Average wind speed	0.53*	0.52	0.42	0.19	− 0.10	− 0.36
Rainfall	0.41	0.18	− 0.09	− 0.38	− 0.54	− 0.55*
Relative humidity	0.32	0.11	− 0.15	− 0.44	− 0.62	− 0.63*
NDVI	0.07	− 0.29	− 0.60	− 0.76*	− 0.73	− 0.56
Population	− 0.25*	− 0.24	− 0.24	− 0.23	− 0.22	− 0.20
Malaria cases	1	0.85*	0.57	0.25	− 0.06	− 0.29

FIGURE 2.1 – Analyse de correlation croisée mentionnée dans cette étude

Deux modèles ont été utilisés : la régression polynomiale et le modèle SARIMA. Le modèle SARIMA a montré une meilleure performance en termes de AIC et aussi de R^2 pour les données de test, tandis que la régression polynomiale a montré de meilleures performances pour le RMSE, le MAE et le MAPE sur les données d'entraînement et de test, et aussi pour le R^2 sur les données d'entraînement. Finalement, la régression polynomiale a été choisie pour sa meilleure interprétabilité (Figure 2.2).

Model	AIC	For fitted (training) values				For forecasted (testing) values			
		R^2	RMSE	MAPE	MAE	R^2	RMSE	MAPE	MAE
Polynomial model	− 252.3	0.87	0.10	2.82	0.07	0.83	0.16	5.27	0.14
SARIMA model	− 242.6	0.90	0.09	2.34	0.06	0.81	0.13	4.17	0.10

AIC Akaike's information criterion, RMSE root mean square errors, MAPE mean absolute percentage error, MAE mean absolute error

FIGURE 2.2 – Résultats des modèles utilisés dans cette étude

Les résultats ont montré que les cas de paludisme à un décalage et la population avaient une forte influence sur l'incidence de la maladie. Pour les facteurs climatiques, la température moyenne, les précipitations et le NDVI ont également joué un rôle significatif dans l'augmentation des cas de paludisme et ont été considéré comme des variables importantes dans cette étude (Figure 2.3).

$$\begin{aligned}
&\text{Log(malaria cases)} \\
&= 1.434e + 01 + (-5.019e - 01 * \text{malaria-at-lag-one}) \\
&\quad + (4.534e - 01 * (\text{malaria-at-lag-one})^2) \\
&\quad + (-6.181e - 02 * (\text{malaria-at-lag-one})^3) \\
&\quad + (-1.176e + 00 * (\text{NDVI-at-lag-three})^2) \\
&\quad + (-1.918e - 01 * \text{mean temperature-at-lag-one}) \\
&\quad + (3.852e - 03 * (\text{mean temperature-at-lag-one})^2) \\
&\quad + (5.186e - 04 * \text{rainfall-at-lag-five}) \\
&\quad + (-4.364e - 07 * (\text{rainfall-at-lag-five})^2) \\
&\quad + (-4.592e - 06 * \text{Population}) \\
&\quad + (5.404e - 13 * \text{I(Population}^2)).
\end{aligned}$$

FIGURE 2.3 – Les variables les plus importants dans le modèle du cette étude

L'approche utilisée dans cette étude offre une méthode performante et interprétable pour prédire les cas de paludisme, ce qui serait utile pour les autorités sanitaires dans la prise de décisions en matière de prévention

et de contrôle de la maladie. Si l'on devait examiner ses avantages, on peut citer l'utilisation d'une méthode efficace de sélection des variables pour la construction du modèle de régression polynomiale (sélection avant), l'utilisation de la méthode de corrélation croisée pour démontrer plus précisément la relation entre les variables explicatives et la variable dépendante, l'utilisation du modèle SARIMA pour capturer la saisonnalité des données, et l'utilisation de données sur d'autres facteurs climatiques tels que le NDVI et la vitesse moyenne du vent.

Cependant, cette étude présente quelques limitations, notamment la non-utilisation de données sur les facteurs sociodémographiques et géospatiaux, qui pourraient avoir un impact significatif sur la prévalence du paludisme. De plus, la non-utilisation d'un modèle plus puissant comme les réseaux de neurones ou les LSTMs pourrait limiter la capacité du modèle à capturer des relations plus complexes qui peuvent exister dans les données.

Article 2 : Predicting Infectious Disease Using Deep Learning and Big Data (Chae, Kwon et Lee (2018))

L'article suivant aborde le problème de la réactivité et de la précision des prévisions des maladies infectieuses en Corée du Sud, en raison de rapports manquants et retardés. Pour améliorer ces prévisions, les auteurs ont utilisé des données massives, y compris des données des médias sociaux, pour prédire les maladies infectieuses telles que la varicelle, la scarlatine et le paludisme en utilisant des techniques avancées d'apprentissage profond et les réseaux de neurones.

Les données sur les maladies infectieuses ont été collectées à partir du système de statistiques web sur les maladies infectieuses, comprenant le nombre quotidien de personnes infectées dans toute la Corée du Sud. Les variables explicatives comprenaient les données de requête de recherche de Naver Data Lab (Un moteur de recherche beaucoup plus utilisé en Corée du Sud que Google), à savoir le nombre quotidien des recherches sur les maladies étudiées ou leur symptômes, les données massives des médias sociaux de Twitter API, à savoir le nombre quotidien des publications (tweets) sur les maladies étudiées ou leur symptômes, ainsi que la température et l'humidité de 576 jours entre le 1er janvier 2016 et le 29 juillet 2017, collectées depuis le portail d'informations météorologiques ouvert de l'Administration météorologique de la Corée (Figure 2.4).

Variable	Source	Description	Number of Observations
Occurrences	KCDC	Daily number of confirmed infectious disease diagnoses	576
Naver	Naver Data Lab	Daily Naver search frequency	
Twitter	Twitter	Daily number of Twitter mentions	
Temperature	KMA	Average daily temperature for all of South Korea	
Humidity		Average daily humidity for all of South Korea	

FIGURE 2.4 – Les données utilisés dans cette étude

Des modèles divers ont été construits et comparés, notamment des modèles basés sur la méthode du réseau des neurones profond (DNN), de la mémoire à court et long terme (LSTM), de la moyenne mobile intégrée autorégressive (ARIMA) et des moindres carrés ordinaires (OLS). Les performances des modèles ont été évaluées en utilisant différentes métriques, telles que le RMSE. Les modèles OLS et ARIMA ont été entraînés en 80% des données tandis que l'autre 20% était réservé pour le test. Pour les modèles DNN et LSTM, ils ont été entraînés avec 60% des données tandis que 20% des données a été utilisé pour l'ajustement des hyperparamètres optimaux (Hyperparameter tuning) et 20% pour le test (Figure 2.5).

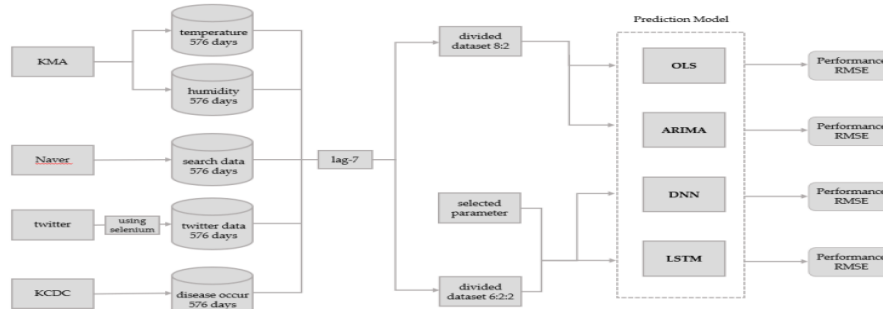


FIGURE 2.5 – Le schéma de la phase de modélisation utilisé dans l'étude

Les résultats ont montré que pour la varicelle, le modèle LSTM était le plus performant, pour la scarlatine, le modèle DNN était le plus performant, et pour le paludisme, le modèle LSTM était le plus performant. Les modèles DNN et LSTM ont amélioré les performances par rapport au modèle ARIMA pour toutes les maladies. Les chiffres entre parenthèses indiquent l'optimiseur, la fonction d'activation et le nombre d'époques pour l'entraînement de chaque modèle, respectivement. (optimiseur) 1 : Adadelta, 2 : Adagrad, 3 : Adam, 4 : Adamax, 5 : Nadam, 6 : RMSProp, et 7 : SGD, (fonction d'activation) 1 : ELU, 2 : ReLU, 3 : SELU, et 4 : SoftPlus, (nombre d'époques) 1 : 400, 2 : 600, 3 : 800, et 4 : 1000.

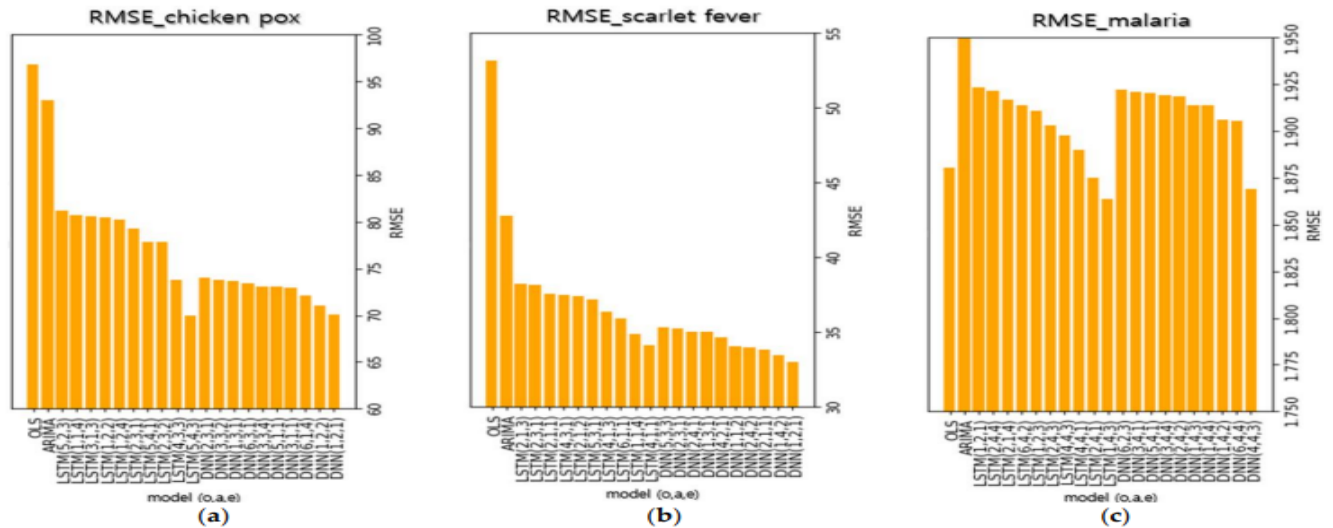


FIGURE 2.6 – Les résultats des modèles utilisés dans cette étude. (a) Les dix meilleurs modèles d'apprentissage profond et les modèles d'analyse traditionnels avec les RMSE les plus bas pour la varicelle, (b) Les dix meilleurs modèles d'apprentissage profond et les modèles d'analyse traditionnels avec les RMSE les plus bas pour la scarlatine, (c) Les dix meilleurs modèles d'apprentissage profond et les modèles d'analyse traditionnels avec les RMSE les plus bas pour le paludisme.

Pour les avantages de cette approche, nous avons constaté que l'utilisation de modèles puissants comme le DNN et le LSTM, l'utilisation de l'approche de réglage des hyperparamètres pour améliorer les performances des modèles DNN et LSTM, ainsi que l'utilisation de données massives provenant de requêtes de recherche et des médias sociaux pour améliorer les prédictions étaient toutes des approches formidables à utiliser pour ce type de problème. Cependant, l'étude présente ses limites, notamment le non-utilisation d'autres données climatiques importantes telles que les données sur les précipitations ou l'indice de végétation, qui pourraient avoir un impact significatif sur la prévalence des maladies infectieuses. De plus, l'étude n'a pas utilisé de données sociodémographiques, qui auraient pu fournir des informations supplémentaires sur les facteurs de risque de ces maladies. Enfin, bien que les modèles DNN et LSTM aient montré de bonnes performances, leur complexité pourrait poser problème lorsqu'il s'agit d'expliquer leurs prédictions à un public non technique.

Article 3 : The efficacy of deep learning based LSTM model in forecasting the outbreak of contagious diseases (Absar, Nurul, et al. (2022))

Cet article tacle la problématique de la pandémie de COVID-19 au Bangladesh, en mettant en l'accent sur le manque d'une politique de santé complète pour faire face à cette crise. Pour aider lutter contre cette maladie, les auteurs ont utilisé des données quotidiennes sur les cas confirmés, les décès et les cas guéris de la COVID-19 au Bangladesh, collectées du site Web du gouvernement, pour la période de mars 2020 à août 2021 pour les utiliser dans l'entraînement des modèles d'apprentissage automatique. Ensuite, les données ont été préparées dans Python et transformées en ondelettes pour atténuer le bruit aléatoire.

Pour prédire les tendances de la COVID-19 au Bangladesh, ils ont choisi d'utiliser des réseaux LSTM (Long Short Term Memory). Les données ont été prétraitées à l'aide de MinMaxScaler. Le modèle LSTM comprend une couche dense, 50 couches cachées, et utilise l'optimiseur Adam avec la fonction de perte *Mean Squared Error*. Le modèle LSTM a été entraîné sur 70% des données et les 30% restants sont utilisés pour la validation. (Figure 2.7)

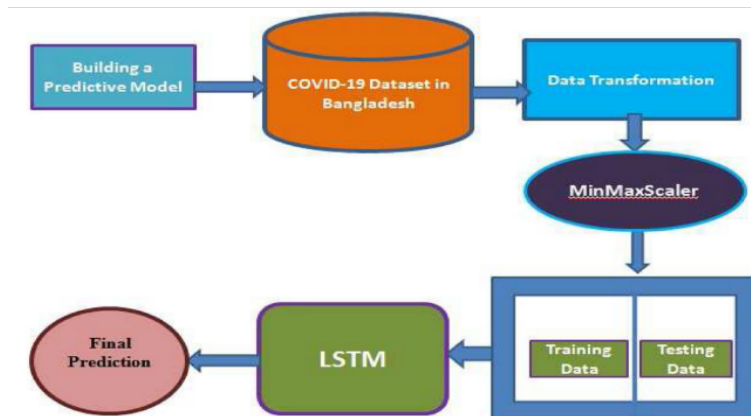


FIGURE 2.7 – L’architecture du modèle utilisé dans cette étude

Pour évaluer la performance du modèle, l’erreur quadratique moyenne (RMSE) a été utilisée comme métrique statistique standard. Les résultats montrent que le modèle LSTM a obtenu des valeurs d’RMSE de 1830.44, 3872.71 et 77.47 pour les cas confirmés, les cas de guérison et les décès respectivement sur l’ensemble de validation pour 80 epochs, de 1978.97, 4112.06 et 91.22 pour 100 epochs, de 2801.56, 3573.72 et 86.18 pour 120 epochs, et de 5422.95, 3942.23 et 59.35 pour 150 epochs. Les auteurs ont conclu que leur modèle LSTM performe significativement bien vis-à-vis des résultats obtenus par les études précédentes sur ce problème (Figure 2.8).

RMSE comparison with present and previous data for confirmed cases, recovered cases, and death cases.

Sr. No	No. of Epoch	RMSE			References
		Confirmed Cases	Recovered Cases	Death Cases	
01	80	1830.44	3872.71	77.47	Present Study (Bangladesh)
	100	1978.97	4112.06	91.22	
	120	2801.56	3573.72	86.18	
	150	5422.95	3942.23	59.35	
02	–	593	1113	36	Hridoy et al. (2020)
03	–	301.99	123.8794	4.306	Duttaet al. (2020)
04	–	3677.0	802.2	266.5	Karim et al. (2019)
05	–	2156.2	609.7	127.2	Dutta & Bandyopadhyay (2020)
06	–	53.38	71.2	4.97	

FIGURE 2.8 – Les résultats du modèle utilisé dans cette étude

Nous avons estimé que l’approche suivante présentait plusieurs avantages significatifs. Tout d’abord, elle offre un modèle mieux adapté à ce type de problème par rapport à d’autres techniques traditionnelles, ce qui en fait un outil précieux pour prédire la propagation du COVID-19 au Bangladesh. En outre, nous avons constaté que l’utilisation de la transformation en ondelettes pour réduire le bruit dans les données était une approche créative, car elle permet au modèle LSTM de devenir plus robuste aux variations et aux fluctuations, améliorant ainsi sa fiabilité dans les prédictions. Cependant, cette approche présente également des limitations, notamment le fait que le modèle LSTM doit être continuellement mis à jour et ajusté pour refléter les changements dans la situation épidémiologique, ce qui peut être complexe tant du point de vue computationnel que des coûts. Enfin, nous avons constaté que le non-recours à une approche offrant une interprétation augmenterait l’incertitude quant à la crédibilité d’un modèle aussi complexe pour gérer les prédictions de cas d’une pandémie aussi importante.

Article 4 : Integration of machine learning algorithms and GIS-based approaches to cutaneous leishmaniasis prevalence risk mapping (Shabanpour, Negar, et al. (2022))

Cet article vise la prédiction spatiale de la leishmaniose cutanée dans la province d’Isfahan, en Iran, en utilisant des algorithmes d’apprentissage automatique. L’objectif est d’identifier les facteurs environnementaux affectant l’occurrence de la leishmaniose cutanée et de préparer une carte des risques pour contrôler et gérer cette maladie. Les données utilisées dans cette étude sont les données sur le nombre de cas de leishmaniose cutanée dans la province d’Isfahan de 2011 à 2018, obtenus depuis Infectious Diseases Management Center (IDMC), ainsi que dix paramètres environnementaux : température, humidité, précipitations, altitude, pente, vitesse du vent, indice de végétation par différence normalisée (NDVI), nombre de jours ensoleillés, nombre de jours de gel, et distance par rapport au cours d’eau.

L'étude s'est déroulée en quatre étapes : développement d'une base de données spatiale, analyses de l'autocorrélation spatiale et de la multicollinéarité, utilisation de la méthode "Fuzzy Logic", et modélisation spatiale avec trois algorithmes d'apprentissage automatique (Arbre de décision (DT), Machine à Vecteurs de Support (SVR) et La Régression Linéaire (LR)). L'évaluation des modèles s'est faite à l'aide de l'erreur quadratique moyenne et de l'indice ROC.

L'analyse de la multicollinéarité a été effectuée pour éviter des coefficients de modèle invalides. Dans l'analyse de la multicollinéarité, le coefficient de variance (VIF) doit être inférieur à 5. Selon le VIF, les valeurs de tous les paramètres étaient inférieures à 5, donc il n'y avait pas de collinéarité entre les variables indépendantes dans cette étude, et toutes ces variables pouvait être utilisées dans la modélisation (Figure 2.9).

Factors	Tolerance	VIF
NDVI	0.883	1.132
Distance to stream	0.81	1.234
Slope	0.729	1.372
Humidity	0.492	1.821
Wind speed	0.431	2.21
Temperature	0.761	1.411
altitude	0.732	1.834
Rainfall	0.425	2.02
Frosty days	0.569	1.693
Sunny days	0.801	1.389

FIGURE 2.9 – Les résultats de l'analyse de la multicollinéarité utilisé dans cette étude

L'autocorrélation spatiale a été évaluée à l'aide des indices de Moran et Getis-ord G_i^* pour identifier les zones à haut risque. En 2016 et 2017, l'hypothèse selon laquelle la distribution de la maladie est aléatoire a été acceptée en se basant sur l'indice de Moran (valeur $P > 0,05$). Selon les résultats, la distribution de la maladie dans la région de 2011 à 2015 et en 2018 était en grappe et les résultats des statistiques de valeur p (p-value) avait montrait la significativité des résultats de l'autocorrélation spatiale (Figure 2.10).

Year	Index value	variance	Expected value	Z-score	P-value	Type of distribution
2011	0.173	0.006	-0.047	20.65	<0.05	Cluster
2012	0.227	0.006	-0.047	30.39	<0.05	Cluster
2013	0.131	0.007	-0.047	20.07	<0.05	Cluster
2014	0.148	0.007	-0.047	20.22	<0.05	Cluster
2015	0.044	0.006	-0.047	10.12	<0.05	Cluster
2016	0.1	0.006	-0.047	10.87	>0.05	Random
2017	0.095	0.006	-0.047	10.87	>0.05	Random
2018	0.2	0.007	-0.047	20.88	<0.05	Cluster

FIGURE 2.10 – Les résultats de l'analyse d'autocorrélation spatiale utilisé dans cette étude

La méthode "Fuzzy Logic", ou la méthode de la logique floue, est une méthode qui permet d'incorporer des jugements individuels et ambigus sur un phénomène unique dans des modèles probabilistes ou mathématiques. Contrairement à la logique classique, qui n'a que deux valeurs (0 et 1), la logique floue représente la valeur d'un élément comme un pourcentage d'appartenance à un ensemble flou, généralement dans l'intervalle (0, 1). Cette méthode est surtout appréciée pour sa meilleure prise en compte de l'incertitude et de la variabilité des données, ce qui peut améliorer la précision des modèles prédictifs. C'est pourquoi elle a été utilisée dans cette étude.

Pour la prédiction des cas, les "Fuzzy values" des variables environnementaux et des données sur la leishmaniose ont été utilisées comme valeurs d'entrée pour la modélisation. 70% des données totales ont été utilisées pour l'entraînement, les 30% restants pour la validation. Les résultats montrent que le modèle le plus performant été l'arbre de décision (DT) avec un RMSE de 0.228 sur la base des données d'entraînement et 0.289 sur les données de validation, bien supérieur au modèle SVR qui a donné un RMSE de 0.274 sur les données d'entraînement et 0.324 sur les données de validation et aussi au modèle de régression linéaire qui a donné un RMSE de 0.292 sur les données d'entraînement et 0.347 sur les données de validation (Figure 2.11).

Model	Training	Validation
DT	0.228	0.289
SVR	0.274	0.324
LR	0.292	0.347

FIGURE 2.11 – Les résultats des modèles utilisés dans cette étude

Les résultats de modélisation des algorithmes d'apprentissage automatique ont été ensuite utilisés pour créer des cartes de risque de la leishmaniose cutanée. La technique de catégorisation des ruptures naturelles a été utilisée pour diviser les cartes de prédiction en cinq classifications de risque, allant de très faible à très élevé. L'évaluation des cartes de prédiction de la leishmaniose montre que pour les algorithmes DT, SVR et LR, la valeur AUC indiquait une précision de 0,951, 0,934 et 0,914, respectivement. L'algorithme DT était plus précis dans la modélisation de la leishmaniose que les deux autres algorithmes, selon les résultats des indices RMSE et ROC (Figure 2.12).

Algorithms	AUC	Std. Error	Asymptotic Sig0.	Asymptotic 95% Confidence Interval	
				Lower Bound	Upper Bound
DT	0.951	0.025	0.000	0.862	0.99
SVR	0.939	0.032	0.000	0.845	0.984
LR	0.914	0.036	0.000	0.82	0.97

FIGURE 2.12 – L'évaluation des cartes de risque de leishmaniose créés par les modèles utilisés dans cette étude

L'approche utilisée dans cette étude présente plusieurs avantages. Tout d'abord, nous avons constaté que l'utilisation de données environnementales et d'algorithmes d'apprentissage automatique permet de modéliser de manière précise et performante la distribution spatiale de la leishmaniose cutanée. De plus, l'analyse de la multicollinéarité et l'utilisation de la méthode "Fuzzy Logic" permettent de réduire l'incertitude associée aux données et d'évaluer l'impact des facteurs environnementaux sur la prévalence de la maladie. Nous pouvons également noter que l'utilisation de la méthode "Fuzzy Logic" peut également offrir plus d'interprétabilité au modèle.

D'autre part, nous avons constaté certaines limitations dans cette étude. Par exemple, nous avons constaté que l'approche utilisée peut être sensible aux paramètres choisis pour la méthode "Fuzzy Logic" et aussi au ajustement des hyperparamètres des algorithmes d'apprentissage automatique.

Article 5 : Employing Machine Learning Techniques for the Malaria Epidemic Prediction in Ethiopia (Chekol, B. E., & Hagaras, H. (2018))

Cet article tacle le problème de précision dans la prédiction des cas des épidémies de malaria en Éthiopie, en soulignant que malgré l'utilisation d'un système de surveillance et d'une analyse conventionnelle pour prédire et donner une alerte précoce, des techniques avancées sont nécessaires pour prédire les épidémies futures avec précision. Pour offrir une solution pertinente, les auteurs ont utilisé des techniques de machine learning accompagnée d'un modèle transparent pour améliorer l'interprétabilité du modèle ML utilisé. Les données utilisées dans cette étude comprennent 5 ans (2013-2017) des données climatiques (pluie, humidité relative, température moyenne, élévation) accompagné d'un variable de décalage (1 lag) des cas de malaria et comme variable dépendant des cas de malaria passés (Figure 2.14).

	Variable Name	Range	Description
Inputs	ELV	287-300 meters	The elevation where meteorology agency station and Woreda is found
	RF	0-900	Monthly Rainfall in milliliter
	RH	20.08-96.97	Relative Humidity in percent
	TMP	5.4-43.6	Average monthly temperature in centigrade
	LMC	0-111	Number of positive malaria confirmed persons per month/1000 population one month lag
Output	CASES	0-111	Number of positive malaria confirmed persons per month/1000 population(current month)

FIGURE 2.13 – Les données utilisés dans cette étude

Dans cette étude, deux modèles ont été utilisés pour prédire les épidémies de malaria en Éthiopie : un modèle de boîte noire SVM et un modèle de boîte transparente ANFIS. L'Adaptive Neuro Fuzzy Inference System (ANFIS) est une approche d'apprentissage automatique qui combine les avantages des systèmes d'inférence qui utilisent l'approche "Fuzzy Logic" et qui offrent une meilleure interprétabilité par rapport aux modèles complexes et des réseaux neuronaux pour créer un modèle capable de modéliser des systèmes complexes.

Les deux modèles ont été évalués à l'aide des métriques RMSE (Root Mean Square Error) et R² (Coefficient de détermination), et les données ont été normalisées en utilisant une approche de min-max. Les modèles ont été entraînés sur 66% des données, validés sur 17%, et testés sur les 17% restants. Les résultats ont montré que les deux modèles étaient performants sur les données d'entraînement et de test. Le modèle SVR a affiché un RMSE de 0.04 sur les données d'entraînement et de 0.12 sur les données de validation, avec un R² de 0.72 sur les données d'entraînement et de 0.69 sur les données de validation. Le modèle ANFIS a donné un RMSE de 0.049 sur les données d'entraînement et de 0.133 sur les données de validation, avec un R² de 0.67 sur les données d'entraînement et de 0.63 sur les données de validation (Figure 2.15).

	SVR		ANFIS	
	Training	Testing	Training	Testing
RMSE	0.04	0.12	0.049	0.133
R ²	0.72	0.69	0.67	0.63

FIGURE 2.14 – Les résultats des modèles utilisés dans cette étude

Nous constatons que l'approche présentée offre plusieurs avantages, notamment une performance relativement élevée dans la prédiction des épidémies de malaria jusqu'à trois mois à l'avance. De plus, le modèle ANFIS offre une transparence dans le raisonnement derrière les prédictions, ce qui peut être utile pour comprendre et interpréter les facteurs influençant les épidémies de malaria en Éthiopie.

Cependant, nous avons également remarqué quelques limitations. Par exemple, cette approche repose fortement sur la capacité des modèles à généraliser à de nouvelles données. De plus, l'interprétabilité du modèle hybride proposé dépend de la proximité, voire de l'identité, des prédictions du modèle boîte noire (SVM) et du modèle transparent (ANFIS). Si tel n'est pas le cas, les résultats du modèle SVM ne peuvent pas être interprétés, ce qui peut limiter leur utilité dans certaines applications où la transparence du modèle est importante.

Article 6 : Modeling climate change impacts on vector-borne disease using machine learning models : Case study of Visceral leishmaniasis (Kala-azar) from Indian state of Bihar (Kumar, S., Srivastava, A., & Maity, R. (2024))

Cet article met en lumière la nécessité d'utiliser les techniques d'apprentissage automatique les plus récentes pour prédire les cas de maladies parasitaires. Cela permet non seulement d'améliorer l'efficacité, mais aussi la rapidité, en exploitant la capacité de ces algorithmes à modéliser des relations complexes entre les variables explicatives et la variable dépendante. L'objectif principal était de prédire efficacement les épidémies de leishmaniose viscérale ou Kala-azar (KA), une maladie parasitaire majeure, dans l'État du Bihar en Inde, spécifiquement dans les districts de Muzaffarpur et Saran. Pour ce faire, des données climatiques couvrant une période de sept ans, de janvier 2016 à juillet 2021, ont été utilisées, notamment les précipitations totales (Pluie), la température moyenne (T_{avg}), la température maximale (T_{max}), la température minimale (T_{min}), la vitesse du vent (WS), la densité de population ($P_{density}$) et l'humidité relative (RH), ainsi que des données cliniques sur les cas confirmés de KA.

Les auteurs ont mené une analyse de corrélation entre la variable dépendante (cas confirmés de KA) et les variables explicatives pour chacun des districts de Muzaffarpur et Saran afin de mieux comprendre la relation linéaire entre elles. Cette analyse a révélé que pour le district de Muzaffarpur, la variable des cas de KA est corrélée positivement avec la température maximale (T_{max}), la température minimale (T_{min}), la température moyenne (T_{moy}) et la vitesse du vent (VS). En revanche, elle est corrélée négativement avec l'humidité relative (HR), les précipitations (Pluie) et la densité de population ($P_{density}$)(Figure 2.15).

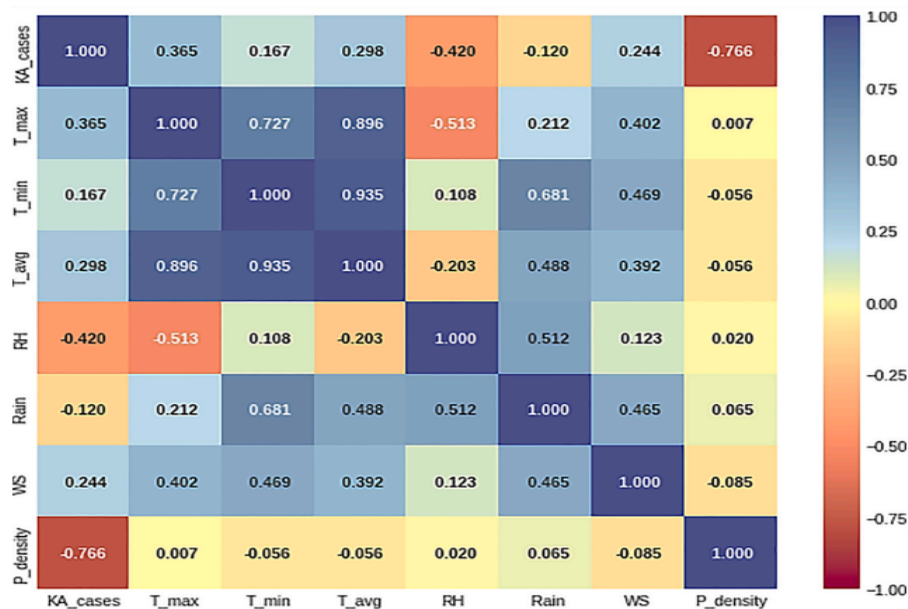


FIGURE 2.15 – La matrice de corrélation pour la districte de Muzaffarpur mentionnée dans cette étude

Pour la districte de Saran, les résultats montrent que le modèle de corrélation est similaire à Muzaffarpur, mais avec des corrélations légèrement plus fortes pour la température minimale et la vitesse moyenne de vent et des corrélations légèrement plus faible pour les autres variables (Figure 2.16).

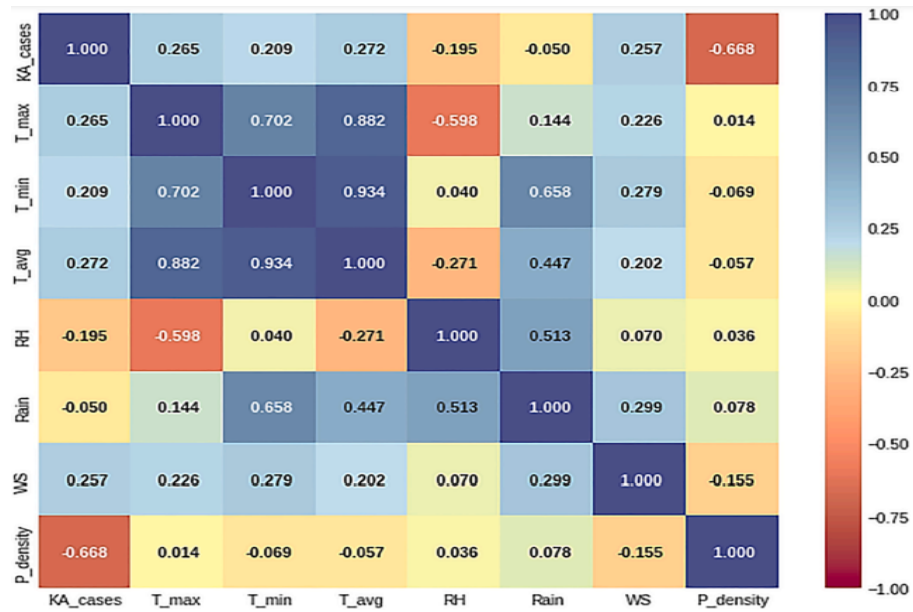


FIGURE 2.16 – La matrice de corrélation pour la district de Saran mentionnée dans cette étude

Dans la phase de modélisation, les auteurs ont utilisé trois modèles : le modèle SVR (Support Vector Regressor), le modèle MLR (Régression Linéaire Multiple) et le modèle MLP (Multi-layer Perceptron). Les modèles ont été entraînés sur 67% des données et testés sur les 33% des données restantes, en utilisant la méthode de validation croisée à 3 blocs (folds) pour mieux évaluer la stabilité des modèles. Pour l'évaluation des modèles, trois métriques ont été utilisées : l'erreur quadratique moyenne (RMSE), le coefficient de corrélation de Pearson et le coefficient d'efficacité de Nash-Sutcliffe (NSE). Les résultats montrent que le modèle SVR a été le modèle le plus performant en moyenne sur les 3 blocs (folds) sur les données de test pour les trois métriques utilisées, tandis que le modèle MLR a été le modèle le plus performant sur la base des données d'entraînement pour les trois métriques utilisées. Le modèle SVR a été choisi comme solution en raison de sa capacité à généraliser et à mieux performer sur les données de test.

Fold	Model	CC		RMSE		NSE	
		Trn	Tst	Trn	Tst	Trn	Tst
1	SVR	0.84	0.81	12.43	11.91	0.69	0.66
	MLR	0.84	0.78	12.48	19.67	0.69	0.07
	MLP	0.84	0.82	12.58	12.39	0.69	0.63
2	SVR	0.86	0.78	12.83	9.69	0.73	0.61
	MLR	0.84	0.78	13.79	9.91	0.68	0.59
	MLP	0.89	0.75	12.18	10.62	0.75	0.53
3	SVR	0.81	0.85	11.01	15.01	0.65	0.71
	MLR	0.83	0.89	10.59	13.02	0.68	0.78
	MLP	0.85	0.85	10.51	15.83	0.68	0.67
Mean	SVR	0.84	0.82	12.09	12.20	0.69	0.66
	MLR	0.86	0.81	11.75	12.95	0.71	0.61
	MLP	0.83	0.81	12.29	14.20	0.68	0.48

FIGURE 2.17 – Les résultats des modèles utilisés dans cette étude

Cette étude présente plusieurs avantages, comme l'utilisation de la validation croisée à 3 blocs, qui aide à limiter le risque de sur-apprentissage et confirme la capacité des modèles à généraliser à de nouvelles données qu'ils n'ont jamais vues auparavant. De plus, nous avons constaté que l'utilisation de métriques d'évaluation pertinentes, telles que l'erreur quadratique moyenne (RMSE), le coefficient de corrélation de Pearson et le coefficient d'efficacité de Nash-Sutcliffe (NSE), permet également une meilleure évaluation de la performance des modèles en termes d'erreur, de corrélation et d'efficacité de la simulation.

D'autre part, cette approche présente également des limitations. Le choix et l'optimisation des hyperparamètres des modèles (SVR, MLR, MLP) peuvent influencer significativement les résultats, car des choix inappropriés peuvent limiter la performance. De plus, étant donné qu'ils ne sont pas couplés à des méthodes d'interprétation, les modèles utilisés peuvent être plus difficiles à interpréter que d'autres, limitant la compréhension des relations entre les variables et la prédiction des cas.

Article 7 : A deep learning based hybrid architecture for weekly dengue incidences forecasting (Zhao, Xinxing, et al. (2023))

Cet article propose une nouvelle approche basée sur l'apprentissage profond pour relever les défis posés par la maladie de la dengue. Cette maladie a un impact négatif considérable sur la santé humaine et les économies mondiales, soulignant ainsi le besoin urgent de mécanismes plus efficaces pour prédire les cas de dengue, en particulier dans les régions tropicales et subtropicales. Les données utilisées dans cette étude comprennent les cas de dengue hebdomadaires rapportés pour une période de 10 ans, de la semaine 24 (10-16 juin) de 2012 à la semaine 21 (15-21 mai) de 2022, obtenus à partir du site Web du ministère de la Santé de Singapour, ainsi que les températures moyennes hebdomadaires en °C et les précipitations hebdomadaires en mm.

Les modèles utilisés dans cette étude comprennent les réseaux de neurones récurrents (RNN) et les réseaux de neurones convolutionnels (CNN), ainsi que leurs variantes telles que les LSTM et les GRU. Les CNN sont généralement associés au traitement d'images, mais ont également été adaptés avec succès à de nombreux problèmes liés aux séries temporelles, tels que la prévision des cas de COVID-19 et des séries temporelles financières. L'article propose une architecture hybride profonde combinant ces deux types de modèles, en utilisant des modules convolutionnels et récurrents (Figure 2.18).

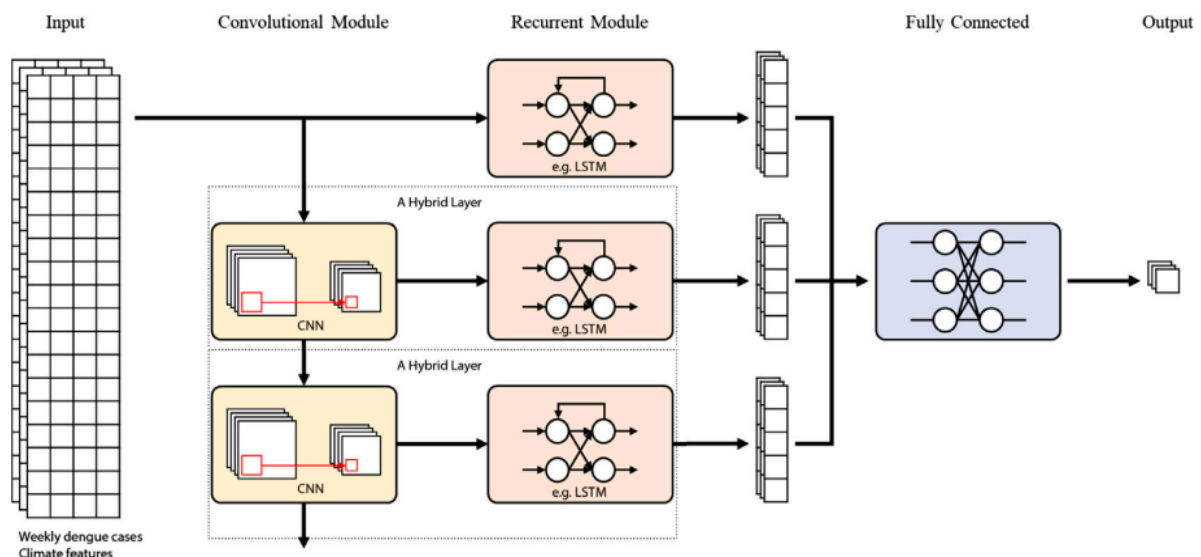


FIGURE 2.18 – L'architecture du modèle proposé dans cette étude

Dans l'architecture proposée, chaque module CNN comporte une partie convolutionnelle 2D avec une taille de noyau de 3 et un padding de 1. Pour chaque module récurrent dans l'architecture, les modèles RNN et ses variantes (GRU, LSTM et BiLSTM) sont utilisés avec un seul type de modèle à chaque fois. Par exemple, si le modèle LSTM est utilisé, tous les modules récurrents sont avec des modèles LSTM (Figure 2.19).

Module	Parameters & settings
Recurrent Module	3 layers, each with 50 hidden units, dropout rate 0.25, activation function ReLu.
CNN Module	Convolutional 2D, kernel size (3, 1) padding (1, 0).

FIGURE 2.19 – Les configurations des modules utilisées dans l'architecture proposée

Les configurations pour chaque module récurrent et pour les modèles LSTM, BiLSTM, RNN et GRU (modèles de référence) sont partagées : 3 couches, 50 unités cachées dans chaque couche, taux de dropout de 0,25 et fonction d'activation ReLu. Pour le CNN de référence, il comporte 3 couches convolutionnelles, chaque couche avec une taille de noyau de 2 et un pas de 2, taux de dropout de 0,25, ReLu comme fonction d'activation. Pour le ConvLSTM de référence, il comporte 3 couches convolutionnelles avec une taille de noyau de 3, un padding de 1, 2 couches de max-pooling, et la partie LSTM comporte 3 couches, 50 unités cachées chacune, taux de dropout de 0,25 dans chaque couche, et ReLu comme fonction d'activation (Figure 2.20).

Model	Parameters & settings
RNN and variants	3 layers, each with 50 hidden units, dropout rate 0.25, activation function ReLu.
CNN model	3 convolutional layers, kernel size 2 and stride 2, dropout rate of 0.25, activation function ReLu.
ConvLSTM model	3 convolutional layers, kernel size 3, padding 1, 2 max-pooling layers, LSTM part has 3 layers, 50 hidden units each, dropout rate 0.25, activation function ReLu.

FIGURE 2.20 – Les configurations des modèles de références utilisées dans cette étude

Pour l'entraînement de ces modèles, 70% des données ont été utilisées pour l'entraînement, tandis que les 30% restants ont été réservés pour les tests. Les modèles ont ensuite été évalués en utilisant les métriques d'erreur quadratique moyenne, d'erreur moyenne absolue et de pourcentage moyen absolu d'erreur. Les résultats montrent que les modèles hybrides d'apprentissage profond ont généralement une meilleure performance en termes d'erreur quadratique moyenne (RMSE), d'erreur moyenne absolue (MAE) et de pourcentage moyen absolu d'erreur (MAPE) par rapport aux modèles de référence. Les modèles ont été entraînés à la fois avec des données météorologiques et sans données météorologiques, et pour des prévisions à une semaine, deux semaines, trois semaines et quatre semaines. Pour la prévision à une semaine, les modèles hybrides ont atteint une précision de prévision allant jusqu'à 87,72% (1-MAPE) sans l'utilisation de données météorologiques (Figure 2.21).

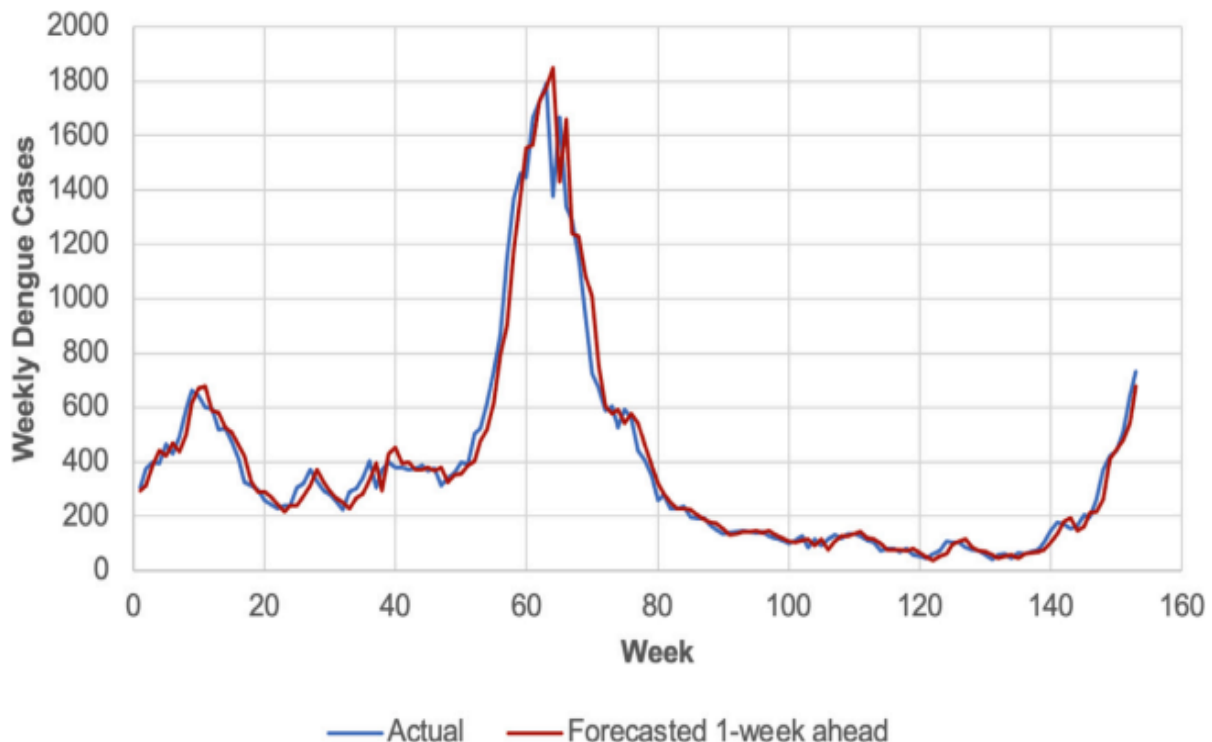


FIGURE 2.21 – la performance du modèle hybride CNN-BiLSTM pour la prévision à une semaine

Pour la prévision à deux semaines, la précision était de 84,61%, et pour la prévision à quatre semaines, elle était de 78,28% (Figure 2.22). Les modèles hybrides, en particulier le modèle CNN-BiLSTM avec une seule pile, ont généralement donné les meilleurs résultats en termes de MAPE, MAE et RMSE, montrant ainsi leur efficacité dans la prévision des cas de dengue.

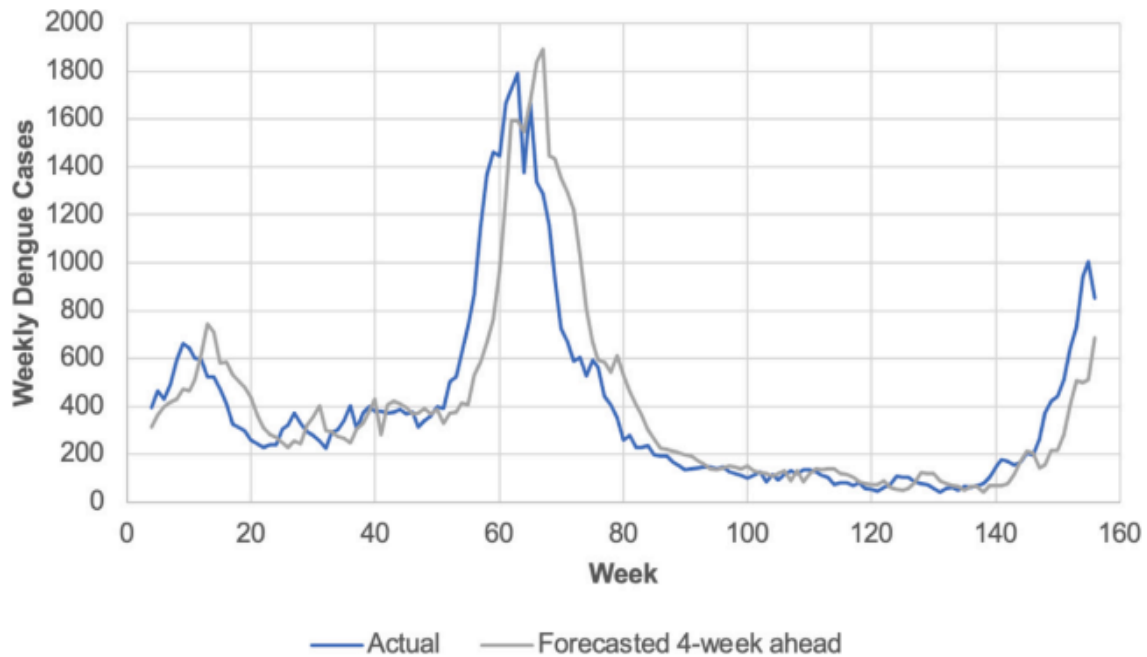


FIGURE 2.22 – la performance du modèle hybride CNN-BiLSTM pour la prévision à quatre semaine

L'approche hybride de modélisation en apprentissage profond pour la prévision des cas de dengue proposée dans cette étude présente plusieurs avantages, notamment une meilleure performance prédictive par rapport aux modèles de référence, grâce à la capture efficace des caractéristiques spatiales et temporelles des données. Nous avons constaté aussi que les modèles hybrides sont également adaptatifs à différents horizons de prévision, ce qui les rend polyvalents pour différents besoins de planification. Cependant, cette approche présente quelques limites, notamment la complexité computationnelle plus élevée des modèles utilisés qui nécessitent beaucoup de ressources informatiques, et un manque d'interprétabilité en raison de leur complexité. De plus, les modèles hybrides nécessitent des données de qualité pour produire des prévisions précises, ce qui peut limiter leur applicabilité en cas de données incomplètes ou de mauvaise qualité.

Article 8 : A multivariate multi-step LSTM forecasting model for tuberculosis incidence with model explanation in Liaoning Province, China (Yang, Enbin, et al. (2022))

Cet article propose une approche innovante pour améliorer la prédiction de l'incidence de la tuberculose (TB) en Chine en utilisant des techniques d'apprentissage automatique et profond. L'objectif de cette étude était de concevoir des modèles de prévision qui capturent les relations complexes entre la maladie et d'autres facteurs que le temps seul, afin d'améliorer la précision des prédictions. Les données utilisées comprennent les données mensuelles d'incidence de la TB dans la province du Liaoning de 2005 à 2017, ainsi que des données météorologiques, comme la température, l'humidité, la durée d'ensoleillement, les précipitations, la pression de l'air, etc. et économiques et sociales mensuelles correspondantes, comme l'indice des prix à la consommation (IPC), le nombre de touristes, la production industrielle, etc.

La méthodologie de l'étude comprend plusieurs étapes. Tout d'abord, les données sont nettoyées pour assurer leur cohérence et éliminer les valeurs aberrantes. Ensuite, les données manquantes sont traitées et imputées en utilisant l'algorithme k-Nearest Neighbor (kNN). La sélection des variables est effectuée en utilisant la méthode LASSO pour éliminer les facteurs non pertinents, suivie du classement de l'importance des variables par Random Forest. Les modèles ont été évalués par trois métriques différents : l'erreur quadratique moyen, l'erreur absolue moyen, le pourcentage absolue moyen d'erreur et le pourcentage absolue moyen d'erreur symétrique.

Pour la modélisation, les données sont divisées en un ensemble d'entraînement (132 mois) et un ensemble de test (24 mois). Les modèles ARIMA et SARIMA sont utilisés comme modèles de base pour les comparer

avec les modèles avancés LSTM multivariés multi-étapes et aussi les modèle hybride multivariés et muti-étapes ARIMA-LSTM. Un système de prédiction interprétable basé sur le modèle le plus performant et la méthode SHapley Additive exPlanation (SHAP) est proposé pour fournir des prédictions précises et explicables.

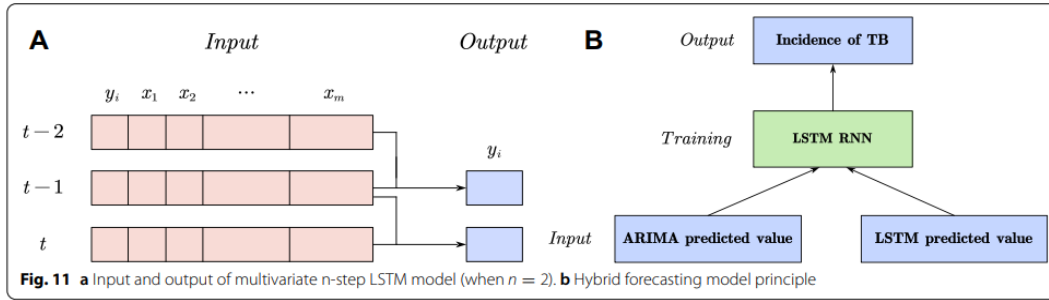


FIGURE 2.23 – A. le format des données utilisés pour l’entrainement des modèles mutivariés muti-étapes LSTM, B. L’architecture du modèle hybride multivarié muti-étapes ARIMA-LSTM

Les résultats montrent une amélioration significative de la précision par rapport aux modèles ARIMA, en particulier avec le modèle ARIMA-LSTM en termes d’erreur réduite. Le modèle 3-étapes ARIMA-LSTM se distingue en atteignant d’excellentes performances, avec une réduction significative des erreurs pour les prévisions à moyen et long termes. Par exemple, chaque erreur du modèle LSTM 2-étapes multivarié est réduite de 12,92%, 15,94%, 15,97% et 14,81% respectivement pour les prévisions à 6, 12 et 24 mois. Le modèle 3-étapes ARIMA-LSTM atteint des performances exceptionnelles, avec une réduction de chaque erreur à 15,19%, 33,14%, 36,79% et 29,76% respectivement pour les prévisions à moyen et long termes (Figure 2.23).

Model	RMSE	MAE	MAPE (%)	sMAPE (%)
6-step ahead prediction between January 2016 to June 2016				
A	0.3244 (–)	0.2811 (–)	6.0454 (–)	5.8097 (–)
B	1.0157 (+ 213.10%)	0.9339 (+ 232.23%)	20.0035 (+ 230.89%)	17.9006 (+ 208.12%)
C*	0.2825 (– 12.92%)	0.2363 (– 15.94%)	5.0797 (– 15.97%)	4.9490 (– 14.81%)
D*	0.4659 (+ 43.62%)	0.3206 (+ 14.05%)	6.8661 (+ 13.58%)	7.4156 (+ 27.64%)
12-step ahead prediction between January 2016 and December 2016				
A	0.4425 (–)	0.3917 (–)	9.7674 (–)	9.1462 (–)
B	0.7825 (+ 63.40%)	0.6508 (+ 66.15%)	14.5400 (+ 48.86%)	13.2301 (+ 44.65%)
C*	0.4060 (– 8.25%)	0.3073 (– 21.55%)	7.8076 (– 20.06%)	7.5203 (– 17.78%)
D*	0.3753 (– 15.19%)	0.2619 (– 33.14%)	6.1742 (– 36.79%)	6.4240 (– 29.76%)
24-step ahead prediction between January 2016 and December 2017				
A	0.4672 (–)	0.4177 (–)	9.9328 (–)	9.3198 (–)
B	0.7634 (+ 63.40%)	0.6384 (+ 52.84%)	14.1518 (+ 42.48%)	13.0495 (+ 40.02%)
C*	0.4108 (– 12.07%)	0.3295 (– 21.12%)	7.7436 (– 22.04%)	7.4895 (– 19.64%)
D*	0.4042 (– 13.48%)	0.3070 (– 26.50%)	6.9664 (– 29.86%)	7.2245 (– 22.48%)

The data format $x(y)$, x is the error value and y is the percentage change compared to the ARIMA model. Particularly, (–) indicates the null value. A is the ARIMA model and B is the SARIMA model. The new model proposed in this paper is labeled by superscript *. C* is the multivariate 2-step LSTM model and D* is the 3-step ARIMA-LSTM hybrid forecasting model

FIGURE 2.24 – Les résultats des modèles utilisés dans cette étude

Selon notre jugement, l’approche présentée dans l’article offre plusieurs avantages, notamment une meilleure performance en matière de prévision par rapport à des méthodes plus traditionnelles comme les modèles ARIMA, surtout pour les prévisions à moyen et long terme, ce qui peut aider à améliorer la gestion de l’incidence de la tuberculose (TB) en Chine. De plus, nous avons constaté que l’utilisation de la méthode SHAP aide à expliquer les prédictions du modèle, ce qui peut aider les non-techniciens à comprendre les facteurs influençant l’incidence de la TB. En outre, en intégrant des données météorologiques, économiques et sociales, l’approche capture les relations complexes entre la maladie et divers facteurs, améliorant ainsi la précision des prévisions.

Cependant, cette approche présente quelques limitations, notamment une complexité computationnelle accrue en raison de l’utilisation de modèles LSTM multivariés à plusieurs étapes et de la méthode SHAP. De plus, nous avons constaté que l’interprétation subjective des résultats SHAP est un aspect à prendre en compte.

2.4.2 Tableau comparatif et Discussion

Nous avons effectué une comparaison qualitative des travaux présentés dans la Section 2.4.1, que nous avons résumé dans le Tableau 2.1. Cette comparaison s’est fait sur plusieurs critères, notamment *les modèles utilisés, les données explicatives utilisées, les différentes méthodes utilisées, la complexité computationnelle, l’interprétabilité, les métriques d’évaluation et les performances du modèle choisi dans chaque étude*.

Après avoir examiné les solutions existantes, nous avons tiré plusieurs conclusions. Tout d’abord, nous avons constaté que l’utilisation de données climatiques, sociodémographiques et géospatiales peut être dans certains cas essentielle pour modéliser les cas de maladies infectieuses, en raison du lien fort entre ces maladies et ces facteurs. Cela peut également être le cas pour la maladie que nous étudions, la leishmaniose. Cependant, certaines études ont également montré que l’utilisation uniquement de données climatiques peut donner de très bonnes performances.

Ensuite, nous avons également remarqué qu’une analyse exploratoire des données est une étape préliminaire essentielle dans la construction de modèles d’apprentissage automatique et d’apprentissage profond efficaces. Cette approche nous aide souvent à mieux comprendre nos données, ce qui conduit à la création de modèles plus solides qui correspondent mieux à nos données. Les méthodes utilisées dans l’analyse exploratoire des données, telles que l’analyse de corrélation et l’analyse de corrélation croisée, se sont montrées utiles car elles permettent la création de nouvelles variables explicatives, améliorant ainsi les capacités prédictives du modèle, notamment lorsqu’on dispose d’un ensemble limité de facteurs explicatifs.

Dans le prétraitement des données, la normalisation et l’imputation à l’aide de techniques telles que le KNN nous semblent être des stratégies efficaces lorsque nous avons des données complexes et incomplètes. Les études ont montré que ces approches aident à maintenir l’intégrité des données et garantissent que les modèles sont entraînés sur des ensembles de données complets, améliorant ainsi leur précision.

En termes de modélisation, nous avons remarqué qu’une variété d’approches peuvent être utilisées pour ce type de problématique. Elles vont des modèles traditionnels comme SARIMA et ARIMA, souvent utilisés, aux modèles plus avancés d’apprentissage automatique (ML) et de deep learning (DL) tels que la Régression par Vecteurs de Support (SVR), les modèles polynomiaux, les Réseaux Neuronaux Convolutifs (CNN), les Réseaux Neuronaux Récurrents (RNN), les Mémoires à Long Terme et Court Terme (LSTM), les Unités Récurrentes à Portes (GRU), et les LSTM Bidirectionnels (BiLSTMs) qui sont également populaires et très efficaces pour prédire les cas de maladies infectieuses. Nous avons aussi constaté que les modèles hybrides comme ARIMA-LSTM, CNN-LSTM, et CNN-BiLSTM deviennent de plus en plus populaires ces dernières années en raison de leur capacité à capturer à la fois les motifs linéaires et non linéaires dans les données.

Nous avons également conclu que l’interprétabilité est un facteur important à considérer dans la construction d’une solution pour ce type de problème, principalement en raison de la nécessité d’expliquer les prédictions des modèles à des personnes non techniques telles que les autorités sanitaires, afin qu’elles puissent comprendre les facteurs qui ont conduit les modèles à prédire un nombre élevé ou faible de cas de la maladie infectieuse. À cet égard, nous avons constaté que la méthode SHAP (SHapley Additive exPlanations) émerge comme l’une des meilleures techniques pour obtenir à la fois une explication globale et locale des modèles. Cette méthode permet de surmonter la complexité des modèles performants, fournissant des informations précieuses sur leur comportement. De plus, l’utilisation de modèles explicatifs tels que les systèmes d’inférence neuro-flous adaptatifs (ANFIS) peut également être utilisée pour améliorer l’interprétabilité des modèles plus complexes.

Enfin, selon notre jugement, les méthodes comme l’analyse de l’autocorrélation spatiale et la logique floue peuvent également être des outils précieux pour analyser et interpréter les données spatiales, améliorant ainsi l’efficacité globale des modèles d’apprentissage profond et automatique dans la prédiction des cas des maladies infectieuses.

Conclusion

La prédiction des cas de leishmaniose est un processus crucial qui peut contribuer de manière significative à la gestion efficace de cette maladie. Dans ce chapitre, nous avons tenté de développer les objectifs et les défis de notre problématique, à savoir la prédiction des cas de maladies infectieuses, en mettant en avant les avantages de la solution que nous pouvons offrir pour aider les décideurs et les responsables du contrôle de l’épidémie de leishmaniose à Ouarzazate. Ensuite, nous avons effectué une recherche approfondie pour réaliser une analyse très détaillée des solutions existantes pour notre problématique, en couvrant chaque méthode utilisée dans huit articles de recherche différents dans ce domaine, tout en présentant les avantages et les limitations de chaque solution. En examinant en détail les solutions existantes et les approches de modélisation, on peut conclure qu’en utilisant des techniques avancées telles que l’apprentissage profond, notamment les réseaux LSTM et BiLSTM, il est possible d’améliorer considérablement la précision des prédictions des cas des maladies infectieuses, s’il est couplé avec une méthode d’explication des prédictions comme SHAP, cette solution peut être la meilleure pour notre problématique. Dans le chapitre suivant, nous aborderons l’analyse exploratoire de nos données afin de bien les comprendre.

Article	Modèles utilisés	Données explicatives utilisées	Méthodes utilisées	Complexité computationnelle	Interprétabilité	Métriques d'évaluation	Performance du modèle choisi
Mopuri et al. (2020)	Modèle polynomiale, SARIMA	Climatiques et socio-démographiques	Analyse de corrélation, corrélation croisée et d'autocorrélation, Transformation en logarithme,	Faible	Moyenne	AIC, R^2 , RMSE, MAPE et MAE	Bonnes performances
Chae, Kwon et Lee (2018)	DNN, LSTM, OLS, ARIMA	Climatiques et de réseaux sociaux	Analyse d'autocorrélation et de régression, Méthode de recherche sur grille (Hyperparameter Tuning)	Elevée	Faible	RMSE	Performances moyennes
Absar, Nurul, et al. (2022)	LSTM	Série chronologique	Transformation des données en ondelettes, Normalisation Min-Max	Elevée	Faible	RMSE	Bonnes performances
Shabanpour, Negar, et al. (2022)	Arbre de décision, Support Vector Regression, Régression Linéaire	Climatiques et Géospatiaux	Méthode de "Fuzzy Logic", Analyse de multicollinéarité et d'autocorrélation spatiale, Méthode de catégorisation des ruptures naturelles	Moyenne	Moyenne	RMSE, AUC	Bonnes performances
Chekol, B. E., et Hagrass, H. (2018)	ANFIS, Support Vector Regression	Climatiques	Méthode de "Fuzzy Logic"	Moyenne	Moyenne	RMSE, R^2	Performances moyennes
Kumar, S., Srivastava, A., et Maity, R. (2024)	Support Vector Regression, Régression Linéaire Multiple, MLP	Climatiques et socio-démographiques	Méthode de validation croisée, Analyse de corrélation,	Moyenne	Faible	RMSE, Coefficient de corrélation et NSE	Bonnes performances
Zhao, Xin-xing, et al. (2023)	LSTM, BiLSTM, CNN, GRU, RNN, ConvLSTM, CNN-LSTM, CNN-BiLSTM	Climatiques	Méthode de validation croisée, Hyperparameter tuning	Elevée	Faible	RMSE, MAE et MAPE	Bonnes performances
Yang, Enbin, et al. (2022)	ARIMA, SARIMA, LSTM, ARIMA-LTSM	Climatiques et socioéconomiques	KNN pour imputation, Nettoyage des valeurs aberrantes, LASSO, Méthode SHAP	Elevée	Elevée	RMSE, MAE, MAPE, sMAPE	Bonnes performances

TABLE 2.1 – Tableau comparatif des solutions analysées

Chapitre 3

Analyse Exploratoire des données

Introduction

Dans ce chapitre, nous abordons l'analyse exploratoire de nos données afin de comprendre les motifs présents et d'extraire des informations utiles qui pourront nous aider lors de la phase de modélisation. Tout d'abord, nous décrivons le processus de collecte des données cliniques et climatiques, puis nous abordons les étapes de prétraitement nécessaires pour transformer les données en une matrice de données prête à l'analyse et à la modélisation. Ensuite, nous effectuons l'analyse univariée et multivariée de nos variables.

3.1 Collecte des données

Pour construire un modèle de ML pour prédire les cas de leishmaniose, la quantité et la qualité des données sont essentielles. Comme mentionné dans la section 1.1.4, les données doivent contenir suffisamment d'informations et être en quantité adéquate. Notre problème relève de la catégorie des problèmes d'apprentissage automatique appelée l'apprentissage supervisé. Ce type d'apprentissage repose sur deux types de variables : une variable dépendante et une ou plusieurs variables explicatives. L'objectif est, à partir des données sur les variables explicatives, d'estimer la variable dépendante. Dans notre cas, la variable dépendante est généralement une variable qui varie dans un intervalle de temps spécifique, indiquant le nombre de cas de leishmaniose dans la région de Ouarzazate pour chaque intervalle de temps. Les données sur ce type de variable peuvent être classées dans la catégorie des données épidémiologiques lorsqu'elles sont présentées sous la forme souhaitée. Sinon, elles peuvent être dérivées à partir des données cliniques des patients atteints de la maladie, en agrégeant chaque cas signalé pour chaque intervalle de temps sélectionné.

Pour la collecte des données de cas de leishmaniose, nous avons opté pour une approche efficace en collectant les données cliniques d'un des hôpitaux de Ouarzazate, Sidi Hsain, pour enfin les transformer en un variable qui donne les cas de leishmaniose dans toute la région de Ouarzazate pour chaque mois pour la période de 2010 à 2022. Ces données se présentaient sous forme de matrice, contenant des informations telles que la date du diagnostic, la méthode de diagnostic, la commune, l'état de la lésion, etc. et qui comprennent seulement les cas positifs (Figure 3.1). Cette étape de collecte a été essentielle car elle nous a permis d'obtenir des données brutes directement issues du terrain, reflétant la réalité des cas de leishmaniose dans la région.

N°		Année	Commune	Secteur	Quartier ou localité	Milieu	Sexe	Age en année	Type de dépistage	Date diagnostic	...	Etat de la lésion	Grossesse	Date début de TTT	Type de TTT	Evolution après TTT	Classification	Effets indésérables	Nombre de prise TTT	Observation
0	1	2018	Ouarzazate	Sidi Daoud	Taourirt	U	F	24.0	P	2017-12-27 00:00:00	...	EG	I	2018-01-02 00:00:00	PL	G	A	Non	8	NaN
1	2	2018	Ouarzazate	Sidi Daoud	Ait Kdif	U	M	4.0	P	2017-12-27 00:00:00	...	EG	NaN	2018-01-02 00:00:00	PL	G	A	Non	8	NaN
2	3	2018	Ouarzazate	Sidi Daoud	Sidi Daoud	U	M	35.0	P	2018-01-02 00:00:00	...	EG	NaN	2018-01-02 00:00:00	PL	G	A	Non	8	NaN
3	4	2018	Ouarzazate	Sidi Daoud	Ait Kdif	U	F	25.0	P	2018-01-02 00:00:00	...	EG	I	2018-01-02 00:00:00	PL	G	A	Non	8	NaN
4	5	2018	Ouarzazate	Sidi Daoud	Ait Kdif	U	M	31.0	P	2018-01-05 00:00:00	...	A	NaN	2018-01-05 00:00:00	PL	G	A	Non	8	NaN
5 rows × 20 columns																				

FIGURE 3.1 – Les données cliniques obtenus au sein d'hôpital Sidi Hsein

La collecte des données à partir de Sidi Hsain a été un processus minutieux, nécessitant une coordination étroite avec le personnel médical pour garantir la qualité et la précision des données recueillies. Les variables qui nous intéressent le plus dans cette matrice et qui vont nous aider pour créer notre variable dépendante sont les suivants :

1. **Date Diagnostic** : Cette variable représente la date à laquelle le patient a été diagnostiqué, ce qui est essentiel pour extraire le mois de diagnostic et agréger les cas pour chaque mois.
2. **Année** : Cette variable représente l'année à laquelle le patient a été diagnostiqué, ce qui est essentiel pour agréger les cas pour chaque mois dans chaque année.
3. **Commune** : Cette variable désigne la commune dans la province de ouarzazate à laquelle le patient appartient, ce qui va nous aider à agréger les cas selon chaque commune pour notre analyse.
4. **Milieu** : Cette variable désigne si la commune dans la région de ouarzazate à laquelle le patient appartient s'agit d'un milieu Urbain (U) ou Rurale (R), ce qui va nous aider à agréger les cas selon chaque milieu pour notre analyse.
5. **Age** : Cette variable représente l'âge de chaque patient, ce qui va nous aider à agréger les cas selon chaque intervalle d'âge pour notre analyse.
6. **Sexe** : Cette variable représente le sexe de chaque patient, ce qui va nous aider à agréger les cas selon chaque sexe pour notre analyse.

Pour les variables explicatives, dans notre cas les variables climatiques, sociodémographiques, socioéconomiques et géospatiales peuvent avoir une force explicative impressionnante, surtout lorsque l'on cherche à estimer le nombre de cas d'une maladie qui est certainement influencée par ces types de facteurs, comme mentionné dans la section 1.1.2. Les données climatiques peuvent être collectées au sein des organisations qui fournissent des données climatiques mensuelles de Ouarzazate, y compris Météo Maroc, ou bien d'autres organisations qui fournissent des interfaces spécialisées en météo et climat mondiale, qui peuvent fournir des données climatiques mensuelles de la province de Ouarzazate.

Pour la collecte de ces données climatiques, nous avons utilisé NASA POWER (Prediction of Worldwide Energy Resource), un système en ligne développé par la NASA qui fournit des données sur les ressources énergétiques renouvelables, telles que l'énergie solaire et éolienne, à l'échelle mondiale. Le système utilise des modèles basés sur des données satellitaires pour estimer la disponibilité et la variabilité de ces ressources énergétiques dans différentes régions du monde. À l'aide de ce système, nous avons pu collecter des données météorologiques mensuelles telles que les températures minimales et maximales, l'humidité, les précipitations mensuelles, la vitesse moyenne du vent, la quantité moyenne de nuages, etc. de chaque commune dans la province de Ouarzazate pour la période de 2010 à 2022 (Figure 3.2).

	PARAMETER	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC	ANN	MUNICIPALITY
0	PS	2010	82.99	82.67	83.06	82.82	82.91	82.95	83.34	83.39	83.27	83.02	82.90	82.90	83.02	Taznakhte
1	T2M	2010	6.45	8.48	10.45	14.87	16.39	21.87	27.16	25.83	22.21	14.79	10.90	7.30	15.59	Taznakhte
2	QV2M	2010	3.91	5.55	4.82	5.68	4.58	5.19	6.04	6.77	7.32	6.04	4.46	4.58	5.43	Taznakhte
3	RH2M	2010	57.69	69.75	55.50	49.31	37.38	31.62	24.50	30.50	40.06	52.00	50.00	62.19	46.56	Taznakhte
4	WD2M	2010	311.50	277.88	293.44	265.31	305.88	325.44	171.94	171.06	234.12	291.62	286.88	255.31	286.38	Taznakhte

FIGURE 3.2 – Exemple des données climatiques obtenue depuis NASA POWER de la commune de Taznakhte

On a essayé de collecter le plus grand nombre possible de variables climatiques, ce qui a conduit à la collecte d'une matrice contenant finalement 11 variables climatiques :

1. **Pression de surface (kPa)** : La pression atmosphérique au niveau du sol, mesurée en kilopascals (kPa), indique la force exercée par l'air atmosphérique sur une unité de surface.
2. **Température minimale à 2 mètres** : La température la plus basse enregistrée à 2 mètres au-dessus du sol pendant une période donnée, généralement mesurée en degrés Celsius.
3. **Température maximale à 2 mètres** : La température la plus élevée enregistrée à 2 mètres au-dessus du sol pendant une période donnée, également mesurée en degrés Celsius.
4. **Vitesse moyenne du vent à 2 mètres (m/s)** : La vitesse à laquelle le vent souffle à 2 mètres au-dessus du sol, mesurée en mètres par seconde.
5. **Direction du vent à 2 mètres (degrés)** : L'orientation du vent à 2 mètres au-dessus du sol, mesurée en degrés par rapport au nord.
6. **UVA (W/m^2)** : L'intensité du rayonnement ultraviolet de type A, mesurée en watts par mètre carré.
7. **UVB (W/m^2)** : L'intensité du rayonnement ultraviolet de type B, mesurée en watts par mètre carré.
8. **Précipitations totales en millimètres (mm)** : La quantité totale de précipitations (pluie, neige, grêle, etc.) qui est tombée sur une région donnée pendant une période de temps spécifique.
9. **Quantité moyenne de nuages (%)** : Le pourcentage du ciel couvert par les nuages, indiquant le degré de couverture nuageuse.

10. **Humidité relative à 2 mètres (%)** : Le pourcentage d'humidité dans l'air par rapport à la quantité maximale d'humidité que l'air peut contenir à une température donnée.
11. **Humidité du sol ()** : La quantité d'humidité contenue dans le sol, mesurée en grammes par kilogramme d'air.

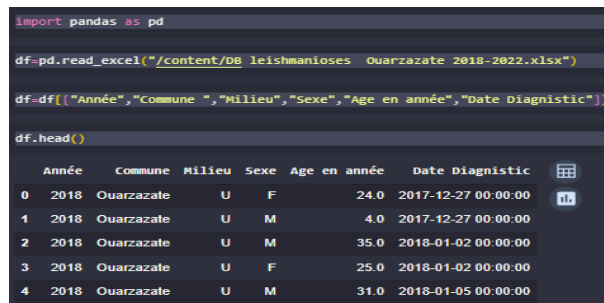
En ce qui concerne les variables sociodémographiques, socioéconomiques et géospatiales, nous n'avons pas pu les collecter pour plusieurs raisons. Notamment, ces données n'étaient pas disponibles dans les organisations que nous avons consultées, ou bien leur qualité était insuffisante lorsqu'elles étaient disponibles en ligne, par exemple à travers des services comme OpenStreetMap (OSM). Cela a rendu nécessaire la réalisation d'une enquête sur le terrain afin de garantir la qualité et l'intégrité de ces données. Heureusement, plusieurs études ont montré que les données sur les facteurs climatiques sont généralement suffisantes pour modéliser les cas de maladies infectieuses, y compris la leishmaniose, et obtenir de bonnes performances sans avoir besoin d'utiliser d'autres facteurs.

3.2 Prétraitement des données

Prétraitement des données cliniques

Afin de prétraiter les données cliniques collectées à l'hôpital Sidi Hsein sous le format souhaité, à savoir, la variable dépendante qui compte le nombre de cas de leishmaniose par mois pour chaque année, il est nécessaire de procéder aux étapes suivantes : sélectionner les variables d'intérêt uniquement pour supprimer les autres non nécessaires à l'analyse, extraire le mois de la date de diagnostic pour pouvoir l'utiliser avec l'année et ainsi agréger les cas de leishmaniose par mois pour chaque année et enfin, il est nécessaire aussi de créer une nouvelle variable de type *Datetime* pour représenter chaque observation dans nos nouvelles données, laquelle représente chaque mois et année de diagnostic.

Pour charger et prétraiter les données, nous avons utilisé la bibliothèque *Pandas* de Python, qui offre une variété d'opérations pour manipuler les données tabulaires. Tout d'abord, nous avons importé la bibliothèque et nous avons chargé les données stockées dans un fichier excel en utilisant la méthode *read_excel*, puis nous les avons stockées dans une variable appelée *df* pour pouvoir effectuer les modifications nécessaires. Ensuite, nous avons sélectionné uniquement les variables *Date Diagnostic*, *Année*, *Commune*, *Milieu*, *Sexe* et *Age*, que nous avons utilisées soit pour la phase d'entraînement, soit pour notre analyse exploratoire (Figure 3.3).



```
import pandas as pd

df=pd.read_excel("/content/DB leishmanioses Ouarzazate 2018-2022.xlsx")

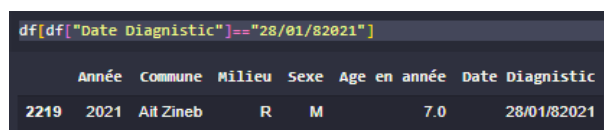
df=df[["Année","Commune ","Milieu","Sexe","Age en année","Date Diagnostic"]]

df.head()
```

	Année	Commune	Milieu	Sexe	Age en année	Date Diagnostic
0	2018	Ouarzazate	U	F	24.0	2017-12-27 00:00:00
1	2018	Ouarzazate	U	M	4.0	2017-12-27 00:00:00
2	2018	Ouarzazate	U	M	35.0	2018-01-02 00:00:00
3	2018	Ouarzazate	U	F	25.0	2018-01-02 00:00:00
4	2018	Ouarzazate	U	M	31.0	2018-01-05 00:00:00

FIGURE 3.3 – Chargement du jeu des données cliniques en sélectionnant les variables importantes

Après avoir chargé les données et sélectionné les variables nécessaires, nous devons extraire le mois de la variable "Date Diagnostic". Cependant, nous avons constaté que cette variable était de type *String*. Pour pouvoir extraire le mois avec Pandas, nous devons la convertir en type *Datetime*. Pour ce faire, nous avons essayé d'utiliser la méthode *to_datetime* de Pandas. Cependant, lors de la transformation, nous avons remarqué la présence de dates incorrectes, ce qui a empêché la méthode de fonctionner correctement. Nous avons donc dû corriger manuellement ces valeurs incorrectes pour que la méthode fonctionne, la figure 3.4 montre un exemple d'un cas erroné du date de diagnostic.



```
df[df["Date Diagnostic"]=="28/01/82021"]
```

	Année	Commune	Milieu	Sexe	Age en année	Date Diagnostic
2219	2021	Ait Zineb	R	M	7.0	28/01/82021

FIGURE 3.4 – Exemple d'une valeur erronée dans la date de diagnostic

Après avoir corrigé les valeurs erronées, on a ensuite utilisé la méthode *to_datetime* pour convertir le variable en type *Datetime* comme prévu. La figure 3.5 montre le processus dans le code.

```
df["Date Diagnostic"]=pd.to_datetime(df["Date Diagnostic"])
df.head()
```

	Année	Commune	Milieu	Sexe	Age en année	Date Diagnostic
0	2018	Ouarzazate	U	F	24.0	2017-12-27
1	2018	Ouarzazate	U	M	4.0	2017-12-27
2	2018	Ouarzazate	U	M	35.0	2018-01-02
3	2018	Ouarzazate	U	F	25.0	2018-01-02
4	2018	Ouarzazate	U	M	31.0	2018-01-05

FIGURE 3.5 – Transformation du variable de date de diagnostic en type *Datetime*

Ensuite, on a extrait le mois depuis la variable de date de diagnostic en utilisant l'attribue *month* des variables de type *Datetime* puis on a supprimé le variable de date de diagnostic parcequ'on n'en a plus besoin. La figure 3.6 montre le processus en code.

```
df['Mois']=df["Date Diagnostic"].dt.month
df["Mois"].iloc[0]=1
df["Mois"].iloc[1]=1
df.drop("Date Diagnostic",axis=1,inplace=True)
df.head()
```

	Année	Commune	Milieu	Sexe	Age en année	Mois
0	2018	Ouarzazate	U	F	24.0	1
1	2018	Ouarzazate	U	M	4.0	1
2	2018	Ouarzazate	U	M	35.0	1
3	2018	Ouarzazate	U	F	25.0	1
4	2018	Ouarzazate	U	M	31.0	1

FIGURE 3.6 – Extraction du mois depuis la variable de date de diagnostic de type *Datetime*

Après avoir obtenu le variable du mois du diagnostic, on l'a utilisé avec le variable d'année pour agréger les cas de leishmaniose et créer notre variable dépendante. Pour faire cela, on a utilisé la méthode *group_by* qui permet de agréger par plusieurs variables et *size* qui permet d'obtenir le nombre des cas de leishmaniose par mois et par année puis on les a stocké dans un variable *leish_cases* pour pouvoir les utiliser ultérieurement. La figure 3.7 montre le processus d'aggregation.

```
leish_cases=df.groupby(by=["Année","Mois"]).size().reset_index(name="Cases")
leish_cases.head()
```

	Année	Mois	Cases
0	2018	1	131
1	2018	2	146
2	2018	3	137
3	2018	4	30
4	2018	5	44

FIGURE 3.7 – Aggrégation des cas de leishmaniose par mois et par année

La dernière étape consistait à créer la variable *Date* pour représenter chaque mois et chaque année sous forme d'une seule variable de type *Datetime*. Pour ce faire, nous avons fusionné les valeurs de l'année de diagnostic avec celles du mois de diagnostic, en prenant le premier jour de chaque mois pour le jour. Ensuite, nous avons converti la variable *Date* en type *Datetime* en utilisant la méthode *to_datetime*, et nous l'avons utilisée comme index de notre jeu de données avec la méthode *set_index*. La figure 3.8 montre le processus.

```
leish_cases["Date"] = leish_cases["Année"].astype(str) + "-" + leish_cases["Mois"].astype(str) + "-1"
leish_cases["Date"] = pd.to_datetime(leish_cases["Date"])
leish_cases.set_index("Date", inplace=True)

leish_cases.head()
```

Date	Année	Mois	Cases
2018-01-01	2018	1	131
2018-02-01	2018	2	146
2018-03-01	2018	3	137
2018-04-01	2018	4	30
2018-05-01	2018	5	44

FIGURE 3.8 – Création du variable *Date* pour l'utiliser comme l'index de notre jeu des données

Prétraitement des données climatiques

Les données climatiques collectées depuis NASA POWER ont été fournies sous forme de plusieurs fichiers CSV, un pour chaque commune. Elles nécessitaient donc des prétraitements pour les rendre exploitables dans le cadre de notre analyse exploratoire et pour l'entraînement de notre modèle. Pour chaque commune, nous avons choisi de collecter les différentes valeurs des variables climatiques dont nous avons besoin pour chaque mois, sur la période de 2010 à 2022, à partir de trois points différents au sein de cette commune. Cette approche nous permet d'obtenir une couverture suffisante pour chaque commune.

Pour prétraiter ces données, la première étape que nous avons dû réaliser était de charger les données des 3 points différents au sein de chaque commune et de les concaténer en un seul tableau pandas. Pour ce faire, nous avons écrit le bloc de code présenté dans la figure 3.9. Ce code effectue plusieurs étapes pour agréger et fusionner des données climatiques de plusieurs points au sein de chaque commune de la province d'Ouarzazate. Tout d'abord, il prend en compte trois fichiers CSV différents, chacun contenant des données climatiques pour trois points distincts au sein de chaque commune. Ensuite, il fusionne ces trois fichiers CSV en un seul dataframe. Pour chaque mois et chaque année, le dataframe final contient les valeurs médianes de chaque variable climatique pour chaque commune. Enfin, il fusionne toutes les communes en un seul dataframe global qui contient l'ensemble des données climatiques agrégées pour la province d'Ouarzazate.

```
climate_data_full = pd.DataFrame()
for commune in communes :
    path = "/content/drive/MyDrive/Climat 2001-2018/" + commune + "/"
    # List all CSV files in the directory
    all_files = glob.glob(path + "*.csv")
    # Initialize an empty list to store the DataFrames
    dfs = []

    # Read each CSV file and append its DataFrame to the list
    for files in all_files:
        df = pd.read_csv(files, header=26)
        dfs.append(df)

    # Concatenate all DataFrames in the list into a single DataFrame
    combined_df = pd.concat(dfs, ignore_index=True)
    combined_df_final = pd.DataFrame()
    for month in ["JAN", "FEB", "MAR", "APR", "MAY", "JUN", "JUL", "AUG", "SEP", "OCT", "NOV", "DEC", "ANN"] :
        if month == "JAN" :
            combined_df_final = pd.concat([combined_df_final, pd.DataFrame(combined_df.groupby(["PARAMETER", "YEAR"], as_index=False)[month].median(),
                                                                           columns=["PARAMETER", "YEAR", month]), ignore_index=True, axis=1)
        else :
            combined_df_final = pd.concat([combined_df_final, pd.DataFrame(combined_df.groupby(["PARAMETER", "YEAR"], as_index=False)[month].median()[month],
                                                                           columns=[month]), ignore_index=True, axis=1)

    combined_df_final["commune"] = commune
    climate_data_full = pd.concat([climate_data_full, combined_df_final], axis=0, ignore_index=True)
climate_data_full.columns = ["PARAMETER", "YEAR", "JAN", "FEB", "MAR", "APR", "MAY", "JUN", "JUL", "AUG", "SEP", "OCT", "NOV", "DEC", "ANN", "commune"]

climate_data_full.head(3)
```

	PARAMETER	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC	ANN	commune
0	ALLSKY_SFC_UVA	2010	9.16	10.51	14.11	16.23	18.59	20.93	17.71	16.63	13.90	12.37	9.96	8.01	14.02	Tarnigle
1	ALLSKY_SFC_UVB	2010	0.16	0.20	0.32	0.38	0.45	0.53	0.47	0.44	0.35	0.27	0.19	0.14	0.33	Tarnigle
2	CLOUD_AMT	2010	29.10	49.11	41.03	45.41	30.30	13.38	45.38	45.58	51.69	32.52	22.49	38.12	36.95	Tarnigle

FIGURE 3.9 – Code de préparation du matrice des variables climatiques mensuelles de chaque commune dans la province de Ouarzazate

Après avoir obtenu notre matrice des variables climatiques mensuelles pour chaque commune, la prochaine étape consiste à la transformer en une matrice où les variables climatiques sont les colonnes et les valeurs correspondantes pour chaque mois et chaque année sont les lignes. Pour réaliser cette transformation, nous avons développé le code présenté dans la figure 3.10. Ce code parcourt les données climatiques pour chaque commune et année de la province d'Ouarzazate. Pour chaque commune et année, il itère sur les mois et extrait

les valeurs des différentes variables climatiques. Ces valeurs sont extraites en filtrant les données en fonction du mois, de l'année, de la commune et du type de paramètre climatique (tel que la pluie, la température, etc.). Les valeurs extraites sont ensuite stockées dans un tableau de données. Le résultat final est un tableau de données regroupant les valeurs médianes de chaque variable climatique pour chaque mois et chaque année pour chaque commune de la province d'Ouarzazate.

```

clm = []
for commune in climate_data_full.commune.unique():
    for annee in climate_data_full.YEAR.unique():
        for cnt, month in enumerate(["JAN", "FEB", "MAR", "APR", "MAY", "JUN", "JUL", "AUG", "SEP", "OCT", "NOV", "DEC", "ANN"], start=1):
            data = climate_data_full[(climate_data_full["YEAR"]==annee) & (climate_data_full["commune"]==commune)]
            values = {
                "Commune": commune,
                "Annee": annee,
                "Mois": cnt,
                "UVA": data.loc[data["PARAMETER"]=="ALLSKY_SFC_UVA", month].iloc[0],
                "UVB": data.loc[data["PARAMETER"]=="ALLSKY_SFC_UVB", month].iloc[0],
                "CLOUD_AMT": data.loc[data["PARAMETER"]=="CLOUD_AMT", month].iloc[0],
                "SW": data.loc[data["PARAMETER"]=="GWETTOP", month].iloc[0],
                "PRECT_TOTAL": data.loc[data["PARAMETER"]=="PRECTOTCORR_SUM", month].iloc[0],
                "SP": data.loc[data["PARAMETER"]=="PS", month].iloc[0],
                "SH2M": data.loc[data["PARAMETER"]=="QV2M", month].iloc[0],
                "RH2M": data.loc[data["PARAMETER"]=="RH2M", month].iloc[0],
                "T2M_MAX": data.loc[data["PARAMETER"]=="T2M_MAX", month].iloc[0],
                "T2M_MIN": data.loc[data["PARAMETER"]=="T2M_MIN", month].iloc[0],
                "WD2M": data.loc[data["PARAMETER"]=="WD2M", month].iloc[0],
                "WS2M": data.loc[data["PARAMETER"]=="WS2M", month].iloc[0]
            }
            clm.append(values)

climate_data_ouarzazate = pd.DataFrame(clm)

climate_data_ouarzazate.head()

```

	Commune	Annee	Mois	UVA	UVB	CLOUD_AMT	SW	PRECT_TOTAL	SP	SH2M	RH2M	T2M_MAX	T2M_MIN	WD2M	WS2M
0	Tarmigte	2010	1	9.16	0.16	29.10	0.45	31.64	84.48	3.78	53.62	22.58	-5.27	319.75	3.09
1	Tarmigte	2010	2	10.51	0.20	49.11	0.52	68.55	84.12	5.25	63.31	24.00	-2.33	283.25	3.66
2	Tarmigte	2010	3	14.11	0.32	41.03	0.52	21.09	84.54	4.70	50.81	26.70	-0.00	298.81	3.52
3	Tarmigte	2010	4	16.23	0.38	45.41	0.41	0.00	84.32	5.49	44.38	29.47	3.89	252.25	2.78
4	Tarmigte	2010	5	18.59	0.45	30.30	0.28	0.00	84.41	4.78	36.88	31.51	4.23	320.75	3.85

FIGURE 3.10 – Code de transformation du matrice des variables climatiques mensuelles de chaque commune dans la province de Ouarzazate

Pour conclure le prétraitement des données climatiques, il nous reste à calculer les moyennes mensuelles de chaque variable climatique pour l'ensemble des communes de la province de Ouarzazate. Pour ce faire, nous avons élaboré le code présenté dans la figure 3.11, qui prend la matrice des variables climatiques mensuelles de chaque commune et calcule les moyennes mensuelles pour les différentes variables climatiques. Ensuite, il regroupe ces moyennes dans un nouveau tableau de données où chaque ligne représente un mois et une année spécifiques, avec les moyennes des variables climatiques correspondantes. Les données sont ensuite transformées pour avoir une indexation temporelle avant de retirer les colonnes "Année" et "Mois" pour obtenir le tableau de données final prêt pour l'analyse.

```

clm={"Annee": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["UVA"]].mean().reset_index()["Annee"],
      "Mois": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["UVA"]].mean().reset_index()["Mois"],
      "UVA": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["UVA"]].mean().reset_index()["UVA"],
      "UVB": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["UVB"]].mean().reset_index()["UVB"],
      "CLOUD_AMT": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["CLOUD_AMT"]].mean().reset_index()["CLOUD_AMT"],
      "SOIL_W": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["SW"]].mean().reset_index()["SW"],
      "PRECT_TOTAL": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["PRECT_TOTAL"]].sum().reset_index()["PRECT_TOTAL"],
      "SURFACE_P": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["SP"]].mean().reset_index()["SP"],
      "SH2M": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["SH2M"]].mean().reset_index()["SH2M"],
      "RH2M": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["RH2M"]].mean().reset_index()["RH2M"],
      "TMP_MAX": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["T2M_MAX"]].mean().reset_index()["T2M_MAX"],
      "TMP_MIN": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["T2M_MIN"]].mean().reset_index()["T2M_MIN"],
      "WD2M": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["WD2M"]].mean().reset_index()["WD2M"],
      "WS2M": climate_data_ouarzazate.groupby(by=["Annee","Mois"])[["WS2M"]].mean().reset_index()["WS2M"]}

climate_data_final=pd.DataFrame(clm)
climate_data_final.index=pd.to_datetime(climate_data_final["Annee"].astype(str)+"-"+climate_data_final["Mois"].astype(str)+"-1")
climate_data_final.drop(["Annee","Mois"],axis=1,inplace=True)

climate_data_final.head()

```

	UVA	UVB	CLOUD_AMT	SOIL_W	PRECT_TOTAL	SURFACE_P	SH2M	RH2M	TMP_MAX	TMP_MIN	WD2M	WS2M
2001-01-01	9.090588	0.150588	28.959412	0.467847	348.06	83.980000	3.569412	54.427059	20.314706	-2.917059	321.630000	2.688471
2001-02-01	11.787947	0.237059	28.531785	0.395294	0.00	83.944708	2.958235	42.924118	20.387647	-2.464706	293.387647	2.445294
2001-03-01	14.914118	0.347059	35.861785	0.330588	79.11	83.672941	3.849412	41.082353	27.087647	0.442941	282.144118	3.092353
2001-04-01	17.697947	0.405294	31.861785	0.234118	10.55	83.956471	3.611178	32.287647	28.293529	1.055294	309.947647	2.781785
2001-05-01	19.468824	0.485294	34.163529	0.170588	0.00	83.582941	4.257647	35.397647	33.701785	0.088235	321.540000	3.418235

FIGURE 3.11 – Code de préparation du matrice finale des données climatiques mensuelles de la province de Ouarzazate

3.3 Exploration Préliminaire des Données

Après avoir prétraité les données climatiques et cliniques, les données sont désormais prêtes pour l'analyse et la modélisation. Avant de commencer l'analyse univariée et multivariée des données, nous allons faire une présentation générale de nos données afin de bien les comprendre pour la suite.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 156 entries, 2010-01-01 to 2022-12-01
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   RH2M        156 non-null    float64
 1   TMP_MIN     156 non-null    float64
 2   TMP_MAX     156 non-null    float64
 3   WS2M        156 non-null    float64
 4   WD2M        156 non-null    float64
 5   UVA         156 non-null    float64
 6   UVB         156 non-null    float64
 7   CLOUD_AMT   156 non-null    float64
 8   SOIL_W      156 non-null    float64
 9   PRECT_TOTAL 156 non-null    float64
10  SURFACE_P   156 non-null    float64
11  Cases       156 non-null    int64  
dtypes: float64(11), int64(1)
memory usage: 19.9+ KB
```

FIGURE 3.12 – Présentation des données

Comme montré dans la figure 3.12, notre matrice des données contient 156 lignes et 12 colonnes, avec 11 variables climatiques et une variable dépendante. Nos données sont toutes numériques et mensuelles, allant de 01-2010 à 12-2022. Les données sont complètes et il n'y a aucune valeur manquante.

```
[12] df.describe()
```

	RH2M	TMP_MIN	TMP_MAX	WS2M	WD2M	UVA	UVB	CLOUD_AMT	SOIL_W	PRECT_TOTAL	SURFACE_P	Cases
count	156.000000	156.000000	156.000000	156.000000	156.000000	156.000000	156.000000	156.000000	156.000000	156.000000	156.000000	156.000000
mean	43.053511	1.201603	31.673269	2.631813	249.554113	13.993039	0.330984	31.322866	0.342221	121.674962	83.729031	31.967949
std	12.526672	6.591168	6.758056	0.482511	65.979048	3.916059	0.138949	9.456710	0.138211	226.065183	0.277707	34.555504
min	18.688235	-10.090000	19.620000	1.447059	35.437647	7.709412	0.115882	12.635294	0.087059	0.000000	83.025294	1.000000
25%	32.949118	-4.420000	25.550000	2.348125	205.494167	9.931765	0.189559	24.763382	0.219412	3.589265	83.573824	7.000000
50%	44.150882	-0.455000	31.490000	2.619020	262.899412	14.203235	0.340588	30.879412	0.362941	20.783824	83.723235	19.000000
75%	52.852941	7.062500	38.182500	2.928235	305.858529	17.571029	0.453676	37.793971	0.446324	106.772500	83.922500	47.250000
max	70.132353	15.430000	42.190000	4.221667	341.640833	20.874706	0.577059	56.995882	0.719412	1302.540000	84.584118	172.000000

FIGURE 3.13 – Quelques statistiques descriptives de nos données

Comme on peut voir dans la figure 3.13, la moyenne de l'humidité relative (RH2M) dans la province de Ouarzazate dans la période de 2010 à 2012 est de 43,05%, avec un écart-type de 12,53%. Cela signifie que la plupart de ses valeurs se situent entre 30,52% et 55,58%. La température minimale (TMP_MIN) moyenne dans la province de Ouarzazate dans cette période est de 1,20 °C, avec un écart-type de 6,59 °C. La température minimale la plus basse enregistrée était de -10,09 °C. La température maximale (TMP_MAX) moyenne dans la même période est de 31,67 °C, avec un écart-type de 6,76 °C. La température maximale la plus élevée enregistrée était de 42,19 °C. L'humidité du sol (SOIL_W) moyenne est de 0,34, avec un écart-type de 0,14. L'humidité du sol la plus basse enregistrée était de 0,09. Les précipitations totales (PRECT_TOTAL) moyennes sont de 121,67, avec un écart-type de 226,07. Les précipitations totales les plus élevées enregistrées étaient de 1302,54.

3.4 Analyse Univarié des données

Pour commencer notre analyse exploratoire des données, nous allons faire une analyse univariée de chaque variable explicative (climatique) et de la variable dépendante dans notre base de données afin de bien comprendre leurs distributions et leurs caractéristiques.

3.4.1 Analyse du variable dépendante

Notre variable dépendante s'agit de nombre des cas mensuelles de la leishmaniose dans la province de Ouarzazate dans la période allant de 2010 jusqu'à 2022. Comme la figure 3.14 montre, la leishmaniose dans la province de Ouarzazate a connu des fluctuations au fur et au mesure des années, avec le plus grand nombre des cas enregistré en Janvier 2020 ou Ouarzazate a connu une vrai épidémie de leishmaniose.

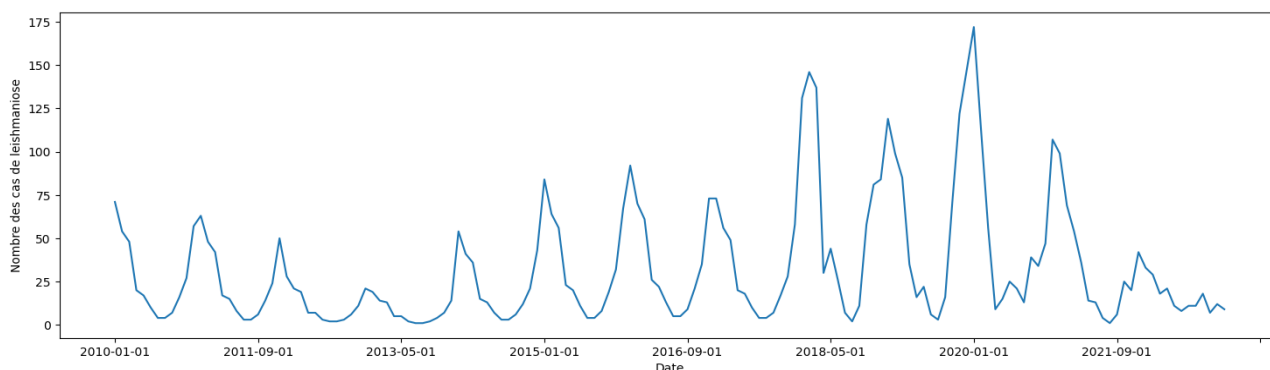


FIGURE 3.14 – La distribution des cas de leishmaniose de 2010 à 2022 dans la province de Ouarzazate

En analysant plus en détail le graphique dans la figure 3.15, on peut constater que les cas de leishmaniose suivent une sorte de tendance temporelle, sachant que la plupart des hausses dans le graphique se situent dans les premiers et les derniers mois de chaque année, et les baisses se trouvent souvent dans les mois de juin, de juillet et d'août. Cela peut indiquer que la leishmaniose dans la province de Ouarzazate est plus dangereuse pendant les mois d'hiver et d'automne.

Pour valider cette conclusion, la distribution des cas de leishmaniose par mois montrée dans la figure 3.15, indique que les mois avec les incidences les plus élevées de la leishmaniose sont janvier, février, décembre, mars, novembre et octobre, tandis que les incidences les plus faibles ont été enregistrées en juin, juillet, août et septembre.

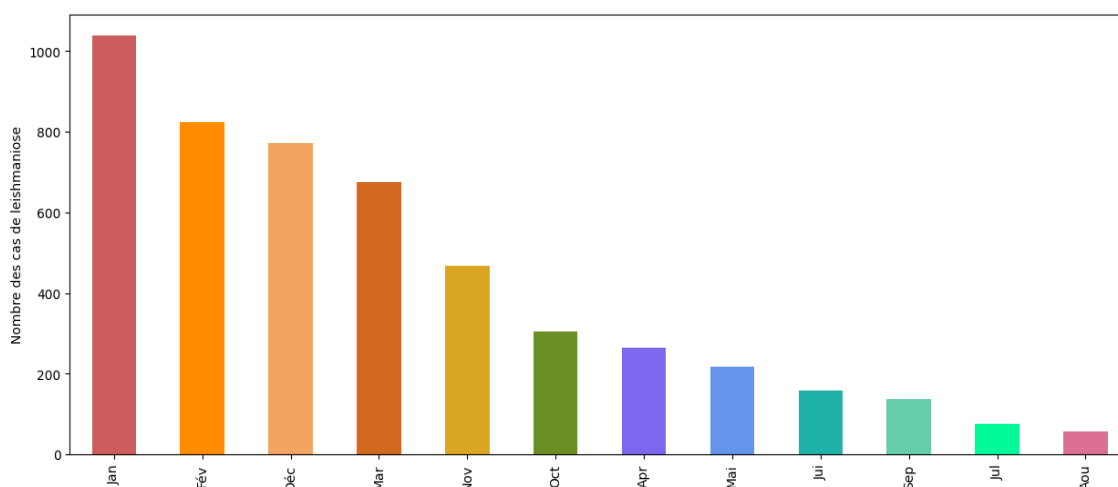


FIGURE 3.15 – Distribution des cas de leishmaniose par mois

Selon la distribution des cas de leishmaniose par année dans la figure 3.16, la province de Ouarzazate a connu un nombre élevé d'incidences de la leishmaniose en 2010 et 2011, mais a ensuite connu une baisse des incidences

en 2012 et 2013. Ensuite, elle a été confrontée à une sorte d'épidémie entre 2014 et 2021, enregistrant ainsi un grand nombre d'incidences au cours de cette période. En 2022, les cas de leishmaniose ont significativement baissé.

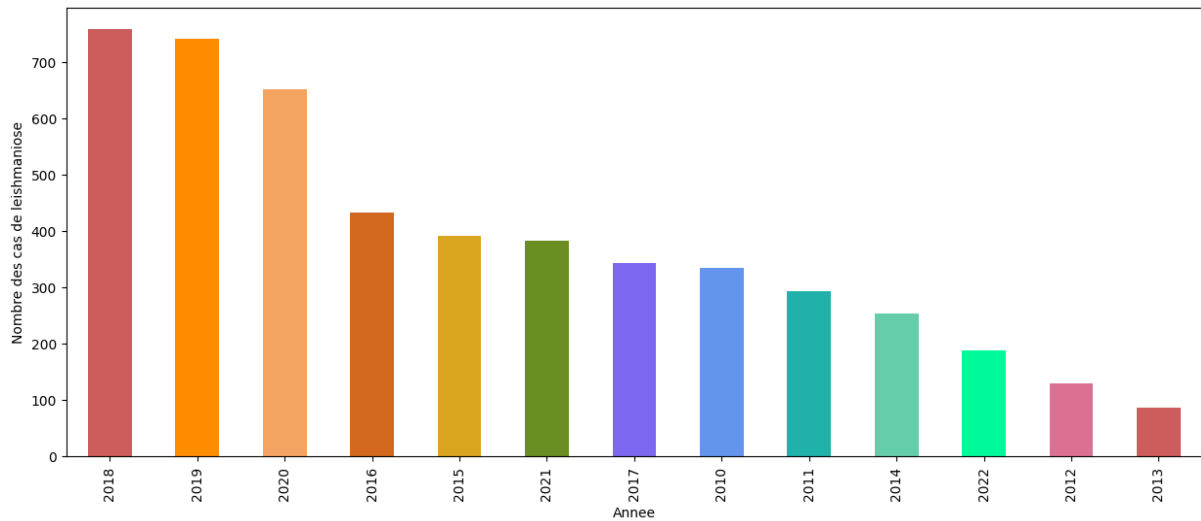


FIGURE 3.16 – Distribution des cas de leishmaniose par année

Les cas de leishmaniose en Ouarzazate dans la période de 2010 à 2022 suivent donc une tendance saisonnière, comme on peut constater dans la figure 3.17, les saisons ayant enregistré le plus grand nombre des cas de leishmaniose sont l'hiver, l'automne et le printemps, tandis qu'en été les cas ont été basses.

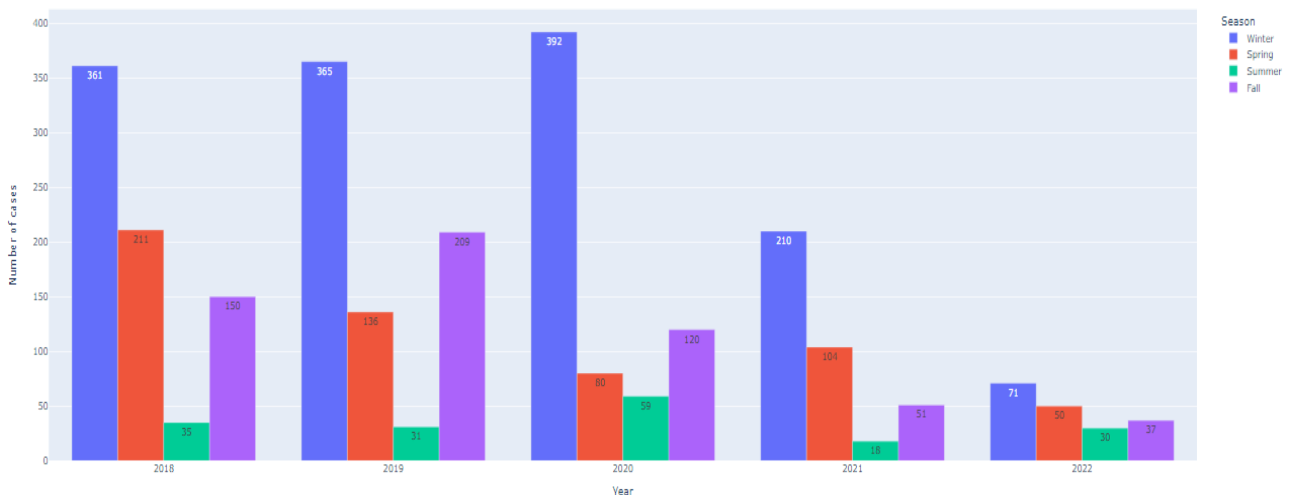
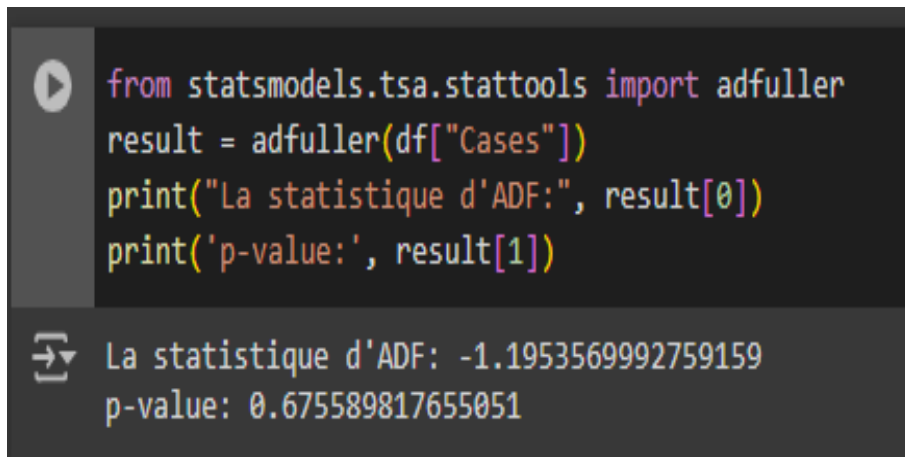


FIGURE 3.17 – Distribution des cas de leishmaniose par année et par saison

Puisque notre variable dépendante est une série temporelle, il est important de vérifier sa stationarité, ce qui est souvent nécessaire pour de nombreux modèles statistiques utilisés pour résoudre des problèmes de séries temporelles. La stationarité est une propriété d'une série temporelle où ses propriétés statistiques, telles que la moyenne et la variance, restent constantes au cours du temps. L'un des tests les plus couramment utilisés pour vérifier la stationarité d'une variable de série temporelle est le test de Dickey-Fuller augmenté (ADF), qui est un test statistique utilisé pour déterminer si une unité racine est présente dans une série temporelle autoregressive. Le test ADF est une extension du test de Dickey-Fuller qui inclut des termes supplémentaires pour capturer des dynamiques plus complexes et améliore ainsi la précision de la détection de la stationarité.

Comme montré dans la figure 3.18, les résultats du test ADF ont montré une statistique ADF de -1.1953, avec une valeur p de 0.6755. La valeur critique à un niveau de signification de 5% était de -2.89. Étant donné

que la statistique ADF est supérieure à la valeur critique et que la valeur p est supérieure à 0,05, nous acceptons l'hypothèse nulle et pouvons dire que notre variable de série temporelle n'est pas stationnaire.

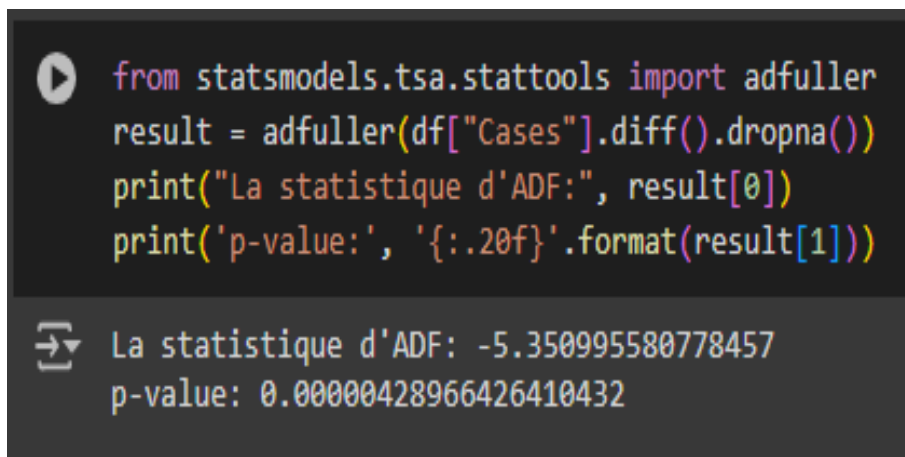


```
from statsmodels.tsa.stattools import adfuller
result = adfuller(df["Cases"])
print("La statistique d'ADF:", result[0])
print('p-value:', result[1])
```

La statistique d'ADF: -1.1953569992759159
p-value: 0.675589817655051

FIGURE 3.18 – Le résultat du test de stationarité ADF

L'une des méthodes utilisées pour surmonter le problème de non-stationnarité dans les séries temporelles est la différenciation. Le fait de différencier une série temporelle c'est de calculer la différence entre une observation et l'observation précédente, ce qui permet de supprimer les tendances et les structures de dépendance temporelle, rendant la série temporelle stationnaire. Comme indiqué dans la figure 3.19, les résultats du test ADF après la différenciation de notre série temporelle ont révélé une statistique ADF de -5.35, avec une valeur p de 0.0000042, ce qui signifie que notre variable de série temporelle est devenue stationnaire après différenciation.



```
from statsmodels.tsa.stattools import adfuller
result = adfuller(df["Cases"].diff().dropna())
print("La statistique d'ADF:", result[0])
print('p-value:', '{:.20f}'.format(result[1]))
```

La statistique d'ADF: -5.350995580778457
p-value: 0.00000428966426410432

FIGURE 3.19 – Le résultat du test ADF après différenciation

Un autre aspect important dans les séries temporelles que nous devons analyser est l'autocorrélation. Dans de nombreuses solutions pour prédire les cas de maladies infectieuses, les décalages de la variable de série temporelle se sont révélés être essentiels lors de la phase de modélisation. Cela remonte au fait que les séries temporelles dépendent du temps, et chaque pas de temps dépend souvent des autres pas de temps précédents. L'analyse de l'autocorrélation est réalisée pour déterminer quels décalages ou quels pas de temps précédents sont les plus corrélés avec la variable dépendante.

Le graphique de la fonction d'autocorrélation et le graphique de la fonction d'autocorrélation partielle montré dans la figure 3.20 sont deux des graphiques les plus populaires pour l'analyse de l'autocorrélation car ils montrent directement quels décalages sont les plus corrélés avec la variable dépendante. La différence entre la fonction d'autocorrélation et la fonction d'autocorrélation partielle est que la fonction d'autocorrélation mesure la corrélation entre une observation et ses observations précédentes, tandis que la fonction d'autocorrélation partielle mesure la corrélation entre une observation et ses observations précédentes après avoir retiré l'effet des observations intermédiaires.

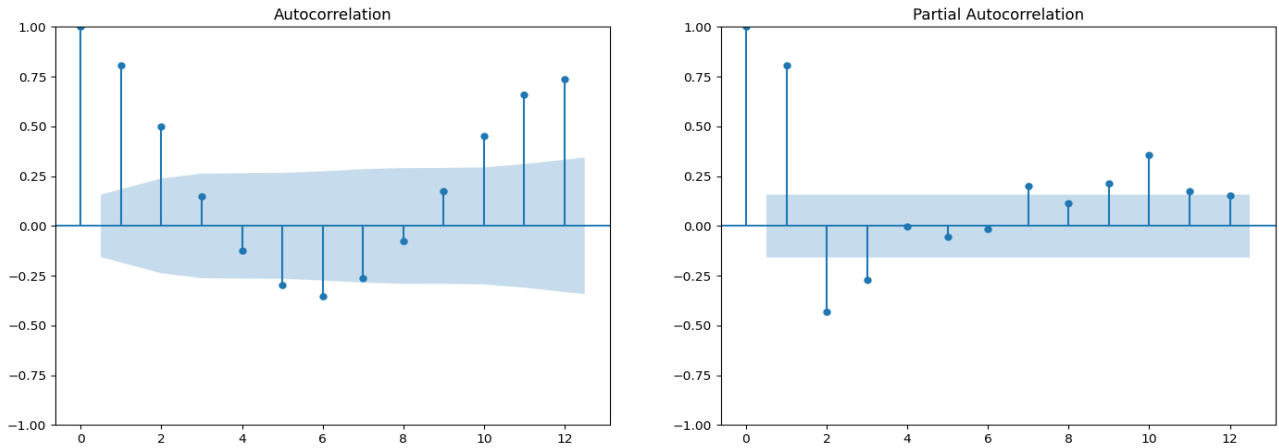


FIGURE 3.20 – Les graphiques ACF et PACF

D'après les deux graphiques présentés dans la figure, il est évident que les décalages les plus corrélés (directement) avec la variable dépendante sont les décalages 1, 2, 3, 7, 9 et 10. Ces décalages seront très utiles dans la phase de modélisation.

3.4.2 Analyse des variables explicatives

Comme nous l'avons discuté précédemment, notre matrice de données explicatives comporte 11 variables climatiques, chacune représentant un facteur important dans la transmission de la leishmaniose. Si nous voulons les analyser individuellement, la première chose à regarder est leurs distributions, comme le montre la figure 3.21, les distributions de chaque variable peuvent être représentées dans un graphique en histogramme, ce graphique nous aide à voir quel type de distribution chaque variable suit, ainsi qu'à nous montrer s'il y a des valeurs aberrantes dans chaque variable.

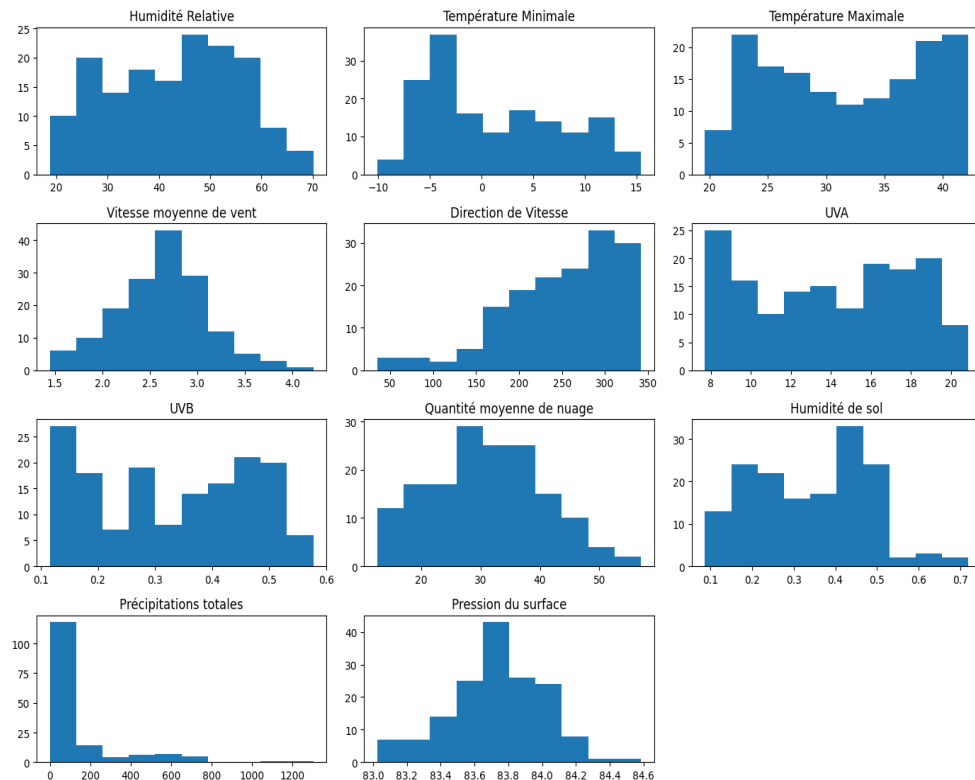


FIGURE 3.21 – Les distributions de chaque variable explicative

La distribution des températures maximales semble bimodale, sans signe de valeurs aberrantes. En revanche, l'humidité relative présente une distribution plutôt normale, ce qui suggère que la majorité des valeurs se situent autour d'un taux d'humidité typique, avec moins d'occurrences aux extrêmes. Les distributions de la vitesse moyenne du vent et de la pression atmosphérique semblent typiquement normales, avec la plupart des valeurs regroupées autour de la médiane. La distribution des quantités de nuages est légèrement asymétrique positive, tout comme la distribution des précipitations, qui semble asymétrique négative, et de la direction du vent qui semble légèrement asymétrique négative. On peut constater également la présence de quelques valeurs aberrantes dans les précipitations totales. Pour les distributions des intensités d'UVA et UVB, on peut constater une répartition plutôt uniforme.

Après avoir analysé les distributions de chaque variable explicative, la prochaine étape consiste à analyser leurs dépendances temporelles. Cela peut nous être utile de plusieurs manières : par exemple, en examinant les dépendances temporelles des variables climatiques, on peut identifier s'il y a de la saisonnalité dans certains variables climatiques, c'est-à-dire des variations périodiques qui se répètent à intervalles réguliers, comme des fluctuations mensuelles ou annuelles.

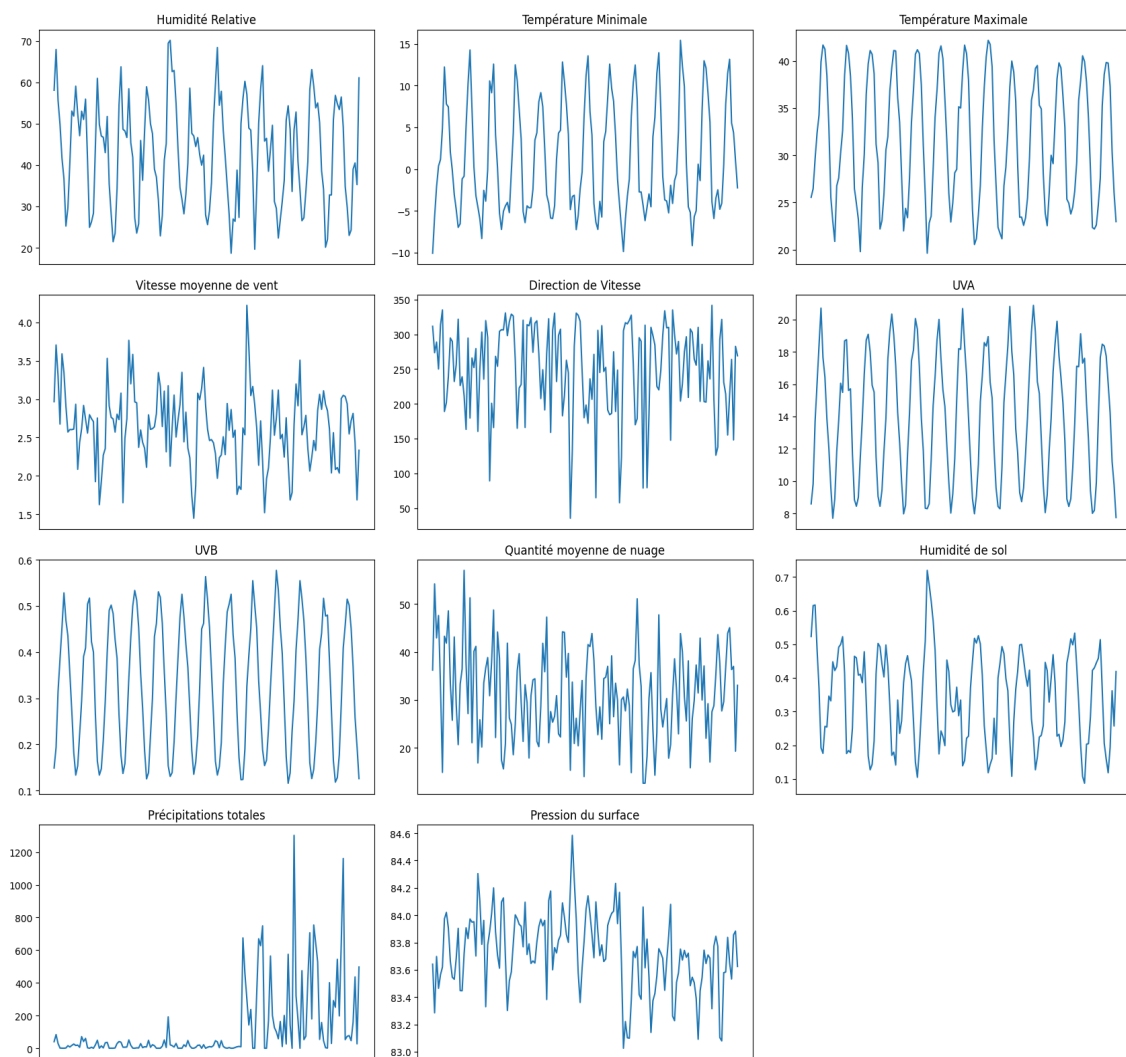


FIGURE 3.22 – Les dépendances temporelles des variables explicatives

Les dépendances temporelles montrées dans la figure 3.22 donnent plusieurs informations sur les fluctuations de chaque variable au fil du temps. Par exemple, dans le cas de l'intensité des UVA et UVB ainsi que des températures minimales et maximales, on peut voir que les fluctuations sont ordinaires. En revanche, pour les précipitations totales, on peut constater qu'elles sont plus fortes pendant la période de 2018 à 2022. Le contraire est observé pour la pression de surface, qui montre des fluctuations plus faibles entre 2018 et 2022 par rapport à la période de 2010 à 2017.

3.5 Analyse Multivarié des données

Pour l'analyse multivariée, nous allons couvrir deux analyses : l'analyse de corrélation et l'analyse de corrélation croisée. L'analyse de corrélation devrait nous aider à comprendre comment chaque variable climatique est liée à la variable dépendante, et la corrélation croisée devrait nous aider à comprendre comment le temps modifie la relation entre les variables climatiques et la variable dépendante.

3.5.1 Analyse de corrélation

L'une des façons les plus intéressantes de visualiser la corrélation entre deux variables est d'utiliser des nuages de points. Les nuages de points sont des graphiques où chaque point représente une observation, avec ses coordonnées sur les axes x et y correspondant aux valeurs des deux variables étudiées. Ils peuvent montrer la corrélation entre deux variables, mais aussi révéler d'autres types de relations non linéaires. Comme le montre la figure 3.23, la relation entre l'humidité relative et les cas de leishmaniose est modérée et positive. Cela peut s'expliquer par le fait que l'humidité crée un environnement favorable pour les phlébotomes, les insectes qui transmettent la leishmaniose, en augmentant leur survie et leur reproduction. En revanche, la relation entre la température maximale et minimale et les cas de leishmaniose semble être modérée et négative, et elle paraît non linéaire. Cela peut expliquer pourquoi les cas de leishmaniose sont élevés pendant les mois d'hiver et d'automne. La corrélation entre la vitesse du vent, les UVA, les UVB et la variable dépendante semble plus faible et quelque peu négative, ce qui montre qu'il peut y avoir une relation non linéaire entre elles. Il en va de même pour la quantité de nuages et la pression de surface. La relation entre l'humidité du sol et les précipitations totales semble être modérée et positive et plutôt linéaire. La relation entre la direction du vent et les cas de leishmaniose semble être très faible.

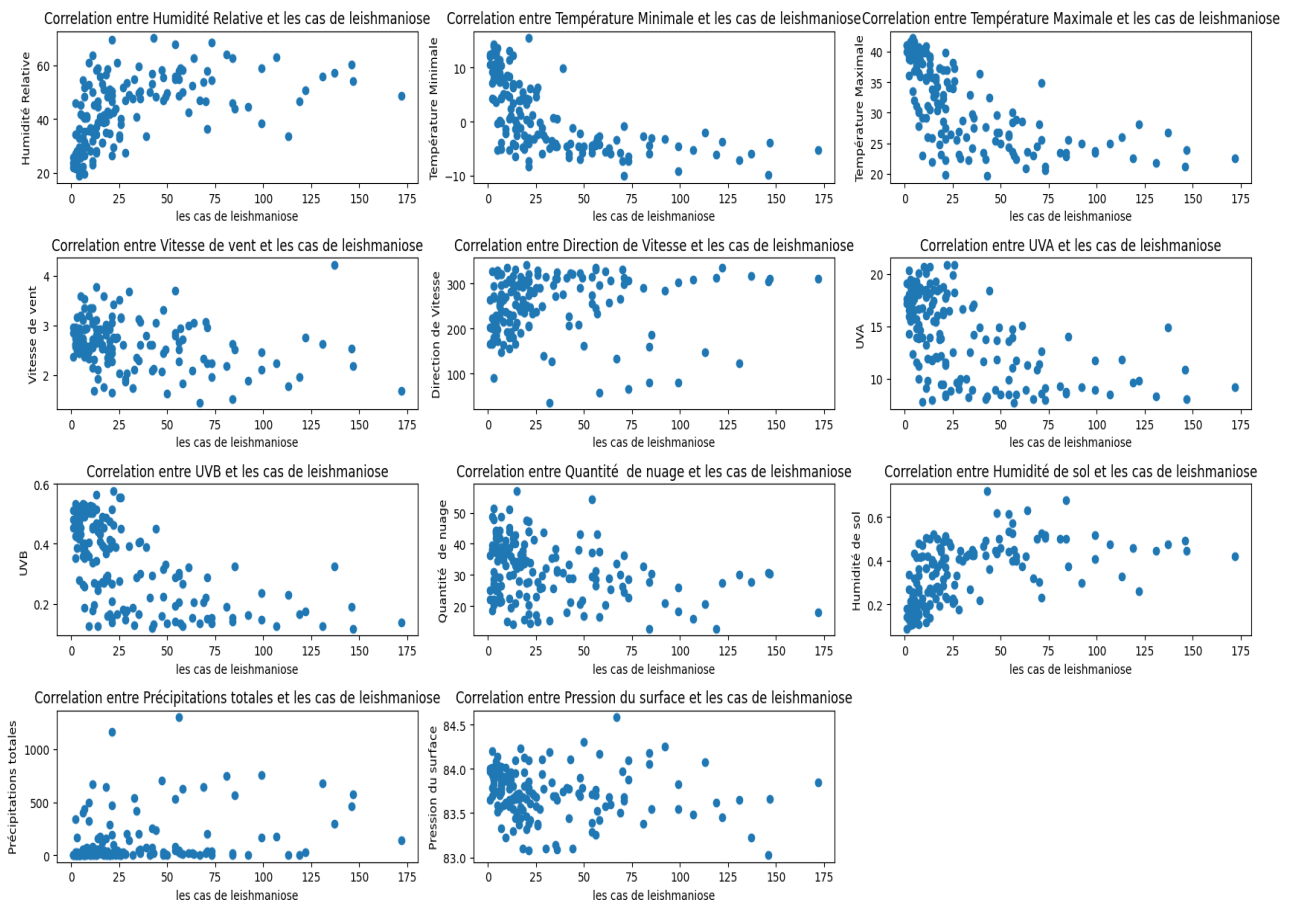


FIGURE 3.23 – Les nuages de points entre les variables explicatives et les cas de leishmaniose

Une autre façon d'examiner les corrélations entre les variables est d'utiliser la matrice de corrélation, qui peut être plus efficace si l'on cherche des valeurs exactes entre -1 et 1 représentant la force de la corrélation ainsi que sa direction. Les matrices de corrélation sont des tableaux où chaque cellule montre le coefficient de corrélation entre deux variables, permettant ainsi de voir rapidement les relations entre toutes les variables. La

matrice de corrélation présentée dans la figure 3.24 suggère les mêmes conclusions que nous avons tirées des nuages de points : l'humidité relative est positivement corrélée avec les cas de leishmaniose, avec un coefficient de corrélation modéré et significatif de 0,5, tandis que les températures minimale et maximale sont négativement corrélées à la variable dépendante avec un coefficient de corrélation plus fort de -0,61. La vitesse du vent et la direction du vent ne sont pas significativement corrélées à la variable dépendante, avec des coefficients de corrélation faibles de 0,25 et 0,08 respectivement. Les UVA et UVB, en revanche, sont modérément et négativement corrélés aux cas de leishmaniose, avec des valeurs de coefficient de corrélation de -0,54 et -0,58 respectivement. Il en va de même pour la quantité de nuages et la pression de surface, avec des valeurs de coefficient de corrélation de -0,31 et -0,15 respectivement, le premier étant significatif mais pas le second. Enfin, les précipitations et l'humidité du sol montrent une corrélation positive et significative avec les cas de leishmaniose, avec des valeurs de coefficient de corrélation de 0,3 pour les précipitations et de 0,5 pour l'humidité du sol.

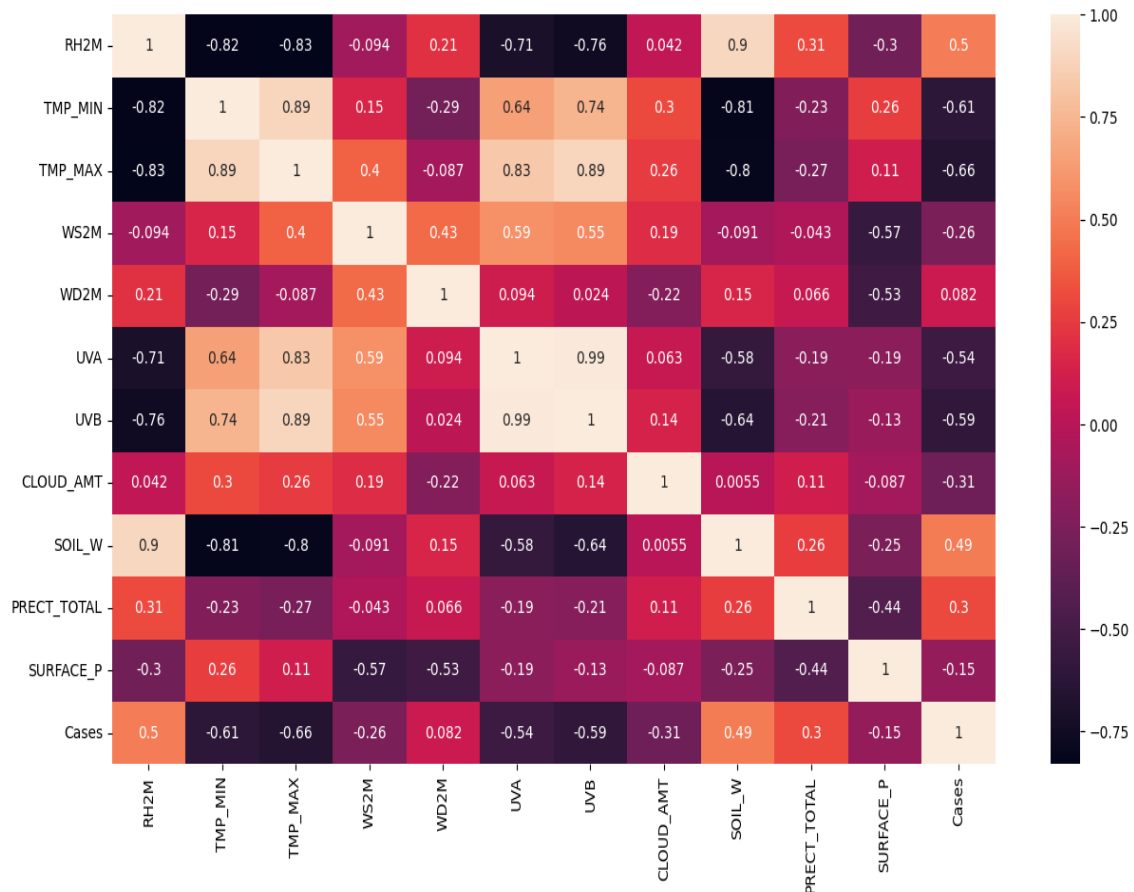


FIGURE 3.24 – la matrice des corrélation entre les variables climatiques et les cas de leishmaniose

La matrice de corrélation montre plus que la corrélation entre les variables explicatives et la variable dépendante, elle montre aussi la corrélation entre les variables explicatives elles-mêmes. Comme indiqué dans la figure 3.24, des variables comme l'humidité relative et les températures minimale et maximale sont fortement corrélées entre elles. Cela s'explique par le fait que l'humidité relative dépend de la température de l'air : à des températures plus élevées, l'air peut contenir plus de vapeur d'eau, ce qui augmente l'humidité relative. Nous remarquons également que les UVA et les UVB sont fortement corrélés avec l'humidité du sol, l'humidité relative, ainsi que les températures minimale et maximale.

3.5.2 Analyse de corrélation croisée

L'analyse de corrélation croisée est l'une des analyses les plus importantes dans la prédiction des séries temporelles, car elle permet de comprendre comment les variables sont liées à différents décalages temporels. L'analyse de corrélation croisée peut être vue comme une extension de l'analyse de corrélation, car elle nous aide à en savoir plus sur la relation entre les variables à divers décalages. Cela peut nous aider dans l'ingénierie des caractéristiques, car une variable peut sembler ne pas être significativement corrélée avec la variable dépendante, mais peut être fortement corrélée à un décalage spécifique avec la variable dépendante. Par exemple,

l'humidité relative des mois précédents peut avoir un impact sur les cas de leishmaniose du mois en cours, car l'humidité peut affecter les conditions de reproduction et de survie des phlébotomes, les insectes vecteurs de la leishmaniose. Ainsi, une période de haute humidité il y a quelques mois peut entraîner une augmentation des cas de leishmaniose aujourd'hui.

Les résultats de la corrélation croisée présentés dans la figure 3.25 montrent que l'humidité relative au décalage 6 a la valeur la plus élevée du coefficient de corrélation avec les cas de leishmaniose parmi tous les décalages de la même variable, tandis que la température minimale au décalage 6 a également la valeur la plus élevée. Il en va de même pour l'humidité du sol. La température maximale et la quantité de nuages aux décalages 12 montrent la valeur la plus élevée du coefficient de corrélation, et les UVA et UVB au décalage 1 montrent la valeur la plus élevée. La corrélation de la pression de surface avec les cas de leishmaniose n'était pas significative dans l'analyse de corrélation, mais l'analyse de corrélation croisée suggère maintenant que ses valeurs au décalage 9 montrent une corrélation modérée et significative avec la variable dépendante. Pour la direction du vent, malgré ses valeurs au décalage 5 montrant une meilleure corrélation avec la variable dépendante, cela reste faible. Nous remarquons également que la direction de la corrélation de certaines variables avec la variable dépendante a changé lorsqu'elles sont décalées, comme la température minimale et l'humidité du sol, par exemple.

	variable	lag	correlation	p_value
0	RH2M	6	-0.586422	0.0000000000
1	TMP_MIN	6	0.663682	0.0000000000
2	TMP_MAX	12	-0.666363	0.0000000000
3	WS2M	2	-0.440414	0.0000000109
4	WD2M	5	-0.201871	0.0129308542
5	UVA	1	-0.668216	0.0000000000
6	UVB	1	-0.676915	0.0000000000
7	CLOUD_AMT	12	-0.337233	0.0000357652
8	SOIL_W	6	-0.582290	0.0000000000
9	PRECT_TOTAL	0	0.301153	0.0001333291
10	SURFACE_P	9	-0.421963	0.0000001020

FIGURE 3.25 – Les résultats de la corrélation croisée

Conclusion

L'analyse exploratoire des données est une étape primordiale dans n'importe quelle étude, car elle permet aux analystes de données de comprendre véritablement les données utilisées, leur structure et leur distribution, ainsi que les relations entre les variables explicatives et la variable dépendante, ce qui peut les aider beaucoup dans la phase de modélisation. Dans ce chapitre, nous avons décrit chaque étape de l'analyse exploratoire des données, de la collecte des données où nous avons décrit le processus étape par étape, au prétraitement des données où nous avons expliqué le processus de transformation des données cliniques et climatiques en une matrice utilisable, à l'analyse univariée et multivariée des données où nous avons exploré la structure et la distribution de chaque variable dans notre matrice, avant d'examiner les corrélations et les corrélations croisées de chaque variable avec la variable dépendante. Dans le chapitre suivant, nous aborderons la modélisation et la mise en œuvre de notre solution pour prédire les cas de leishmaniose dans la province de Ouarzazate.

Chapitre 4

Implémentation et Résultats de la solution

Introduction

Dans ce chapitre, nous abordons l'implémentation de la solution proposée pour la prédiction des cas de leishmaniose dans la province de Ouarzazate. Cette solution repose principalement sur un modèle CNN-BiLSTM, qui combine les avantages des réseaux de neurones convolutionnels (CNN) dans l'extraction des caractéristiques et des réseaux de neurones récurrents bidirectionnels (BiLSTM) dans la prédiction des séries chronologiques. Nous présentons d'abord l'environnement de travail, qu'il soit matériel ou logiciel. Ensuite, nous décrivons l'architecture du modèle. Après, nous abordons l'entraînement des modèles et leur évaluation, en mettant en lumière les métriques utilisées.

4.1 Environnement du travail

4.1.1 Matériels de travail

Pour modéliser la solution à notre problématique, nous avons dû utiliser les matériaux et services suivants :

- * **Un ordinateur HP avec un processeur Intel(R) Core(TM) i3 et 4 Go de RAM, fonctionnant sous Windows 10.**
- * **Google Colaboratory :** Un service gratuit fourni par Google qui permet d'écrire et d'exécuter du code Python directement dans un navigateur web. Il offre une machine virtuelle avec un processeur Intel(R) Core(TM) i7, 12 Go de RAM et 107 Go de stockage.



FIGURE 4.1 – Google Colaboratory

4.1.2 Outils de programmation

- * **Python :** un langage de programmation interprété, orienté objet et de haut niveau qui possède des structures de données intégrées de haut niveau, combinées à un typage dynamique et à une liaison dynamique. Python est le langage de programmation le plus utilisé pour les projets de science des données et la modélisation de modèles d'apprentissage automatique et d'apprentissage profond.



FIGURE 4.2 – Le langage de programmation Python

- * **Pandas** : Une bibliothèque d'analyse et de manipulation de données open source, rapide et facile à utiliser, construit sur le langage de programmation Python. Pandas est largement utilisé dans la communauté de la science des données pour manipuler et prétraiter les données tabulaires.



FIGURE 4.3 – la bibliothèque Pandas

- * **NumPy (Numerical Python)** : Une bibliothèque Python utilisée pour travailler avec des tableaux, elle dispose de fonctions pour travailler dans le domaine de l'algèbre linéaire et des matrices. NumPy est également l'une des bibliothèques les plus utilisées en science des données et en apprentissage automatique.



FIGURE 4.4 – la bibliothèque NumPy

- * **Matplotlib** : une bibliothèque Python pour créer des visualisations statiques, animées et interactives. C'est également l'une des bibliothèques Python les plus largement utilisées dans la communauté de la science des données et de l'apprentissage automatique.



FIGURE 4.5 – la bibliothèque Matplotlib

- * **Scikit-learn** : Une bibliothèque Python open source, simple et efficace pour l'analyse prédictive des données qui est construite sur NumPy, SciPy et Matplotlib. Scikit-learn est l'une des principales bibliothèques Python en modélisation d'apprentissage automatique dans le monde de l'IA et de la science des données.automatique.



FIGURE 4.6 – la bibliothèque Scikit-learn

- * **TensorFlow** : Un framework open source développé par Google pour exécuter des modèles d'apprentissage profond et de réseaux neuronaux en Python. Il est largement utilisé par les data scientists et les ingénieurs en apprentissage automatique pour faciliter le développement d'applications d'analyse avancée.



FIGURE 4.7 – la bibliothèque TensorFlow

- * **Keras** : Un framework d'apprentissage profond basé sur Python, avec prise en charge de TensorFlow et PyTorch. Il offre aux utilisateurs la possibilité de construire et d'entraîner facilement des modèles pour la vision par ordinateur, le traitement du langage naturel, le traitement audio, la prévision de séries temporelles et de nombreuses autres tâches.



FIGURE 4.8 – la bibliothèque Keras

- * **CatBoost** : Une bibliothèque Python open source et haute performance pour le boosting de gradient sur les arbres de décision, elle offre aux utilisateurs la possibilité de créer facilement des arbres de boosting de gradient pour diverses tâches.



FIGURE 4.9 – la bibliothèque CatBoost

4.2 Architecture du modèle proposé

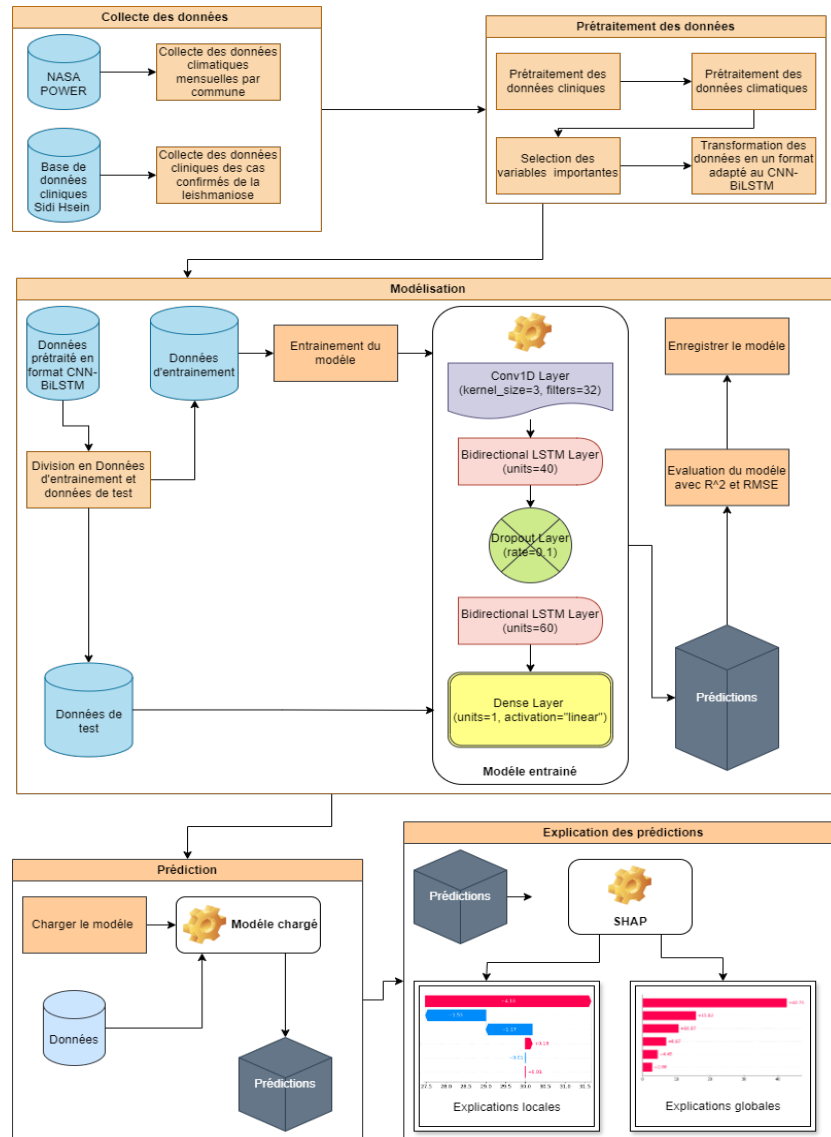


FIGURE 4.10 – L'architecture globale du système proposé

Notre projet vise à créer un système de prédiction des cas de leishmaniose dans la province d'Ouarzazate, qui soit à la fois puissant, facile à comprendre et fiable. Ce système est conçu pour aider les autorités sanitaires à prendre des décisions éclairées et à élaborer des stratégies efficaces de contrôle et de traitement concernant cette maladie dangereuse. En combinant un modèle hybride d'apprentissage profond avec une méthode intéressante pour l'interprétabilité, nous avons proposé un système qui vise à fournir à la fois des prédictions précises et des informations précieuses sur les facteurs influençant les cas de leishmaniose.

La figure 3.1 montre notre architecture innovante pour la prédiction de la leishmaniose, qui exploite les données climatiques et cliniques pour prédire le nombre mensuel de cas de cette maladie dans la province d'Ouarzazate. Tout d'abord, nous avons rassemblé les informations climatiques mensuelles par commune depuis NASA Power et les données cliniques sur les cas confirmés de leishmaniose de la base de données de Sidi Hsein. Ces données ont ensuite passé par une phase de prétraitement, dont nous avons parlé dans le chapitre précédent, pour les nettoyer et les adapter au format requis par le modèle. Au cœur de ce système se trouve un modèle Conv-BiLSTM, un type de réseau de neurones récurrent particulièrement efficace dans la classification des séquences temporelles, couplé aux capacités des couches convolutionnelles pour extraire des caractéristiques pertinentes à partir de données complexes. Ce modèle est ensuite entraîné sur les données prétraitées d'entraînement, puis évalué sur un ensemble de données de test également prétraitées. Après l'entraînement et l'évaluation, notre modèle a été sauvegardé et peut être utilisé pour faire des prédictions à tout moment, et ses prédictions ont été utilisées par la méthode SHAP pour obtenir à la fois des explications locales et globales du comportement du modèle, améliorant ainsi son interprétabilité.

4.3 Implémentation du structure du système proposée

Si nous voulons implémenter notre modèle composé de couches convolutionnelles et BiLSTM, nous devons d'abord effectuer certaines transformations de données. Cela est nécessaire car les couches LSTM et BiLSTM ne traitent pas les données comme le ferait un modèle d'apprentissage automatique classique ou un réseau des neurones artificielles, elles les traitent plutôt sous forme de pas de temps ou de séquences. Après la phase de transformation des données, nous devons choisir les métriques d'évaluation qu'on va utiliser pour ensuite déterminer le sous-ensemble des variables et les paramètres du modèles (appelés Hyperparamètres) optimaux qui donnent la performance optimale sur nos données.

4.3.1 Phase de transformation des données

La couche BiLSTM s'attend à ce que nos variables explicatives soient formatées en une matrice 3D de la forme suivante : (nombre d'observations, nombre de pas de temps, nombre de variables), et nos variables dépendantes en une matrice 1D de la forme suivante : (nombre d'observations). Dans notre cas, nous avons une matrice de données de 156 observations et 11 variables. Donc, si nous voulons que nos données soient dans le format attendu par la couche BiLSTM, nous devons choisir un nombre de pas de temps (qui sera le nombre de séquences passées que la BiLSTM utilisera pour prédire une séquence) et ensuite transformer nos données en une matrice 3D de la forme : (nombre d'observations, nombre de pas de temps, nombre de variables). Le nombre d'observations sera le nombre de lignes de notre matrice de données originale moins le nombre de pas de temps. Pour notre variable cible, elle contiendra les valeurs que nous voulons prédire, et le nombre d'observations sera également égal au nombre de lignes moins le nombre de pas de temps. Pour faire cette transformation, nous avons préparé une fonction *df_to_X_y* montré dans la figure 4.11. Cette fonction prend en entrée notre DataFrame original, les noms des variables qu'on veut utiliser comme paramètre *variables* et le nombre de pas de temps que nous voulons utiliser comme paramètre *window_size*. Ensuite, elle utilise ce DataFrame original, les noms des variables qu'on veut utiliser et le paramètre *window_size* pour transformer la matrice originale en deux matrices : X, qui est la matrice des caractéristiques, et y, qui est la matrice de la variable dépendante, dans les formats corrects attendus par la couche BiLSTM.

```
[4] #preprocessing the data so it's compatible with Conv-BiLSTM architecture
def df_to_X_y(df,variables>window_size=12) :
    df_np=df[variables].values
    X=[]
    y=[]
    for i in range(len(df)-window_size) :
        row=[row for row in df_np[i:(i>window_size-1)]]
        row.append([df_np[i>window_size,0],df_np[i>window_size,1],df_np[i>window_size,2],
                    df_np[i>window_size,3],df_np[i>window_size,4],df_np[i>window_size-1,-1]])
        label=df_np[i>window_size,-1]
        X.append(row)
        y.append(label)
    return np.array(X),np.array(y)
```

FIGURE 4.11 – Fonction de transformation du DataFrame en matrices X et y pour l'entraînement de notre modèle.

Comme le montre la figure 4.12, nous avons testé la fonction avec des valeurs aléatoires pour les paramètres *variables* et *window_size* et nous pouvons voir qu'elle renvoie deux matrices X et y qui ont la forme correcte nécessaire pour notre modèle ConvBiLSTM.

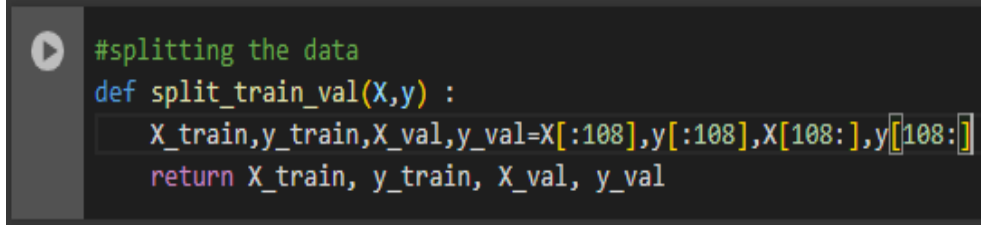
```
[10] X,y=df_to_X_y(df,variables=['TMP_MIN','TMP_MAX','WD2M','UVA','CLOUD_AMT',"Cases"],window_size=6)

[11] X.shape,y.shape

((150, 6, 6), (150,))
```

FIGURE 4.12 – Le résultat d'application de la fonction *df_to_X_y* sur notre matrice des données

Après la transformation des données, il est nécessaire de les diviser en données d'entraînement, qui seront utilisées pour l'entraînement du modèle, et en données de validation, qui seront utilisées pour la validation du modèle. Pour les données d'entraînement, nous avons utilisé les données de janvier 2010 à décembre 2019, et le reste pour les données de validation. La fonction présentée dans la figure 4.13 permet de s'occuper de cette division.



```
#splitting the data
def split_train_val(X,y) :
    X_train,y_train,X_val,y_val=X[:108],y[:108],X[108:],y[108:]
    return X_train, y_train, X_val, y_val
```

FIGURE 4.13 – Fonction de division de nos matrices X et y en données d'entraînement et de validation

4.3.2 Les métriques d'évaluation des performances du modèle

Pour évaluer un modèle d'apprentissage automatique ou un modèle d'apprentissage profond dans une tâche de prédiction de séries temporelles, nous devons utiliser un ensemble de métriques d'évaluation qui mesurent la performance du modèle. Ces métriques mesurent généralement le degré de différence entre les valeurs prédites et les valeurs réelles, mais chacune le fait à sa manière. Étant donné que les tâches de prédiction de séries temporelles peuvent également être considérées comme des tâches de régression, les métriques les plus populaires utilisées pour l'évaluation de ces tâches sont également celles utilisées pour les tâches de régression, telles que :

- * **l'erreur quadratique moyenne** : mesure la différence quadratique entre les valeurs prédites et les valeurs réelles. Sa formule mathématique est la suivante :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- * **l'erreur quadratique moyenne en racine carré** : C'est tout simplement la racine carrée de l'erreur quadratique moyenne. Voici sa formule mathématique :

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- * **l'erreur absolue moyenne** : mesure la différence absolue entre les valeurs prédites et les valeurs réelles. Elle s'exprime mathématiquement comme suit :

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- * **le coefficient de détermination** : une métrique statistique qui mesure la proportion de la variance de la variable dépendante expliquée par les valeurs prédites. Elle s'exprime mathématiquement comme suit :

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- * **l'erreur moyenne absolue en pourcentage** : métriques qui gagnent en popularité ces dernières temps. Elle mesure la différence moyenne en pourcentage absolue entre les valeurs prédites et les valeurs réelles. Sa formule mathématique est :

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left(\frac{|y_i - \hat{y}_i|}{|y_i|} \right) \times 100$$

4.3.3 Détermination du nombre optimale des pas de temps (ou des séquences)

Pour obtenir le nombre optimal de pas de temps qui maximise les performances du modèle, nous devons entraîner les modèles ConvBiLSTM avec différentes valeurs de *window_size* qui représentent le nombre de pas de temps, puis nous devons évaluer chaque version du modèle sur les données de validation en utilisant la métrique RMSE et sélectionner la valeur de *window_size* qui donne le meilleur score RMSE de validation.

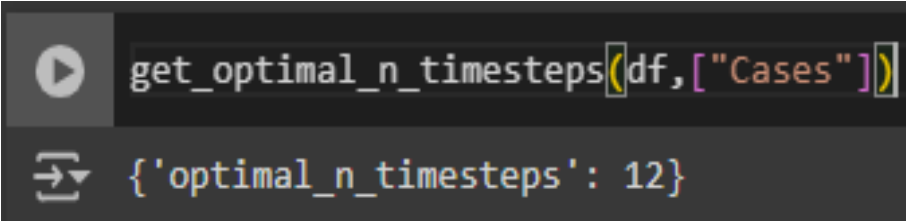
Pour y parvenir, nous devons écrire une fonction qui initialise le modèle avec la taille de la fenêtre à chaque fois, l'entraîne et calcule le score de ce modèle avec la valeur de la taille de la fenêtre, puis retourne la valeur optimale pour la taille de la fenêtre qui donne le meilleur score de validation. C'est exactement ce que fait la fonction *get_optimal_n_timesteps* montré dans la figure 4.14 : elle prend la matrice de données originale et une liste aléatoire de variables, initialise ensuite le ConvBiLSTM avec des valeurs initiales pour les hyperparamètres, puis appelle la fonction *df_to_X_y* avec la liste de variables fournies et la taille de la fenêtre pour la ième itération, et divise ensuite les données en ensemble d'entraînement et de validation en appelant la fonction *split_train_val*. Après cela, elle entraîne le modèle pendant 100 époques sur les données d'entraînement, puis calcule son score RMSE sur les données de validation. Après toutes les itérations, elle retourne un dictionnaire qui enregistre la valeur de la taille de la fenêtre qui donne la meilleure performance sur les données de validation.

```
[8] from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import *
    from tensorflow.keras.losses import MeanSquaredError
    from tensorflow.keras.metrics import RootMeanSquaredError
    from tensorflow.keras.optimizers import Adam
    from sklearn.metrics import mean_squared_error

[22] def get_optimal_n_timesteps(df,variables) :
    scores={}
    for window_size in [6,7,8,9,10,11,12] :
        model = Sequential()
        model.add(Conv1D(filters=64, kernel_size=4, input_shape=(window_size, len(variables))))
        model.add(Bidirectional(LSTM(50,return_sequences=True)))
        model.add(Bidirectional(LSTM(50)))
        model.add(Dense(1))
        model.compile(loss=MeanSquaredError(),optimizer=Adam(),metrics=[RootMeanSquaredError()])
        X,y=df_to_X_y(df,variables,window_size)
        X_train,y_train,X_val,y_val=split_train_val(X,y)
        model.fit(X_train,y_train,verbose=0,epochs=100)
        preds=model.predict(X_val,verbose=0).flatten()
        scores[window_size]=np.sqrt(mean_squared_error(y_val,preds))
    return {"optimal_n_timesteps" : min(scores,key=scores.get)}
```

FIGURE 4.14 – Fonction de détermination du nombre optimale de pas de temps sur la base des données de validation

Pour le sous-ensemble des variables explicatives à passer à la fonction, nous avons choisi uniquement les cas de leishmaniose des mois précédents, et comme le montre la figure 4.16, nous obtenons que le nombre optimal de pas de temps est 12.



```
get_optimal_n_timesteps(df,["Cases"])
```

```
{'optimal_n_timesteps': 12}
```

FIGURE 4.15 – Le nombre optimale de pas de temps obtenu par la fonction

4.3.4 Détermination du sous-ensemble des variables explicatives optimal

Vue que notre matrice de données contient un grand nombre de variables explicatives et pas beaucoup d'observations, entraîner un modèle sur cette matrice de données dans son ensemble conduirait le modèle à surajuster les données d'entraînement (Overfitting). Pour résoudre ce problème, les chercheurs ont mis au point une solution en introduisant une technique appelée sélection de caractéristiques, qui consiste essentiellement à identifier et à utiliser uniquement les variables les plus pertinentes pour le modèle. La sélection de caractéristiques est importante pour réduire le surajustement car elle réduit la complexité globale du modèle en sélectionnant que les variables informatives et non redondantes. Non seulement elle est cruciale pour surmonter le problème du surajustement, mais elle aide également à augmenter la performance du modèle en termes de complexité computationnelle et à améliorer son interprétabilité. Pour obtenir le sous-ensemble de variables explicatives qui offrent les meilleures performances sur les données de validation, nous avons déployé une approche inspirée de la méthode de sélection progressive. La sélection progressive est une méthode de sélection de caractéristiques (feature selection) qui consiste à commencer avec un modèle vide et à ajouter progressivement des variables explicatives, une par une, en évaluant à chaque étape si l'ajout de la variable améliore les performances du modèle selon un critère donné (par exemple, le score R^2).

Notre méthode se déroule en 4 étapes importantes comme montré dans la figure 4.16 : tout d'abord, nous initialisons notre modèle ConvBiLSTM et l'entraînons sur les données d'entraînement qui ne contiennent que les cas de leishmaniose des 12 mois précédents, puis nous calculons le score R^2 de base de ce modèle sur les données de validation. Ensuite, nous ajoutons une variable explicative aléatoire à la matrice de données, entraînons le modèle avec cette nouvelle matrice, et calculons le nouveau score R^2 du modèle après l'ajout de cette variable. Si ce nouveau score R^2 est meilleur que le score R^2 de base, nous ajoutons cette variable à notre liste de variables importantes ; sinon, nous passons cette variable et ajoutons une autre variable explicative sélectionnée aléatoirement, puis nous répétons le processus jusqu'à obtenir notre liste finale de variables qui offrent les meilleures performances du modèle sur les données de validation.

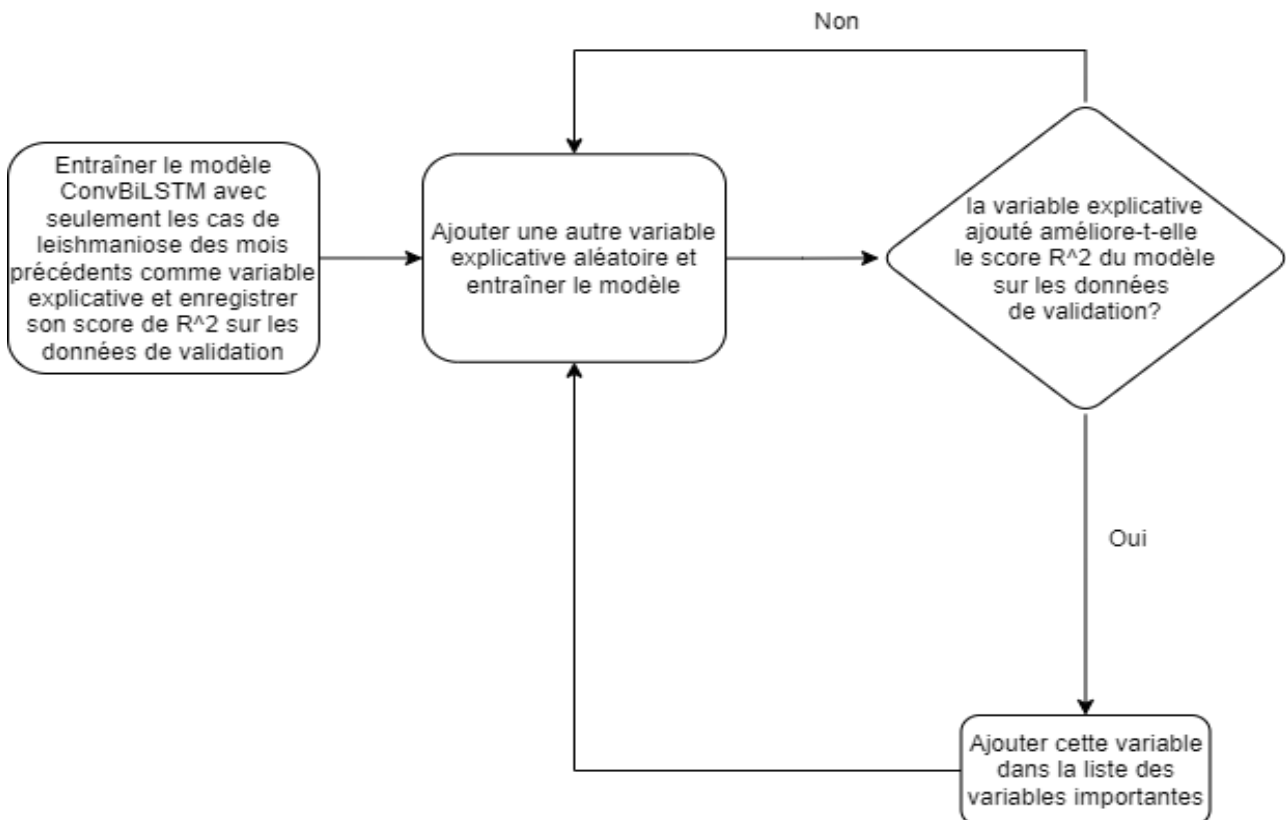


FIGURE 4.16 – La méthode utilisé pour déterminer les variables explicatives qui donnent la performance optimale du modèle sur les données de validation

Après avoir appliqué cette méthode en Python, nous avons conclu que la liste des variables les plus importantes qui donnent le meilleur score R^2 sur les données de validation est la suivante : humidité relative, température minimale, humidité du sol, vitesse moyenne du vent et intensité des rayons ultraviolets B.

4.3.5 Ajustement des hyperparamètres du modèle.

Après avoir obtenu le nombre optimal de pas de temps à utiliser pour notre modèle et le sous-ensemble des variables explicatives qui donnent les meilleures performances du modèle sur les données de validation, la prochaine étape consiste à trouver les hyperparamètres optimaux du modèle. Nous faisons cela en appliquant une méthode appelée ajustement des hyperparamètres, qui utilise une approche systématique pour trouver les meilleures valeurs d'hyperparamètres pour un modèle donné. L'ajustement des hyperparamètres utilise généralement une grille qui contient plusieurs valeurs pour chaque hyperparamètre du modèle que nous voulons régler, définissant ainsi ce que nous appelons un espace de recherche qui contient différentes combinaisons de valeurs d'hyperparamètres du modèle. Ensuite, elle utilise une technique d'optimisation, par exemple la recherche par grille, qui consiste à évaluer le modèle pour chaque combinaison possible d'hyperparamètres dans la grille, en testant chaque combinaison sur un ensemble de validation pour trouver celle qui donne les meilleures performances du modèle.

L'architecture de notre modèle contient de nombreux hyperparamètres que nous pouvons ajuster. Nous avons 4 couches : premièrement, la couche Conv1D, qui est une couche de convolution utilisée pour extraire des caractéristiques pertinentes des données d'entrée. Cette couche a trois principaux hyperparamètres : *filters*, *kernel_size* et *activation*. Deuxièmement, nous avons la couche BiLSTM, qui est la couche chargée de traiter les séquences de notre série temporelle, cette couche a un hyperparamètre appelé *units*. Troisièmement, nous avons ajouté une couche de dropout, qui sert à réduire le surajustement dans les réseaux des neurones. Cette couche a un hyperparamètre appelé *rate*. Enfin, nous avons une autre couche BiLSTM qui a un hyperparamètre appelé *units* comme l'autre couche BiLSTM. Finalement, nous pouvons aussi considérer le paramètre *learning_rate* de l'optimiseur Adam comme un autre hyperparamètre que nous pouvons ajuster. Chaque hyperparamètre mentionné a sa propre signification, comme nous l'expliquons ci-dessous :

- * **filters** : Le nombre de filtres à appliquer à la sortie de la couche Conv1D. Les filtres sont utilisés pour extraire des caractéristiques des données d'entrée.
- * **kernel_size** : La taille du noyau de convolution à utiliser dans la couche Conv1D. Le noyau de convolution spécifie la taille de la fenêtre sur laquelle appliquer la convolution.
- * **activation** : La fonction d'activation à utiliser dans la couche Conv1D. La fonction d'activation détermine la sortie de chaque neurone dans la couche.
- * **units** : Le nombre d'unités dans les couches BiLSTM. Les unités déterminent la dimension de l'espace de sortie de chaque couche.
- * **rate** : Le taux de dropout à appliquer dans la couche de dropout. Le dropout est une technique de régularisation utilisée pour réduire le surajustement en désactivant aléatoirement un certain nombre de neurones pendant l'entraînement.
- * **learning_rate** : Le taux d'apprentissage dans Adam contrôle la taille des pas de mise à jour des poids du modèle pendant l'entraînement.

Pour utiliser la méthode d'ajustement des hyperparamètres pour notre modèle en Python, nous avons utilisé la méthode *GridSearchCV* fournie par la bibliothèque Scikit-learn, qui implémente la méthode de recherche par grille couplée avec la méthode de validation croisée, qui consiste simplement à diviser les données en plusieurs sous-ensembles pour entraîner et tester le modèle sur des ensembles de données différents. Le problème avec *GridSearchCV* est qu'elle n'est pas directement compatible avec les modèles Keras, c'est pourquoi nous avons utilisé la méthode *KerasRegressor* de la bibliothèque scikeras, qui est une interface permettant d'utiliser des modèles Keras avec Scikit-learn. Le code mentionné dans la figure 4.17 implémente l'ajustement des hyperparamètres pour notre modèle, nous avons d'abord défini la grille qui contient les différentes valeurs pour chaque hyperparamètre, puis nous avons écrit la méthode *create_ConvBiLSTM_model* qui crée le modèle avant de le passer à la méthode *KerasRegressor* avec la grille que nous avons définie, et pour la validation croisée, nous avons utilisé une méthode hybride appelée *TimeSeriesSplit* qui implémente la validation croisée pour les données de séries temporelles. Ensuite, nous avons passé le modèle généré par la méthode *KerasRegressor* à la méthode *GridSearchCV* qui génère un objet *GridSearchCV* prêt à utiliser.

```

from sklearn.model_selection import GridSearchCV, TimeSeriesSplit
from scikeras.wrappers import KerasRegressor

def create_ConvBiLSTM_model(learning_rate, filters, BiLSTM_units_1, BiLSTM_units_2, kernel_size, activation, do_rate):
    model = Sequential()
    model.add(Conv1D(filters=filters, kernel_size=kernel_size, activation=activation, input_shape=(12, 6)))
    model.add(Bidirectional(LSTM(BiLSTM_units_1, return_sequences=True)))
    model.add(Dropout(do_rate))
    model.add(Bidirectional(LSTM(BiLSTM_units_2)))
    model.add(Dense(1))
    model.compile(optimizer=Adam(learning_rate=learning_rate), loss='mse')
    return model

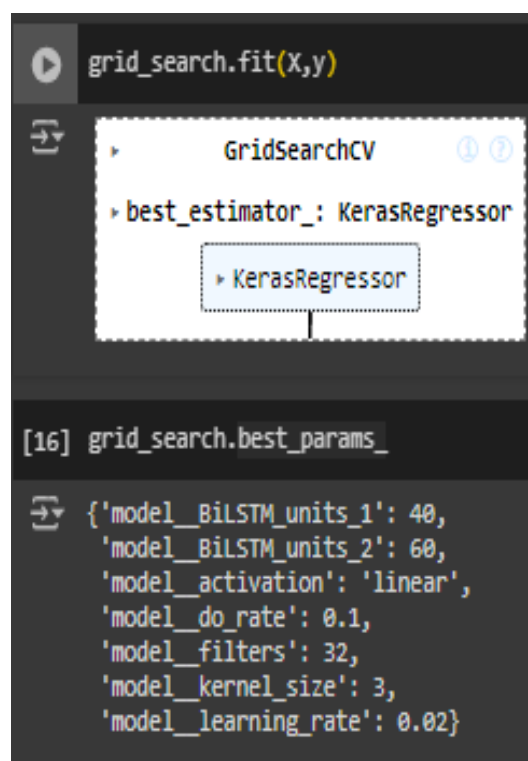
# Define the parameter grid
param_grid = {
    "model_filters" : [16, 32, 64, 96, 128],
    "model_kernel_size" : [1, 2, 3, 4, 5, 6],
    "model_activation" : ["relu", "tanh", "linear", "exponential", "sigmoid", "softmax"],
    "model_BiLSTM_units_1" : [30, 40, 50, 60, 70, 80],
    "model_BiLSTM_units_2" : [30, 40, 50, 60, 70, 80],
    "model_do_rate" : [0.1, 0.2, 0.3, 0.4, 0.5, 0.6],
    "model_learning_rate" : np.linspace(0.01, 0.02, 10)
}

tscv = TimeSeriesSplit(n_splits=2, test_size=3)
model = KerasRegressor(model=create_ConvBiLSTM_model, epochs=500, verbose=0)
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, scoring='neg_mean_squared_error', cv=tscv, n_jobs=-1)

```

FIGURE 4.17 – Le code qui implémente la méthode d’ajustement des hyperparamètres

Pour commencer le processus d’ajustement des hyperparamètres, nous avons appelé la méthode `fit` sur l’objet généré par `GridSearchCV`, comme le montre la figure 4.18, qui utilise les matrices `X` et `y` complètes pour appliquer le processus de validation croisée sur les modèles. Cette méthode génère un objet `GridSearchCV` qui a les meilleurs paramètres accessibles en appelant l’attribut `best_params__` qui donne les hyperparamètres montré dans la figure 4.18.



```

grid_search.fit(X,y)

```

GridSearchCV

- best_estimator_: KerasRegressor
 - KerasRegressor

```

[16] grid_search.best_params_

```

```

{'model_BiLSTM_units_1': 40,
 'model_BiLSTM_units_2': 60,
 'model_activation': 'linear',
 'model_do_rate': 0.1,
 'model_filters': 32,
 'model_kernel_size': 3,
 'model_learning_rate': 0.02}

```

FIGURE 4.18 – Les résultats de GridSearchCV

Après avoir trouvé les meilleurs hyperparamètres, le nombre optimal de pas de temps et également le sous-ensemble optimal de variables explicatives qui donnent les meilleures performances sur les données de validation pour notre modèle, nous pouvons entraîner notre modèle aux données d'entraînement et l'évaluer à l'aide des données de test. La figure 4.17 montre l'architecture finale de notre modèle.

```
[ ] from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import *
from tensorflow.keras.losses import MeanSquaredError
from tensorflow.keras.metrics import RootMeanSquaredError
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint

model = Sequential()
model.add(Conv1D(filters=32, kernel_size=3, activation='linear', input_shape=(12, 6)))
model.add(Bidirectional(LSTM(40, return_sequences=True)))
model.add(Dropout(0.1))
model.add(Bidirectional(LSTM(60)))
model.add(Dense(1))
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 10, 32)	608
bidirectional (Bidirectional)	(None, 10, 80)	23360
dropout (Dropout)	(None, 10, 80)	0
bidirectional_1 (Bidirectional)	(None, 120)	67680
dense (Dense)	(None, 1)	121

Total params: 91769 (358.47 KB)
Trainable params: 91769 (358.47 KB)
Non-trainable params: 0 (0.00 Byte)

FIGURE 4.19 – L'architecture finale de notre modèle

4.4 Evaluation, Résultats et Comparaisons

4.4.1 Résultats

La prochaine étape consistait à entraîner le modèle sur les données d'entraînement et à l'évaluer sur les données de test en utilisant les métriques mentionnées précédemment. Après avoir entraîné le modèle, nous avons enregistré ses résultats dans le tableau 4.1 présenté ci-dessous. Nous avons également tracé sa courbe d'apprentissage sur 100 époques pour mieux comprendre sa stabilité, comme le montre la figure 4.20.

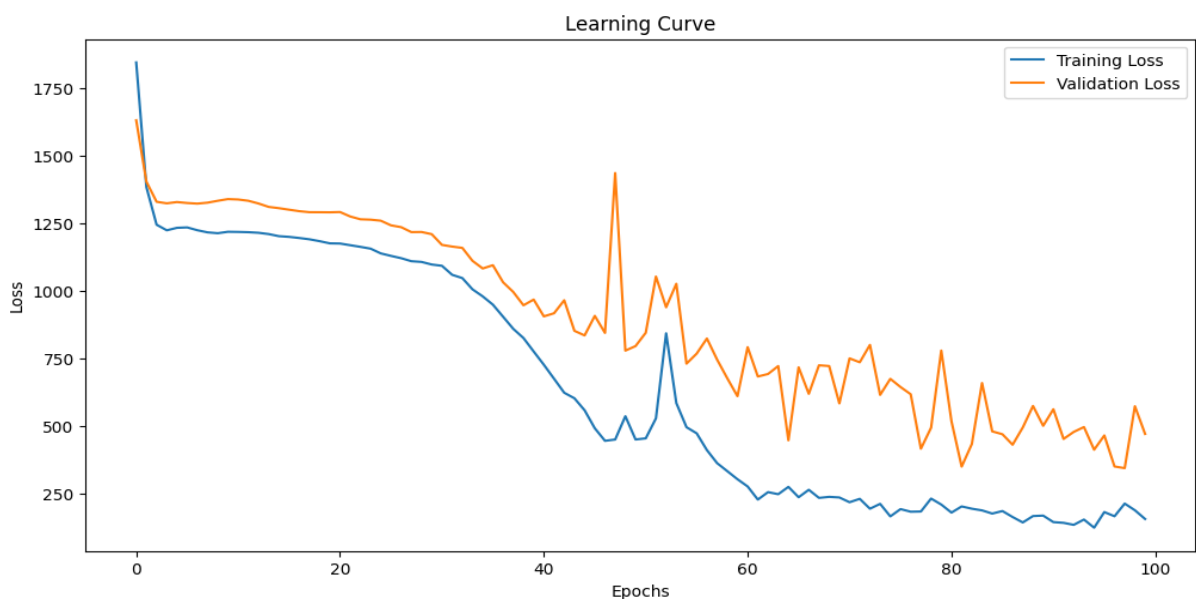


FIGURE 4.20 – La courbe d'apprentissage du modèle

Si nous analysons la courbe d'apprentissage du modèle, nous remarquons des fluctuations au cours de la phase d'apprentissage, à la fois dans la perte d'apprentissage et la perte de validation. Cela peut s'expliquer par le rapport entre la taille des données et le nombre de paramètres à apprendre. En effet, lorsqu'on essaie d'apprendre 10 000 paramètres en utilisant 150 échantillons, il est normal que le modèle ait du mal à trouver le minimum de la fonction objective. Cependant, malgré la taille limitée de nos données, la courbe d'apprentissage montre que le modèle a été capable d'apprendre efficacement à mesure que nous augmentions le nombre d'époques, car la tendance générale de la perte d'entraînement et de la perte de validation est à la baisse.

Métriques d'évaluations	Performance de notre modèle sur les données d'entraînement	Performance de notre modèle sur les données de test
R^2	0.94	0.89
MAE	5.69	9.56
RMSE	8.65	11.99
MAPE	17.93%	28.18%

TABLE 4.1 – Performances du modèle pour différentes métriques sur les données d'entraînement et du test

Les performances du modèle sont très bonnes, à la fois sur les données d'entraînement et les données de test, comme le montre le tableau 4.1. Malgré la taille limitée de nos données, le modèle a pu obtenir d'excellents scores R^2 sur les données d'entraînement et de test, de 0.94 et 0.89 respectivement, tout en obtenant également de faibles scores MAE de 5.69 sur les données d'entraînement et 9.56 sur les données de test, ainsi que de faibles scores RMSE de 8.65 et 11.99 pour les données d'entraînement et de test respectivement. En ce qui concerne les scores MAPE, le modèle a bien performé en obtenant 17.93% et 28.18% sur les ensembles d'entraînement et de test respectivement. Nous avons également tracé les prédictions du modèle sur les ensembles d'entraînement et de test pour mieux comprendre à quel point le modèle s'adapte aux données, obtenant les courbes montrées à la figure 4.21. Les courbes montrent que le modèle s'adapte très bien aux données et qu'il a appris les tendances saisonnières et non saisonnières, ainsi que les relations entre les variables climatiques et les cas de leishmaniose.

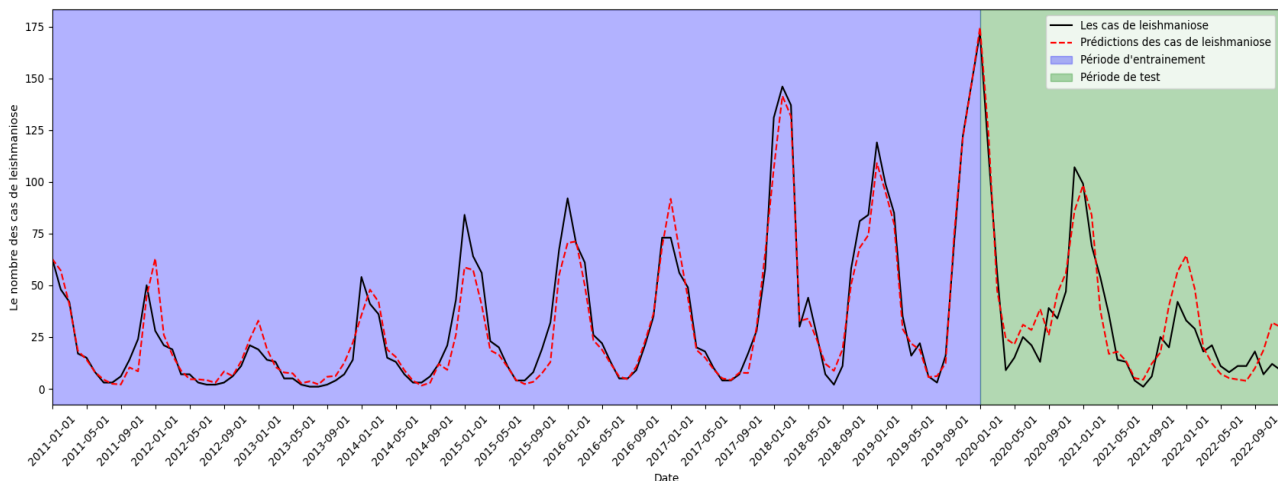


FIGURE 4.21 – Les prédictions de modèle sur les données d'entraînement et du test comparé avec les cas de leishmaniose réels

4.4.2 Comparaison avec d'autres modèles

Pour comparer les performances de notre modèle, nous avons établi plusieurs modèles. Tout d'abord, nous avons essayé un modèle SARIMA qui a la capacité de modéliser les tendances saisonnières et non saisonnières des données, puis nous avons ajusté ses paramètres pour obtenir celui qui performe le mieux sur les données de

validation. Ce modèle a eu des performances médiocres, car il n’a pas réussi à détecter les variations complexes des cas de leishmaniose qui dépendent des variables climatiques. Ensuite, nous avons essayé un modèle polynomial couplé à la méthode de sélection directe pour ne sélectionner que les variables pertinentes, puis nous avons ajusté ses hyperparamètres pour obtenir sa meilleure version. Les performances du modèle polynomial étaient correctes, mais nous avons besoin d’un modèle encore plus puissant pour obtenir un modèle de base solide que nous pouvons utiliser pour valider les bonnes performances de notre modèle. Pour ce faire, nous avons essayé un modèle SVR avec un réglage des hyperparamètres et une sélection directe pour obtenir la version la plus performante sur les données de validation. Ce modèle a très bien performé en raison de la capacité du noyau rbf à capturer des tendances plus complexes dans les données en utilisant des variables climatiques et des dépendances temporelles. Enfin, nous avons essayé un modèle encore plus complexe qui combine à la fois le SVR et le modèle polynomial en utilisant la méthode de stacking avec le modèle de Gradient Boosting CatBoost comme méta-modèle. Ce modèle surpasse à la fois le SVR et le modèle polynomial en raison de sa complexité et de sa capacité à capturer des tendances complexes. Les résultats dans le table 4.2 montrent que notre modèle ConvBiLSTM les surpasse tous sur les données de test.

	R2		RMSE		MAE		MAPE	
	Train	Test	Train	Test	Train	Test	Train	Test
Polynomiale	0.86	0.79	12.16	18.38	8.31	14.8	27.86	39.55
SVR	0.9	0.87	9.99	14.62	4.10	11.52	13.63	30.8
CatBoost(SVR-Poly)	0.99	0.877	0.0002	14.23	0.0002	11.01	0.0006	29.42
ConvBiLSTM	0.94	0.89	8.65	11.99	5.69	9.56	17.93	28.18

TABLE 4.2 – Comparaison entre notre modèle et d’autres modèles

4.5 Explications des prédictions du modèle avec SHAP

L’architecture du modèle proposé peut être un peu complexe, car il est difficile de savoir comment le Conv-BiLSTM utilise les différentes variables climatiques pour faire une prédiction. Cependant, la méthode SHAP nous offre une solution merveilleuse pour surmonter cette complexité et être capable d’expliquer comment notre modèle utilise les différentes variables pour faire ses prédictions, à la fois localement et globalement, à des audiences non techniques et aux décideurs. Pour générer ces explications, nous avons utilisé la méthode *GradientExplainer* fournie par SHAP, qui est mieux adaptée aux modèles d’apprentissage profond complexes comme le nôtre.

4.5.1 Explications locales des prédictions

Le GradientExplainer de SHAP prend deux éléments lorsqu’il génère les explications : il prend d’abord le modèle que nous voulons utiliser pour nos prédictions et un sous-ensemble des données dont nous voulons expliquer les prédictions. La méthode renvoie normalement un tableau de forme (nombre d’observations, nombre de variables), avec le tableau contenant les valeurs de Shapley de chaque variable pour chaque observation.

Dans notre cas, le modèle ConvBiLSTM traite nos données comme une série de séquences pour chaque observation. Par exemple, si nous avons 6 variables et 150 observations, le ConvBiLSTM utilise chaque observation comme une séquence de timesteps (unités de temps), chaque timestep contenant les valeurs des 6 variables. Cela signifie que chaque observation est représentée par une séquence de données sur plusieurs timesteps, plutôt qu’une seule ligne de données. Ainsi, le GradientExplainer renvoie un tableau de forme (nombre d’observations, nombre de timesteps, nombre de variables), nous donnant une valeur de Shapley pour chaque timestep d’une variable pour une observation. Cela nous fournit une explication beaucoup plus détaillée des prédictions du modèle, car nous pouvons examiner l’impact de chaque timestep d’une variable donnée sur la prédiction pour une observation. Tout cela signifie que si nous voulons obtenir une explication d’une prédiction donnée par notre modèle, nous devons passer notre modèle et les données d’entrée utilisées pour obtenir cette prédiction sous la forme d’un tableau de forme (1, 12, 6) au GradientExplainer. Celui-ci renverra un tableau de valeurs de Shapley pour chacun des 12 timesteps de chacune des 6 variables, décrivant leur impact sur cette prédiction. Nous pouvons ensuite utiliser l’un des graphiques fournis par SHAP, le graphique en cascade (waterfall plot), qui nous aide à mieux visualiser les valeurs de Shapley et à les expliquer avec facilité. Il est à noter que dans notre cas, nous devons générer 12 graphiques pour visualiser la contribution de chaque timestep de nos variables à la prédiction.

Pour démontrer cela, nous pouvons prendre l'exemple d'une prédiction unique générée par le modèle, par exemple la prédiction des cas de leishmaniose pour février 2018. Comme nous pouvons le voir dans les figure 4.22 et 4.23, nous avons obtenu 12 graphiques en cascade qui montrent l'impact de chaque timestep des variables climatiques sur notre prédiction.

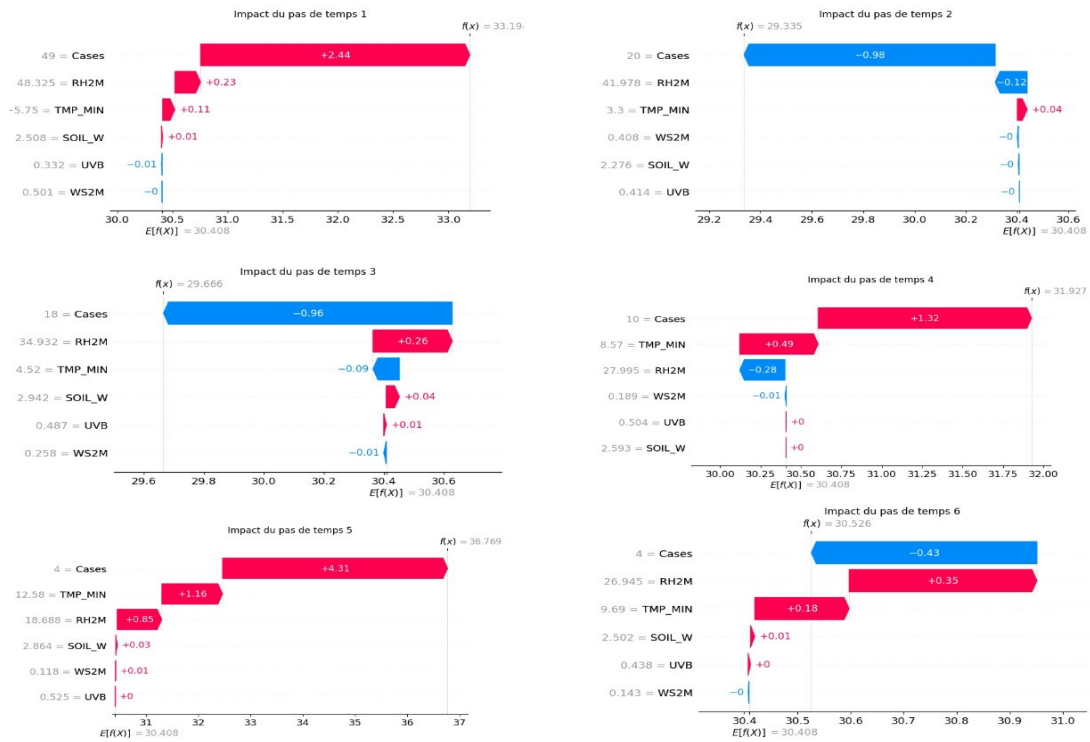


FIGURE 4.22 – Les impacts des pas de temps de chaque variable climatique sur la prédiction (Partie 1)



FIGURE 4.23 – Les impacts des pas de temps de chaque variable climatique sur la prédiction (Partie 2)

Comme nous pouvons le voir dans les graphiques en cascade, nous obtenons un graphique qui montre l'impact de chaque pas de temps de chaque variable sur la prédiction sous la forme de valeurs de Shapley, ainsi que deux valeurs $f(x)$ et $E(f(x))$. La valeur pour $E(f(x))$ représente la prédiction moyenne du modèle sur l'ensemble des données d'entraînement, elle sert essentiellement de valeur de référence qui, lorsqu'elle est ajoutée à chacune des valeurs de Shapley, nous donne la valeur $f(x)$, qui représente la prédiction du modèle dans ce cas. Ainsi, les valeurs de Shapley représentent combien chaque pas de temps de chaque variable ajoute à la valeur de référence. Par exemple, dans le graphique en cascade pour le pas de temps 1, nous pouvons voir que la prédiction est fortement influencée par la valeur de la température minimale, la valeur de l'humidité relative et la valeur des cas de leishmaniose à ce pas de temps, ce qui indique que les valeurs de ces 3 variables au pas de temps 1 étaient les plus importantes pour le modèle pour faire cette prédiction. Nous pouvons expliquer plus en détail l'impact de chaque variable au pas de temps 1 en voyant que certaines augmentent la prédiction tandis que d'autres la diminuent, ce qui signifie que par exemple, lorsque la valeur de la température minimale au pas de temps 1 est faible et qu'elle augmente la prédiction, alors nous pouvons dire que lorsque les valeurs de température minimale sont faibles au pas de temps 1 (ce qui indique la valeur de la température minimale il y a 12 mois), les cas de leishmaniose ont tendance à augmenter pour ce mois.

4.5.2 Explications globales des prédictions

Comme mentionné précédemment, le GradientExplainer nécessite deux entrées : le modèle utilisé pour effectuer les prédictions et un sous-ensemble de données utilisé pour obtenir les prédictions que nous voulons expliquer. Cela signifie que si nous souhaitons obtenir des explications globales, nous devons fournir au GradientExplainer un sous-ensemble de données de notre choix, utilisé pour obtenir les prédictions que nous voulons expliquer, ainsi que le modèle. Le GradientExplainer générera alors un tableau de valeurs de Shapley pour chaque instant temporel et chaque variable pour toutes les observations de ce sous-ensemble. Les explications globales signifient que nous obtiendrons des valeurs de Shapley représentant l'impact global d'une variable sur toutes les prédictions de ce sous-ensemble de données. Notez que l'impact global d'une variable est calculé comme l'impact moyen de tous ses instants temporels sur l'ensemble des prédictions.

Pour démontrer cela, nous avons voulu expliquer les prédictions du modèle pour la période de janvier 2018 à décembre 2022. Nous avons fourni le sous-ensemble de notre matrice de données utilisé pour effectuer les prédictions des cas de leishmaniose pour cette période et notre modèle ConvBiLSTM. Nous avons obtenu un tableau de dimensions (60, 12, 6) contenant toutes les valeurs de Shapley pour les instants temporels pour chaque variable et chaque observation. Ensuite, nous avons utilisé ce tableau pour obtenir nos explications globales en calculant simplement la valeur moyenne de Shapley sur tous les instants temporels pour une variable donnée, pour toutes les observations des données. Pour visualiser les résultats obtenus, nous avons utilisé un simple graphique en barres, présenté dans la figure 4.24 pour représenter l'impact global de chaque variable sur les prédictions générées par le modèle pour la période allant de janvier 2018 à décembre 2022.

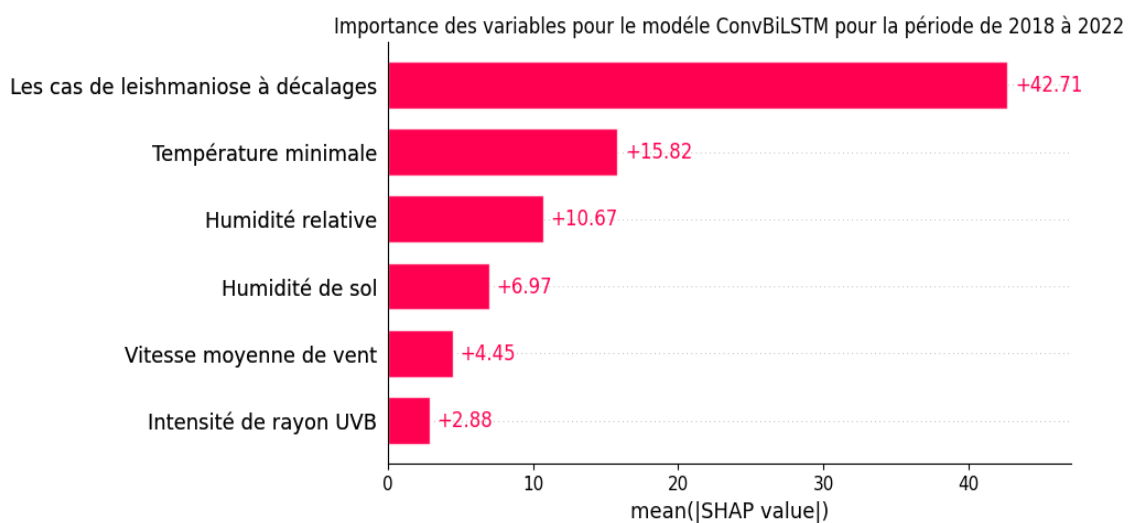


FIGURE 4.24 – Les impacts des variables explicatives sur les prédictions du modèle ConvBiLSTM pour la période allant de janvier 2018 à décembre 2022.

Le graphique en barres nous indique que les variables ayant le plus d'impact sur les prédictions du modèle sont les décalages des cas de leishmaniose (de 1 à 12), la température minimale et l'humidité relative. Pour l'humidité du sol, la vitesse du vent et l'intensité des UVB, l'impact est médiocre mais reste significatif.

Conclusion

L'impact des modèles d'apprentissage profond dans le domaine de la prédiction des épidémies et dans le secteur de la santé a été considérable ces dernières années. Utiliser une méthode hybride qui combine deux des modèles d'apprentissage profond les plus puissants, les CNNs et les BiLSTMs, nous a donné la possibilité de modéliser les incidences d'une maladie aussi complexe que la leishmaniose en utilisant uniquement des variables climatiques et des dépendances temporelles. Dans ce chapitre, nous avons décrit le processus de mise en œuvre de notre solution, en commençant par spécifier les matériaux, les outils de programmation et les bibliothèques utilisés, avant de passer à l'explication de chaque étape du processus de mise en œuvre, depuis la transformation des données au format attendu par notre modèle jusqu'à l'explication de ses prédictions à l'aide de SHAP.

Conclusion générale et perspectives

Dans ce mémoire de master, nous avons abordé le processus de création d'une solution pour la problématique de la prédiction des cas de leishmaniose dans la province de Ouarzazate, qui est connue pour être l'une des maladies les plus complexes à modéliser en raison de la variété des facteurs influençant sa transmission. Au cours de ce travail, nous avons discuté des différentes méthodes et techniques utilisées pour résoudre ce type de problématique et nous avons particulièrement mis en avant la réputation des méthodes d'apprentissage profond, notamment les réseaux de neurones récurrents et leurs variations telles que les LSTM et les BiLSTM dans le domaine de la modélisation épidémiologique, car elles sont considérées comme l'une des approches les plus réussies dans ce domaine. Nous avons également souligné l'impact puissant que peuvent avoir les réseaux de neurones convolutionnels pour détecter et extraire des caractéristiques pertinentes à partir de données complexes.

Pour modéliser les incidences de la leishmaniose à Ouarzazate, nous avons utilisé des données cliniques et climatiques collectées auprès d'un hôpital local à Ouarzazate et de la NASA Power. Ensuite, Nous avons adopté une approche hybride qui combine la puissance des couches convolutionnelles pour extraire des caractéristiques et des informations pertinentes à partir des données de séries temporelles, et la capacité des couches BiLSTM à traiter les séquences temporelles grâce à leurs mémoires à court et long terme, créant ainsi un modèle ConvBiLSTM puissant et précis. Les résultats ont été très bons, confirmant les éloges des chercheurs envers les modèles d'apprentissage profond pour la modélisation des incidences de maladies complexes. Le ConvBiLSTM a surpassé les méthodes statistiques traditionnelles telles que le SARIMA, ainsi que certains modèles de machine learning plus avancés comme les machines à vecteurs de support (SVM) et les modèles de régression polynomiale, et même un modèle hybride combinant les deux avec un métamodèle de GradientBoosting. De plus, nous avons utilisé la méthode SHAP pour simplifier et expliquer nos prédictions, résolvant ainsi les limitations d'interprétabilité de notre modèle hybride complexe.

Au cours de ce projet, nous avons rencontré de nombreux défis qui ont nécessité de la créativité et beaucoup de travail pour les surmonter, tels que la taille limitée de notre ensemble de données et la complexité des transformations de données que nous devons effectuer. Mais avec l'aide d'Allah et grâce à notre travail acharné, nous avons surmonté toutes ces complications pour créer une solution que nous croyons être d'une grande aide pour les décideurs et les autorités sanitaires pour contrôler cette maladie infectieuse.

Pour les perspectives que nous avons concernant notre projet, nous envisageons de créer une application Android qui faciliterait l'utilisation de ce modèle pour informer les décideurs et les responsables du contrôle des maladies infectieuses. Cette application leur permettrait d'entraîner et de générer des prédictions en temps réel en utilisant notre modèle d'un simple clic. Cette application Android fournirait également des explications globales et locales des prédictions du modèle, ce qui faciliterait la compréhension des facteurs qui jouent un rôle dans l'augmentation ou la diminution des cas de leishmaniose pour les utilisateurs. Nous pourrions également entraîner notre modèle pour modéliser les cas de leishmaniose par commune, ce qui fournirait aux responsables du contrôle de la leishmaniose à Ouarzazate beaucoup plus d'informations sur les communes qui sont en danger de propagation de cette maladie.

En conclusion, ce projet m'a fourni une expérience précieuse dans la résolution de problèmes de prédiction de séries temporelles, ce qui m'a permis de perfectionner mes compétences en prétraitement des données, en analyse des données et en modélisation d'apprentissage profond, mais aussi de développer un sens du travail sur une problématique réel de science des données dans le domaine de la prédiction épidémiologique, un domaine qui me passionne beaucoup. Notre approche pour résoudre ce problème pourrait être d'une grande aide pour les chercheurs dans ce domaine, en leur fournissant toutes les techniques et les modèles qui fonctionnent le mieux pour ce type de problème et une preuve que les modèles d'apprentissage profond peuvent obtenir d'excellents résultats même lorsqu'ils sont entraînés avec un ensemble de données relativement limité, et que les cas de leishmaniose peuvent être modélisés uniquement à l'aide de variables climatiques et de dépendances temporelles.

Bibliographie

- [1] Auteur Ailleurs. Titre3. <<http://www.url2.org/>>, 2014. [Online ; accessed 16-January-2014].
- [2] Auteur Autre. Titre2. <<http://www.url1.org/>>, 2014. [Online ; accessed 16-January-2014].
- [3] Auteur Elle. Titre5. <<http://www.url4.org/>>, 2014. [Online ; accessed 16-January-2014].
- [4] Auteur Livre1. *Titre Livre1*. Editeur1, 2014.
- [5] Auteur Lui. Titre4. <<http://www.url3.org/>>, 2014. [Online ; accessed 16-January-2014].
- [6] Auteur Untel. Titre1. <<http://www.url0.org/>>, 2014. [Online ; accessed 16-January-2014].